Ledger

Source code

5 September 2026

76 files

Personal money tracker. Dark analytics UI, RON, local persistence. Snapshot of the app source (src, public, config).

Contents

01 src/components/ledger/add-transaction.tsx
02 src/components/ledger/balance-chart.tsx
03 src/components/ledger/budget-panel.tsx
04 src/components/ledger/category-table.tsx
05 src/components/ledger/chart-tooltip.tsx
06 src/components/ledger/client-only.tsx
07 src/components/ledger/dashboard.tsx
08 src/components/ledger/date-range.tsx
09 src/components/ledger/income-expense.tsx
10 src/components/ledger/income-mix.tsx
11 src/components/ledger/kpi-grid.tsx
12 src/components/ledger/manage-budgets.tsx
13 src/components/ledger/manage-categories.tsx
14 src/components/ledger/manage-payees.tsx
15 src/components/ledger/manage-products.tsx
16 src/components/ledger/manage-recurring.tsx
17 src/components/ledger/payee-picker.tsx
18 src/components/ledger/recurring-panel.tsx
19 src/components/ledger/sparkline.tsx
20 src/components/ledger/spend-mix.tsx
21 src/components/ledger/transactions-table.tsx
22 src/components/preview-host-bridge.tsx
23 src/components/ui/alert-dialog.tsx
24 src/components/ui/badge.tsx
25 src/components/ui/button.tsx
26 src/components/ui/card.tsx
27 src/components/ui/drawer.tsx
28 src/components/ui/dropdown-menu.tsx
29 src/components/ui/input.tsx
30 src/components/ui/label.tsx
31 src/components/ui/separator.tsx
32 src/components/ui/skeleton.tsx
33 src/components/ui/tooltip.tsx
34 src/lib/auth/client.ts
35 src/lib/auth/email-password.ts
36 src/lib/auth/gates.tsx
37 src/lib/auth/isolation.server.ts
38 src/lib/auth/middleware.ts
39 src/lib/auth/pglite-dialect.ts
40 src/lib/auth/popup.server.ts
41 src/lib/auth/preview.ts
42 src/lib/auth/provider.tsx
43 src/lib/auth/providers.ts
44 src/lib/auth/server.ts
45 src/lib/auth/use-current-user.ts
46 src/lib/auth/verify.server.ts
47 src/lib/db.ts
48 src/lib/error-component.tsx
49 src/lib/ledger/analytics.ts
50 src/lib/ledger/categories.ts

```
src/lib/ledger/csv.ts
src/lib/ledger/format.ts
src/lib/ledger/persist.ts
src/lib/ledger/recurrence.ts
src/lib/ledger/schema.ts
src/lib/ledger/store.ts
src/lib/ledger/types.ts
src/lib/multiplayer/index.ts
src/lib/multiplayer/p2p.ts
src/lib/og/site.json
src/lib/preview-embedder-origin.ts
src/lib/preview-host-bridge.ts
src/lib/utils.ts
src/routeTree.gen.ts
src/router.tsx
src/routes/__root.tsx
src/routes/index.tsx
src/styles.css
public/favicon.svg
public/sw.js
package.json
tsconfig.json
vite.config.ts
eslint.config.mjs
scripts/with-app-env.mjs
scripts/migrate.mjs
```

src/components/ledger/add-transaction.tsx

```tsx
import { useEffect, useMemo, useRef, useState } from "react";
import { Plus } from "lucide-react";
import { isoDate } from "@/lib/ledger/format";
import { categoriesFor, labelsEqual, normalizeLabel, payeeNames, productNames } from "@/lib/ledger/categories";
import { useLedger } from "@/lib/ledger/store";
import type { CategoryId, TxType } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import {
  Drawer,
  DrawerContent,
  DrawerDescription,
  DrawerFooter,
  DrawerHeader,
  DrawerTitle,
} from "@/components/ui/drawer";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { cn } from "@/lib/utils";
import { PayeePicker } from "./payee-picker";

type Draft = {
  type: TxType;
  amount: string;
  category: CategoryId;
  date: string;
  payee: string;
  product: string;
  quantity: string;
  note: string;
};

function emptyDraft(date: string): Draft {
  return {
    type: "expense",
    amount: "",
    category: "",
    date,
    payee: "",
    product: "",
    quantity: "1",
    note: "",
  };
}

export function AddTransaction({
  open,
  onOpenChange,
  onSubmit,
  defaultDate,
}: {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  onSubmit: (input: {
    type: TxType;
    amount: number;
    category: CategoryId;
    date: string;
    payee: string;
    note?: string;
    product?: string;
    quantity?: number;
  }) => void;
  defaultDate?: string;
}) {
  const categories = useLedger((s) => s.categories);
  const payees = useLedger((s) => s.payees);
  const products = useLedger((s) => s.products);
  const addCategory = useLedger((s) => s.addCategory);
  const addPayee = useLedger((s) => s.addPayee);
  const deletePayee = useLedger((s) => s.deletePayee);
  const addProduct = useLedger((s) => s.addProduct);
  const deleteProduct = useLedger((s) => s.deleteProduct);
  const today = defaultDate ?? isoDate(new Date());
  const [draft, setDraft] = useState<Draft>(() => emptyDraft(today));
  const [error, setError] = useState<string | null>(null);
  const [newCat, setNewCat] = useState("");
  const [addingCat, setAddingCat] = useState(false);
  const newCatRef = useRef<HTMLInputElement>(null);
```

```

  useEffect(() => {
    if (!addingCat) return;
    const id = window.setTimeout(() => {
      newCatRef.current?.focus();
      newCatRef.current?.scrollIntoView({ block: "center", behavior: "smooth" });
    }, 50);
    return () => window.clearTimeout(id);
  }, [addingCat]);

  const cats = useMemo(
    () => categoriesFor(categories, draft.type),
    [categories, draft.type],
  );
  const payeeList = useMemo(
    () => (draft.category ? payeeNames(payees, draft.type, draft.category) : []),
    [payees, draft.type, draft.category],
  );
  const productList = useMemo(
    () => (draft.category ? productNames(products, draft.category) : []),
    [products, draft.category],
  );

  function reset() {
    setDraft(emptyDraft(today));
    setError(null);
    setNewCat("");
    setAddingCat(false);
  }

  function handleType(type: TxType) {
    const nextCats = categoriesFor(categories, type);
    setDraft((d) => ({
      ...d,
      type,
      category: nextCats.some((c) => c.id === d.category) ? d.category : "",
      payee: "",
      product: "",
      quantity: type === "income" ? d.quantity || "1" : "1",
    }));
    setAddingCat(nextCats.length === 0);
  }

  function pickCategory(id: CategoryId) {
    const names = payeeNames(payees, draft.type, id);
    const items = productNames(products, id);
    setDraft((d) => ({
      ...d,
      category: id,
      payee: names.some((n) => labelsEqual(n, d.payee)) ? d.payee : "",
      product: items.some((n) => labelsEqual(n, d.product)) ? d.product : "",
    }));
  }

  function createCategory() {
    const cleaned = normalizeLabel(newCat);
    if (!cleaned) {
      setError("Name the category.");
      return;
    }
    const existing = cats.find((c) => labelsEqual(c.label, cleaned));
    if (existing) {
      setDraft((d) => ({ ...d, category: existing.id }));
      setNewCat("");
      setAddingCat(false);
      setError(null);
      return;
    }
    const row = addCategory({ label: cleaned, type: draft.type });
    if (!row) {
      setError("Use a unique category name.");
      return;
    }
    setDraft((d) => ({ ...d, category: row.id, payee: "", product: "" }));
    setNewCat("");
    setAddingCat(false);
    setError(null);
  }

  function submit() {
```

```
    const amount = Number(draft.amount.replace(",", "."));
    if (!Number.isFinite(amount) || amount <= 0) {
      setError("Enter an amount greater than zero.");
      return;
    }
    if (!draft.payee.trim()) {
      setError("Pick a payee from your list.");
      return;
    }
    if (!draft.category) {
      setError("Pick or add a category.");
      return;
    }
    if (!/^\d{4}-\d{2}-\d{2}$/.test(draft.date)) {
      setError("Pick a valid date.");
      return;
    }
    let product: string | undefined;
    let quantity: number | undefined;
    if (draft.type === "income") {
      product = draft.product.trim();
      if (!product) {
        setError("Pick a product from your list.");
        return;
      }
      const qtyRaw = draft.quantity.trim() || "1";
      quantity = Number(qtyRaw.replace(",", "."));
      if (!Number.isFinite(quantity) || quantity <= 0) {
        setError("Enter a quantity greater than zero.");
        return;
      }
    }
    onSubmit({
      type: draft.type,
      amount,
      category: draft.category,
      date: draft.date,
      payee: draft.payee.trim(),
      note: draft.note.trim() || undefined,
      product,
      quantity,
    });
    reset();
    onOpenChange(false);
  }

  return (
    <Drawer
      open={open}
      onOpenChange={(next) => {
        onOpenChange(next);
        if (!next) reset();
        if (next) {
          const nextCats = categoriesFor(categories, draft.type);
          setAddingCat(nextCats.length === 0);
        }
      }}
    >
      <DrawerContent>
        <DrawerHeader>
          <DrawerTitle>Add transaction</DrawerTitle>
          <DrawerDescription>Saved on this device only.</DrawerDescription>
        </DrawerHeader>
        <form
          className="flex min-h-0 flex-1 flex-col gap-4 overflow-y-auto px-5"
          onSubmit={(e) => {
            e.preventDefault();
            submit();
          }}
        >
          <div className="flex rounded-xl bg-muted p-1">
            {(["expense", "income"] as const).map((type) => (
              <button
                key={type}
                type="button"
                onClick={() => handleType(type)}
                className={cn(
                  "h-11 min-h-11 flex-1 rounded-lg text-sm font-medium capitalize transition-colors duration-150",
                  draft.type === type
                    ? "bg-card text-foreground shadow-[var(--shadow-border)]"
```

```
                    : "text-muted-foreground",
                )}
              >
                {type}
              </button>
            ))}
          </div>

          <div className="grid gap-1.5">
            <Label htmlFor="amount">Amount (RON)</Label>
            <Input
              id="amount"
              inputMode="decimal"
              placeholder="0.00"
              value={draft.amount}
              onChange={(e) => setDraft((d) => ({ ...d, amount: e.target.value }))}
              className="font-mono tabular-nums"
              autoComplete="off"
            />
          </div>

          <div className="grid gap-1.5">
            <Label>Category</Label>
            {cats.length === 0 && !addingCat ? (
              <p className="text-sm text-muted-foreground">
                No {draft.type} categories yet.
              </p>
            ) : null}
            <div className="flex flex-wrap gap-1.5">
              {cats.map((c) => (
                <button
                  key={c.id}
                  type="button"
                  onClick={() => pickCategory(c.id)}
                  className={cn(
                    "h-9 min-h-9 rounded-full border px-3 text-xs font-medium transition-colors duration-150",
                    draft.category === c.id
                      ? "border-primary bg-primary text-primary-foreground"
                      : "border-border bg-transparent text-muted-foreground hover:text-foreground",
                  )}
                >
                  {c.label}
                </button>
              ))}
              <button
                type="button"
                onClick={() => setAddingCat(true)}
                className="inline-flex h-9 min-h-9 items-center gap-1 rounded-full border border-dashed border-border px-3 text-xs font-medium text-muted-foreground hover:text-foreground"
              >
                <Plus className="size-3.5" />
                New
              </button>
            </div>
            {addingCat ? (
              <div className="flex gap-2 pt-1">
                <Input
                  ref={newCatRef}
                  value={newCat}
                  onChange={(e) => setNewCat(e.target.value)}
                  placeholder={`${draft.type === "income" ? "Salary, freelance…" : "Rent, groceries…"}`}
                  maxLength={40}
                  className="h-12 min-h-12"
                  enterKeyHint="done"
                  autoComplete="off"
                  onKeyDown={(e) => {
                    if (e.key === "Enter") {
                      e.preventDefault();
                      createCategory();
                    }
                    if (e.key === "Escape") setAddingCat(false);
                  }}
                />
                <Button type="button" size="sm" variant="outline" onClick={createCategory}>
                  Add
                </Button>
              </div>
            ) : null}
          </div>

```

```
          <div className="grid gap-1.5">
            <Label>Payee</Label>
            {draft.category ? (
              <PayeePicker
                payees={payeeList}
                value={draft.payee}
                onChange={(name) => setDraft((d) => ({ ...d, payee: name }))}
                onAdd={(name) => addPayee(name, draft.type, draft.category)}
                onDelete={(name) => deletePayee(name, draft.type, draft.category)}
              />
            ) : (
              <p className="text-sm text-muted-foreground">Pick a category first. Payees are per category.</p>
            )}
          </div>

          {draft.type === "income" ? (
            <>
              <div className="grid gap-1.5">
                <Label>Product</Label>
                {draft.category ? (
                  <PayeePicker
                    payees={productList}
                    value={draft.product}
                    onChange={(name) => setDraft((d) => ({ ...d, product: name }))}
                    onAdd={(name) => addProduct(name, draft.category)}
                    onDelete={(name) => deleteProduct(name, draft.category)}
                    noun="product"
                    selectPlaceholder="Select a product"
                  />
                ) : (
                  <p className="text-sm text-muted-foreground">Pick a category first. Products are per income category
    .</p>
                )}
              </div>
              <div className="grid gap-1.5">
                <Label htmlFor="quantity">Quantity</Label>
                <Input
                  id="quantity"
                  inputMode="decimal"
                  placeholder="1"
                  value={draft.quantity}
                  onChange={(e) => setDraft((d) => ({ ...d, quantity: e.target.value }))}
                  className="font-mono tabular-nums"
                  autoComplete="off"
                />
              </div>
            </>
          ) : null}

          <div className="grid gap-1.5">
            <Label htmlFor="date">Date</Label>
            <Input
              id="date"
              type="date"
              value={draft.date}
              onChange={(e) => setDraft((d) => ({ ...d, date: e.target.value }))}
            />
          </div>

          <div className="grid gap-1.5">
            <Label htmlFor="note">Note</Label>
            <Input
              id="note"
              placeholder="Optional"
              value={draft.note}
              onChange={(e) => setDraft((d) => ({ ...d, note: e.target.value }))}
              maxLength={160}
            />
          </div>

          {error ? <p className="text-sm text-expense">{error}</p> : null}
        </form>
        <DrawerFooter>
          <Button type="button" onClick={submit}>
            Save transaction
          </Button>
        </DrawerFooter>
      </DrawerContent>
    </Drawer>
  );
```

```
397  }
```

src/components/ledger/balance-chart.tsx

```tsx
import {
  Area,
  AreaChart,
  CartesianGrid,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis,
} from "recharts";
import type { BalancePoint } from "@/lib/ledger/analytics";
import { formatAxis } from "@/lib/ledger/format";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { ClientOnly } from "./client-only";
import { ChartTooltip } from "./chart-tooltip";
import { Skeleton } from "@/components/ui/skeleton";

export function BalanceChart({
  data,
  focused,
}: {
  data: BalancePoint[];
  focused: boolean;
}) {
  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: "160ms" }}>
      <CardHeader>
        <CardTitle>{focused ? "Focused cumulative" : "Balance trend"}</CardTitle>
      </CardHeader>
      <CardContent>
        {data.every((d) => d.balance === 0) ? (
          <p className="py-10 text-center text-sm text-muted-foreground">
            {focused ? "No activity in this focus." : "No balance yet. Log a transaction to start the trend."}
          </p>
        ) : (
        <div className="h-64 w-full">
          <ClientOnly fallback={<Skeleton className="h-full w-full" />}>
            <ResponsiveContainer width="100%" height="100%">
              <AreaChart data={data} margin={{ top: 8, right: 8, left: 0, bottom: 0 }}>
                <defs>
                  <linearGradient id="balFill" x1="0" y1="0" x2="0" y2="1">
                    <stop offset="0%" stopColor="var(--color-chart-line)" stopOpacity={0.22} />
                    <stop offset="100%" stopColor="var(--color-chart-line)" stopOpacity={0} />
                  </linearGradient>
                </defs>
                <CartesianGrid
                  stroke="var(--color-chart-grid)"
                  vertical={false}
                  strokeDasharray="3 6"
                />
                <XAxis
                  dataKey="label"
                  tick={{ fill: "var(--color-muted-foreground)", fontSize: 11 }}
                  tickLine={false}
                  axisLine={false}
                  minTickGap={28}
                />
                <YAxis
                  tick={{ fill: "var(--color-muted-foreground)", fontSize: 11 }}
                  tickLine={false}
                  axisLine={false}
                  width={36}
                  tickFormatter={(v: number) => formatAxis(v)}
                  domain={[
                    (min: number) => min - Math.max(Math.abs(min) * 0.05, 50),
                    (max: number) => max + Math.max(Math.abs(max) * 0.05, 50),
                  ]}
                />
                <Tooltip
                  cursor={{ stroke: "var(--color-border)" }}
                  content={ChartTooltip}
                />
                <Area
                  type="monotone"
                  dataKey="balance"
                  name="Balance"
                  stroke="var(--color-chart-line)"
                  fill="url(#balFill)"
                  strokeWidth={1.75}
```

```
                  dot={false}
                  activeDot={{ r: 3, strokeWidth: 0, fill: "var(--color-primary)" }}
                />
              </AreaChart>
            </ResponsiveContainer>
          </ClientOnly>
        </div>
        )}
      </CardContent>
    </Card>
  );
}
```

src/components/ledger/budget-panel.tsx

```tsx
import type { BudgetProgress } from "@/lib/ledger/analytics";
import { formatMonthLabel, formatRon } from "@/lib/ledger/format";
import type { CategoryId } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { cn } from "@/lib/utils";

export function BudgetPanel({
  rows,
  month,
  focus,
  onFocus,
  onManage,
}: {
  rows: BudgetProgress[];
  month: Date;
  focus: CategoryId | null;
  onFocus: (id: CategoryId) => void;
  onManage: () => void;
}) {
  const visible = focus ? rows.filter((r) => r.category === focus) : rows;
  const spent = visible.reduce((s, r) => s + r.spent, 0);
  const cap = visible.reduce((s, r) => s + r.monthly, 0);

  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: "140ms" }}>
      <CardHeader>
        <div className="flex items-start justify-between gap-3">
          <div className="min-w-0">
            <CardTitle>Budgets</CardTitle>
            <p className="mt-0.5 text-xs text-muted-foreground">{formatMonthLabel(month)}</p>
          </div>
          <Button type="button" variant="ghost" size="sm" onClick={onManage}>
            Manage
          </Button>
        </div>
      </CardHeader>
      <CardContent className="pt-1">
        {rows.length === 0 ? (
          <div className="flex flex-col items-start gap-3 py-4">
            <p className="text-sm text-muted-foreground">
              Set a monthly cap on an expense category. Spent this month is tracked against it.
            </p>
            <Button type="button" size="sm" variant="outline" onClick={onManage}>
              Add budget
            </Button>
          </div>
        ) : visible.length === 0 ? (
          <p className="py-6 text-center text-sm text-muted-foreground">
            No budget on the focused category.
          </p>
        ) : (
          <ul className="flex flex-col">
            {visible.map((row) => {
              const over = row.remaining < 0;
              const width = Math.min(Math.max(row.ratio * 100, row.spent > 0 ? 2 : 0), 100);
              return (
                <li key={row.id}>
                  <button
                    type="button"
                    onClick={() => onFocus(row.category)}
                    className={cn(
                      "flex min-h-14 w-full flex-col justify-center gap-1.5 rounded-lg px-2 py-2 text-left transition-
colors duration-150 hover:bg-accent",
                      focus === row.category && "bg-accent",
                    )}
                  >
                    <div className="flex items-baseline justify-between gap-3">
                      <span className="flex min-w-0 items-center gap-2 truncate text-sm">
                        <span
                          className="size-2 shrink-0 rounded-full"
                          style={{ background: row.color }}
                        />
                        <span className="truncate">{row.label}</span>
                      </span>
                      <span className="font-mono text-xs tabular-nums text-foreground">
                        {formatRon(row.spent)}
                        <span className="text-subtle"> / {formatRon(row.monthly)}</span>
```

```
                      </span>
                    </div>
                    <div className="flex items-center gap-2">
                      <div className="h-1.5 flex-1 overflow-hidden rounded-full bg-muted">
                        <div
                          className={cn("h-full rounded-full", over ? "bg-expense" : "bg-primary")}
                          style={{ width: `${width}%` }}
                        />
                      </div>
                      <span
                        className={cn(
                          "w-12 text-right font-mono text-xs tabular-nums",
                          over ? "text-expense" : "text-subtle",
                        )}
                      >
                        {(row.ratio * 100).toFixed(0)}%
                      </span>
                    </div>
                  </button>
                </li>
              );
            })}
          </ul>
        )}
        {visible.length > 1 ? (
          <p className="mt-3 font-mono text-xs tabular-nums text-muted-foreground">
            {formatRon(spent)} of {formatRon(cap)} this month
          </p>
        ) : null}
      </CardContent>
    </Card>
  );
}
```

src/components/ledger/category-table.tsx

```tsx
import type { MixSlice } from "@/lib/ledger/analytics";
import { formatRon } from "@/lib/ledger/format";
import type { CategoryId } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { cn } from "@/lib/utils";

export function CategoryTable({
  rows,
  focus,
  onFocus,
  onManage,
}: {
  rows: MixSlice[];
  focus: CategoryId | null;
  onFocus: (id: CategoryId) => void;
  onManage: () => void;
}) {
  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: "280ms" }}>
      <CardHeader>
        <div className="flex items-center justify-between gap-3">
          <CardTitle>Categories</CardTitle>
          <Button type="button" variant="ghost" size="sm" onClick={onManage}>
            Manage
          </Button>
        </div>
      </CardHeader>
      <CardContent className="pt-1">
        {rows.length === 0 ? (
          <p className="py-6 text-center text-sm text-muted-foreground">
            Nothing in this range. Add a category, then log a transaction.
          </p>
        ) : (
          <ul className="flex flex-col">
            {rows.map((row) => {
              const max = rows[0]?.value || 1;
              return (
                <li key={row.category}>
                  <button
                    type="button"
                    onClick={() => onFocus(row.category)}
                    className={cn(
                      "flex min-h-14 w-full flex-col justify-center gap-1.5 rounded-lg px-2 py-2 text-left transition-colors duration-150 hover:bg-accent",
                      focus === row.category && "bg-accent",
                    )}
                  >
                    <div className="flex items-baseline justify-between gap-3">
                      <span className="flex min-w-0 items-center gap-2 truncate text-sm">
                        <span
                          className="size-2 shrink-0 rounded-full"
                          style={{ background: row.color }}
                        />
                        <span className="truncate">{row.label}</span>
                      </span>
                      <span
                        className={cn(
                          "font-mono text-xs tabular-nums",
                          row.type === "income" ? "text-income" : "text-foreground",
                        )}
                      >
                        {row.type === "income" ? "+" : ""}
                        {formatRon(row.value)}
                      </span>
                    </div>
                    <div className="flex items-center gap-2">
                      <div className="h-1.5 flex-1 overflow-hidden rounded-full bg-muted">
                        <div
                          className="h-full rounded-full bg-primary"
                          style={{ width: `${Math.max((row.value / max) * 100, 2)}%` }}
                        />
                      </div>
                      <span className="w-8 text-right font-mono text-xs tabular-nums text-subtle">
                        {(row.share * 100).toFixed(0)}%
                      </span>
                    </div>
                  </button>
```

```
                </li>
              );
            })}
          </ul>
        )}
      </CardContent>
    </Card>
  );
}
```

src/components/ledger/chart-tooltip.tsx

```tsx
import { formatRon } from "@/lib/ledger/format";

type PayloadItem = {
  name?: string;
  value?: number | string;
  color?: string;
  dataKey?: string | number;
};

export function ChartTooltip({
  active,
  payload,
  label,
  valueFormatter = formatRon,
}: {
  active?: boolean;
  payload?: PayloadItem[];
  label?: string;
  valueFormatter?: (n: number) => string;
}) {
  if (!active || !payload?.length) return null;
  return (
    <div className="rounded-lg bg-popover px-3 py-2 text-xs shadow-[var(--shadow-border)]">
      {label ? <p className="mb-1 text-muted-foreground">{label}</p> : null}
      <ul className="space-y-0.5">
        {payload.map((item) => {
          const n = typeof item.value === "number" ? item.value : Number(item.value);
          return (
            <li
              key={String(item.dataKey ?? item.name)}
              className="flex items-center justify-between gap-6 tabular-nums text-foreground"
            >
              <span className="flex items-center gap-1.5">
                <span
                  className="size-1.5 rounded-full"
                  style={{ background: item.color ?? "var(--color-primary)" }}
                />
                {item.name}
              </span>
              <span className="font-mono">{Number.isFinite(n) ? valueFormatter(n) : "—"}</span>
            </li>
          );
        })}
      </ul>
    </div>
  );
}
```

src/components/ledger/client-only.tsx

```tsx
import { useEffect, useState, type ReactNode } from "react";

export function ClientOnly({
  children,
  fallback = null,
}: {
  children: ReactNode;
  fallback?: ReactNode;
}) {
  const [mounted, setMounted] = useState(false);
  useEffect(() => {
    setMounted(true);
  }, []);
  if (!mounted) return fallback;
  return children;
}
```

src/components/ledger/dashboard.tsx

```tsx
import { useEffect, useLayoutEffect, useMemo, useRef, useState, lazy, Suspense } from "react";
import { Check, Download, FileSpreadsheet, FileUp, MoreHorizontal, Package, Plus, Repeat, Smartphone, Tag, Upload, Users, Wallet } from "lucide-react";
import { toast } from "sonner";
import {
  balanceSeries,
  budgetProgress,
  categoryRows,
  filterFocused,
  incomeExpenseBars,
  incomePayeeMix,
  inWindow,
  kpisFor,
  spendMix,
  windowFor,
  type LedgerFocus,
} from "@/lib/ledger/analytics";
import { categoryLabel, labelsEqual } from "@/lib/ledger/categories";
import { exportTransactionsCsv, readCsvFile } from "@/lib/ledger/csv";
import { isoDate } from "@/lib/ledger/format";
import { splitRecurrences } from "@/lib/ledger/recurrence";
import { useLedger } from "@/lib/ledger/store";
import type { CategoryId } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { Skeleton } from "@/components/ui/skeleton";
import { BudgetPanel } from "./budget-panel";
import { CategoryTable } from "./category-table";
import { DateRange } from "./date-range";
import { KpiGrid } from "./kpi-grid";
import { RecurringPanel } from "./recurring-panel";
import { TransactionsTable } from "./transactions-table";

const AddTransaction = lazy(() =>
  import("./add-transaction").then((m) => ({ default: m.AddTransaction })),
);
const ManageCategories = lazy(() =>
  import("./manage-categories").then((m) => ({ default: m.ManageCategories })),
);
const ManagePayees = lazy(() =>
  import("./manage-payees").then((m) => ({ default: m.ManagePayees })),
);
const ManageProducts = lazy(() =>
  import("./manage-products").then((m) => ({ default: m.ManageProducts })),
);
const ManageBudgets = lazy(() =>
  import("./manage-budgets").then((m) => ({ default: m.ManageBudgets })),
);
const ManageRecurring = lazy(() =>
  import("./manage-recurring").then((m) => ({ default: m.ManageRecurring })),
);

const BalanceChart = lazy(() =>
  import("./balance-chart").then((m) => ({ default: m.BalanceChart })),
);
const SpendMix = lazy(() => import("./spend-mix").then((m) => ({ default: m.SpendMix })));
const IncomeMix = lazy(() => import("./income-mix").then((m) => ({ default: m.IncomeMix })));
const IncomeExpense = lazy(() =>
  import("./income-expense").then((m) => ({ default: m.IncomeExpense })),
);
```

```

function LogoMark() {
  return (
    <svg viewBox="0 0 24 24" className="size-6 text-primary" aria-hidden="true">
      <rect x="3" y="14" width="4" height="7" rx="1" fill="currentColor" opacity="0.55" />
      <rect x="10" y="8" width="4" height="13" rx="1" fill="currentColor" />
      <rect x="17" y="4" width="4" height="17" rx="1" fill="currentColor" opacity="0.7" />
    </svg>
  );
}

function ChartFallback({ className = "h-80" }: { className?: string }) {
  return <Skeleton className={`rounded-2xl ${className}`} />;
}

function EmptyPanel({
  title,
  message,
  delay,
}: {
  title: string;
  message: string;
  delay: string;
}) {
  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: delay }}>
      <CardHeader>
        <CardTitle>{title}</CardTitle>
      </CardHeader>
      <CardContent>
        <p className="py-10 text-center text-sm text-muted-foreground">{message}</p>
      </CardContent>
    </Card>
  );
}

export function Dashboard({ nowIso }: { nowIso: string }) {
  const hydrate = useLedger((s) => s.hydrate);
  const hydrated = useLedger((s) => s.hydrated);
  const transactions = useLedger((s) => s.transactions);
  const categories = useLedger((s) => s.categories);
  const payees = useLedger((s) => s.payees);
  const products = useLedger((s) => s.products);
  const budgets = useLedger((s) => s.budgets);
  const recurrences = useLedger((s) => s.recurrences);
  const range = useLedger((s) => s.range);
  const setRange = useLedger((s) => s.setRange);
  const persistOk = useLedger((s) => s.persistOk);
  const addTransaction = useLedger((s) => s.addTransaction);
  const deleteTransaction = useLedger((s) => s.deleteTransaction);
  const postRecurrence = useLedger((s) => s.postRecurrence);
  const skipRecurrence = useLedger((s) => s.skipRecurrence);
  const exportBackup = useLedger((s) => s.exportBackup);
  const importBackup = useLedger((s) => s.importBackup);
  const importCsv = useLedger((s) => s.importCsv);
  const clearAll = useLedger((s) => s.clearAll);

  const [focus, setFocus] = useState<LedgerFocus | null>(null);
  const [addOpen, setAddOpen] = useState(false);
  const [catsOpen, setCatsOpen] = useState(false);
  const [payeesOpen, setPayeesOpen] = useState(false);
  const [productsOpen, setProductsOpen] = useState(false);
  const [budgetsOpen, setBudgetsOpen] = useState(false);
  const [recurringOpen, setRecurringOpen] = useState(false);
  const [clearOpen, setClearOpen] = useState(false);
  const [csvConfirm, setCsvConfirm] = useState<{ count: number; truncated: boolean } | null>(null);
  const fileRef = useRef<HTMLInputElement>(null);
  const csvRef = useRef<HTMLInputElement>(null);
  const csvFileRef = useRef<File | null>(null);

  useLayoutEffect(() => {
    void (async () => {
      const posted = await hydrate();
      if (posted.length === 1) {
        toast.success(`Posted ${posted[0]!.payee}`);
      } else if (posted.length > 1) {
        toast.success(`Posted ${posted.length} recurring items`);
      }
    })();
  }, [hydrate]);
```

```

  useEffect(() => {
    if (!("serviceWorker" in navigator)) return;
    const boot = () => {
      void navigator.serviceWorker.register("/sw.js").catch(() => undefined);
    };
    if (document.readyState === "complete") boot();
    else window.addEventListener("load", boot, { once: true });
  }, []);

  useEffect(() => {
    if (focus?.kind === "category" && !categories.some((c) => c.id === focus.id)) {
      setFocus(null);
    }
    if (
      focus?.kind === "payee" &&
      !payees.some((p) => labelsEqual(p.name, focus.name)) &&
      !transactions.some((t) => labelsEqual(t.payee, focus.name))
    ) {
      setFocus(null);
    }
  }, [focus, categories, payees, transactions]);

  const model = useMemo(() => {
    const win = windowFor(range);
    const scoped = filterFocused(transactions, focus);
    const inRange = scoped.filter((t) => inWindow(t, win.start, win.end));
    const now = new Date(nowIso);
    return {
      win,
      now,
      kpis: kpisFor(scoped, win),
      mix: spendMix(transactions, win, categories),
      incomeMix: incomePayeeMix(transactions, win),
      cats: categoryRows(scoped, win, categories),
      bars: incomeExpenseBars(scoped, win, range),
      balance: balanceSeries(scoped, win, Boolean(focus)),
      table: inRange,
      budgets: budgetProgress(budgets, transactions, categories, now),
      recurring: splitRecurrences(recurrences, isoDate(now)),
    };
  }, [transactions, range, focus, categories, budgets, recurrences, nowIso]);

  const categoryFocus = focus?.kind === "category" ? focus.id : null;
  const payeeFocus = focus?.kind === "payee" ? focus.name : null;

  function toggleFocus(id: CategoryId) {
    setFocus((prev) => (prev?.kind === "category" && prev.id === id ? null : { kind: "category", id }));
  }

  function togglePayee(name: string) {
    setFocus((prev) =>
      prev?.kind === "payee" && labelsEqual(prev.name, name) ? null : { kind: "payee", name },
    );
  }

  async function onImport(file: File) {
    try {
      await importBackup(file);
      toast.success("Backup restored");
    } catch (err) {
      toast.error(err instanceof Error ? err.message : "Import failed");
    }
  }

  async function applyCsv(file: File) {
    try {
      const result = await importCsv(file);
      const extra = result.truncated ? " First 5000 rows used." : "";
      toast.success(`Loaded ${result.imported} transactions from spreadsheet.${extra}`);
    } catch (err) {
      toast.error(err instanceof Error ? err.message : "CSV import failed");
    }
  }

  async function onPickCsv(file: File) {
    try {
      const preview = await readCsvFile(file);
      if (transactions.length === 0) {
        await applyCsv(file);
```

```
        return;
      }
      csvFileRef.current = file;
      setCsvConfirm({ count: preview.transactions.length, truncated: preview.truncated });
    } catch (err) {
      toast.error(err instanceof Error ? err.message : "CSV import failed");
    }
  }

  async function onExportCsv() {
    const result = await exportTransactionsCsv(transactions, categories);
    if (result === "empty") toast.error("No transactions to export");
    else if (result === "shared") toast.success("Share the CSV — open in Numbers");
    else if (result === "downloaded") toast.success("CSV downloaded — open in Numbers");
  }

  const isEmpty = transactions.length === 0;
  const hasAny =
    !isEmpty ||
    categories.length > 0 ||
    payees.length > 0 ||
    products.length > 0 ||
    budgets.length > 0 ||
    recurrences.length > 0;

  return (
    <div
      className="min-h-dvh bg-background pb-32 text-foreground md:pb-10"
      data-hydrated={hydrated ? "true" : "false"}
    >
      <header className="sticky top-0 z-30 border-b border-border bg-background">
        <div className="mx-auto flex max-w-6xl items-center gap-3 px-4 py-3 sm:px-6">
          <div className="flex min-w-0 items-center gap-2.5">
            <LogoMark />
            <div className="min-w-0">
              <p className="text-sm font-semibold tracking-widest uppercase">Ledger</p>
              <p className="flex items-center gap-1 text-xs text-subtle">
                {persistOk ? (
                  <>
                    <Check className="size-3" />
                    Saved on this device
                  </>
                ) : (
                  "Storage blocked — download a backup"
                )}
              </p>
            </div>
          </div>
          <div className="ml-auto hidden min-w-0 flex-1 justify-center md:flex">
            <div className="w-full max-w-md">
              <DateRange value={range} onChange={setRange} />
            </div>
          </div>
          <div className="ml-auto flex items-center gap-1 md:ml-0">
            <Button
              type="button"
              size="sm"
              className="hidden md:inline-flex"
              onClick={() => setAddOpen(true)}
            >
              <Plus />
              Add
            </Button>
            <DropdownMenu>
              <DropdownMenuTrigger asChild>
                <Button type="button" variant="ghost" size="icon" aria-label="More">
                  <MoreHorizontal />
                </Button>
              </DropdownMenuTrigger>
              <DropdownMenuContent align="end">
                <DropdownMenuLabel>Lists</DropdownMenuLabel>
                <DropdownMenuItem onSelect={() => setCatsOpen(true)}>
                  <Tag />
                  Categories
                </DropdownMenuItem>
                <DropdownMenuItem onSelect={() => setPayeesOpen(true)}>
                  <Users />
                  Payees
                </DropdownMenuItem>
                <DropdownMenuItem onSelect={() => setProductsOpen(true)}>
```

```
                  <Package />
                  Products
                </DropdownMenuItem>
                <DropdownMenuItem onSelect={() => setBudgetsOpen(true)}>
                  <Wallet />
                  Budgets
                </DropdownMenuItem>
                <DropdownMenuItem onSelect={() => setRecurringOpen(true)}>
                  <Repeat />
                  Recurring
                </DropdownMenuItem>
                <DropdownMenuSeparator />
                <DropdownMenuItem
                  onSelect={() => {
                    window.location.assign("/?install=1&platform=ios");
                  }}
                >
                  <Smartphone />
                  Add to iPhone
                </DropdownMenuItem>
                <DropdownMenuSeparator />
                <DropdownMenuLabel>Data</DropdownMenuLabel>
                <DropdownMenuItem
                  onSelect={() => {
                    void onExportCsv();
                  }}
                  disabled={isEmpty}
                >
                  <FileSpreadsheet />
                  Export CSV
                </DropdownMenuItem>
                <DropdownMenuItem onSelect={() => csvRef.current?.click()}>
                  <FileUp />
                  Import CSV
                </DropdownMenuItem>
                <DropdownMenuItem
                  onSelect={() => {
                    exportBackup();
                    toast.success("Backup downloaded");
                  }}
                >
                  <Download />
                  Download backup
                </DropdownMenuItem>
                <DropdownMenuItem onSelect={() => fileRef.current?.click()}>
                  <Upload />
                  Import backup
                </DropdownMenuItem>
                <DropdownMenuSeparator />
                <DropdownMenuItem
                  variant="destructive"
                  onSelect={() => setClearOpen(true)}
                  disabled={!hasAny}
                >
                  Clear all data
                </DropdownMenuItem>
              </DropdownMenuContent>
            </DropdownMenu>
            <input
              ref={fileRef}
              type="file"
              accept="application/json,.json"
              className="sr-only"
              onChange={(e) => {
                const file = e.target.files?.[0];
                e.target.value = "";
                if (file) void onImport(file);
              }}
            />
            <input
              ref={csvRef}
              type="file"
              accept=".csv,text/csv,text/plain,text/comma-separated-values"
              className="sr-only"
              onChange={(e) => {
                const file = e.target.files?.[0];
                e.target.value = "";
                if (file) void onPickCsv(file);
              }}
            />
```

```
          </div>
        </div>
        <div className="border-t border-border px-4 py-2 md:hidden">
          <DateRange value={range} onChange={setRange} />
        </div>
      </header>

      <main className="mx-auto flex max-w-6xl flex-col gap-4 px-4 py-4 sm:px-6 sm:py-6">
        {isEmpty ? (
          <section className="stagger-in rounded-2xl bg-card px-5 py-6 shadow-[var(--shadow-border)]">
            <p className="text-sm font-medium">Your ledger is empty</p>
            <p className="mt-1 text-sm text-muted-foreground">
              Add categories, payees, and products, set monthly budgets, then log a transaction or a
              repeating bill — or load a spreadsheet. Everything stays on this device in RON.
            </p>
            <div className="mt-4 flex flex-wrap gap-2">
              <Button type="button" size="sm" variant="outline" onClick={() => setCatsOpen(true)}>
                <Tag />
                Categories
              </Button>
              <Button type="button" size="sm" variant="outline" onClick={() => setPayeesOpen(true)}>
                <Users />
                Payees
              </Button>
              <Button type="button" size="sm" variant="outline" onClick={() => setProductsOpen(true)}>
                <Package />
                Products
              </Button>
              <Button type="button" size="sm" variant="outline" onClick={() => setBudgetsOpen(true)}>
                <Wallet />
                Budgets
              </Button>
              <Button type="button" size="sm" variant="outline" onClick={() => setRecurringOpen(true)}>
                <Repeat />
                Recurring
              </Button>
              <Button type="button" size="sm" variant="outline" onClick={() => csvRef.current?.click()}>
                <FileUp />
                Import CSV
              </Button>
              <Button
                type="button"
                size="sm"
                variant="outline"
                onClick={() => window.location.assign("/?install=1&platform=ios")}
              >
                <Smartphone />
                Add to iPhone
              </Button>
              <Button type="button" size="sm" onClick={() => setAddOpen(true)}>
                <Plus />
                Add transaction
              </Button>
            </div>
          </section>
        ) : null}

        {focus ? (
          <div className="flex min-w-0 items-center">
            <button
              type="button"
              onClick={() => setFocus(null)}
              className="inline-flex h-9 min-h-9 max-w-full items-center gap-2 rounded-full border border-border bg-ca
    rd px-3 text-xs text-foreground"
            >
              <span className="min-w-0 truncate">
                Focus: {focus.kind === "category" ? categoryLabel(focus.id, categories) : focus.name}
              </span>
              <span className="shrink-0 text-subtle">Clear</span>
            </button>
          </div>
        ) : null}

        <KpiGrid
          net={model.kpis.net}
          income={model.kpis.income}
          expenses={model.kpis.expenses}
          savingsRate={model.kpis.savingsRate}
        />

```

```
        <div className="grid min-w-0 gap-4 lg:grid-cols-2">
          <BudgetPanel
            rows={model.budgets}
            month={model.now}
            focus={categoryFocus}
            onFocus={toggleFocus}
            onManage={() => setBudgetsOpen(true)}
          />
          <RecurringPanel
            due={model.recurring.due}
            upcoming={model.recurring.upcoming}
            paused={model.recurring.paused}
            categories={categories}
            onManage={() => setRecurringOpen(true)}
            onPost={(id) => {
              const posted = postRecurrence(id);
              toast.success(
                posted.length === 1
                  ? `Posted ${posted[0]!.payee}`
                  : posted.length > 1
                    ? `Posted ${posted.length} items`
                    : "Nothing due",
              );
            }}
            onSkip={(id) => {
              skipRecurrence(id);
              toast.success("Skipped this period");
            }}
          />
        </div>

        <div className="grid min-w-0 gap-4 lg:grid-cols-3">
          <div className="min-w-0 lg:col-span-2">
            {isEmpty ? (
              <EmptyPanel
                title="Balance trend"
                message="No balance yet. Log a transaction to start the trend."
                delay="160ms"
              />
            ) : (
              <Suspense fallback={<ChartFallback />}>
                <BalanceChart data={model.balance} focused={Boolean(focus)} />
              </Suspense>
            )}
          </div>
          {isEmpty ? (
            <EmptyPanel
              title="Spend mix"
              message="No expenses in this range. Add an expense to see the mix."
              delay="200ms"
            />
          ) : (
            <Suspense fallback={<ChartFallback className="h-80" />}>
              <SpendMix data={model.mix} focus={categoryFocus} onFocus={toggleFocus} />
            </Suspense>
          )}
        </div>

        <div className="grid min-w-0 gap-4 lg:grid-cols-3">
          {isEmpty ? (
            <EmptyPanel
              title="Income by payee"
              message="No income in this range. Add income to see who paid you."
              delay="220ms"
            />
          ) : (
            <Suspense fallback={<ChartFallback className="h-80" />}>
              <IncomeMix data={model.incomeMix} focus={payeeFocus} onFocus={togglePayee} />
            </Suspense>
          )}
          <div className="min-w-0 lg:col-span-2">
            {isEmpty ? (
              <EmptyPanel
                title="Income vs expenses"
                message="No income or expenses in this range."
                delay="240ms"
              />
            ) : (
              <Suspense fallback={<ChartFallback />}>
                <IncomeExpense data={model.bars} />
```

```
              </Suspense>
            )}
          </div>
        </div>

        <div className="grid min-w-0 gap-4 lg:grid-cols-5">
          <div className="min-w-0 lg:col-span-2">
            <CategoryTable
              rows={model.cats}
              focus={categoryFocus}
              onFocus={toggleFocus}
              onManage={() => setCatsOpen(true)}
            />
          </div>
          <div className="min-w-0 lg:col-span-3">
            <TransactionsTable
              transactions={model.table}
              categories={categories}
              emptyHint="No transactions yet."
              canExport={!isEmpty}
              onExportCsv={() => {
                void onExportCsv();
              }}
              onImportCsv={() => csvRef.current?.click()}
              onDelete={(id) => {
                deleteTransaction(id);
                toast.success("Transaction deleted");
              }}
            />
          </div>
        </div>
      </main>

      <div className="fixed inset-x-0 bottom-0 z-20 border-t border-border bg-background p-3 safe-bottom md:hidden">
        <Button type="button" className="w-full" onClick={() => setAddOpen(true)}>
          <Plus />
          Add transaction
        </Button>
      </div>

      <Suspense fallback={null}>
        <AddTransaction
          open={addOpen}
          onOpenChange={setAddOpen}
          defaultDate={isoDate(new Date(nowIso))}
          onSubmit={(input) => {
            addTransaction(input);
            toast.success("Transaction saved");
          }}
        />
      </Suspense>

      <Suspense fallback={null}>
        <ManageCategories open={catsOpen} onOpenChange={setCatsOpen} />
        <ManagePayees open={payeesOpen} onOpenChange={setPayeesOpen} />
        <ManageProducts open={productsOpen} onOpenChange={setProductsOpen} />
        <ManageBudgets open={budgetsOpen} onOpenChange={setBudgetsOpen} />
        <ManageRecurring
          open={recurringOpen}
          onOpenChange={setRecurringOpen}
          defaultDate={isoDate(new Date(nowIso))}
        />
      </Suspense>

      <AlertDialog open={clearOpen} onOpenChange={setClearOpen}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>Clear all data?</AlertDialogTitle>
            <AlertDialogDescription>
              This removes every transaction, category, payee, product, budget, and recurring bill stored
              on this device. Download a backup first if you want to keep your numbers.
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Cancel</AlertDialogCancel>
            <AlertDialogAction
              className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
              onClick={() => {
                clearAll();
                setFocus(null);
```

```
                toast.success("Ledger cleared");
              }}
            >
              Clear
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>

      <AlertDialog
        open={csvConfirm !== null}
        onOpenChange={(open) => {
          if (!open) {
            setCsvConfirm(null);
            csvFileRef.current = null;
          }
        }}
      >
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>Load this spreadsheet?</AlertDialogTitle>
            <AlertDialogDescription>
              Ledger will show only the {csvConfirm?.count ?? 0} transactions from the file.
              Current transactions are replaced — like swapping a disc.
              {csvConfirm?.truncated ? " Only the first 5000 rows will be used." : ""}
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Cancel</AlertDialogCancel>
            <AlertDialogAction
              onClick={() => {
                const file = csvFileRef.current;
                csvFileRef.current = null;
                setCsvConfirm(null);
                if (file) void applyCsv(file);
              }}
            >
              Load spreadsheet
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </div>
  );
}
```

src/components/ledger/date-range.tsx

```tsx
import { RANGE_LABELS } from "@/lib/ledger/analytics";
import { RANGE_PRESETS, type RangePreset } from "@/lib/ledger/types";
import { cn } from "@/lib/utils";

export function DateRange({
  value,
  onChange,
}: {
  value: RangePreset;
  onChange: (next: RangePreset) => void;
}) {
  return (
    <div
      role="tablist"
      aria-label="Date range"
      className="flex rounded-xl bg-muted p-1"
    >
      {RANGE_PRESETS.map((preset) => {
        const active = preset === value;
        return (
          <button
            key={preset}
            type="button"
            role="tab"
            aria-selected={active}
            onClick={() => onChange(preset)}
            className={cn(
              "h-9 min-h-9 min-w-11 flex-1 rounded-lg px-2.5 font-mono text-xs font-medium transition-[color,backgroun
d-color,box-shadow] duration-150",
              active
                ? "bg-card text-foreground shadow-[var(--shadow-border)]"
                : "text-muted-foreground hover:text-foreground",
            )}
          >
            {RANGE_LABELS[preset]}
          </button>
        );
      })}
    </div>
  );
}
```

src/components/ledger/income-expense.tsx

```tsx
import {
  Bar,
  BarChart,
  CartesianGrid,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis,
} from "recharts";
import type { BarPoint } from "@/lib/ledger/analytics";
import { formatAxis } from "@/lib/ledger/format";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Skeleton } from "@/components/ui/skeleton";
import { ClientOnly } from "./client-only";
import { ChartTooltip } from "./chart-tooltip";

export function IncomeExpense({ data }: { data: BarPoint[] }) {
  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: "240ms" }}>
      <CardHeader>
        <CardTitle>Income vs expenses</CardTitle>
      </CardHeader>
      <CardContent>
        {data.every((d) => d.income === 0 && d.expense === 0) ? (
          <p className="py-10 text-center text-sm text-muted-foreground">
            No income or expenses in this range.
          </p>
        ) : (
        <div className="h-64 w-full">
          <ClientOnly fallback={<Skeleton className="h-full w-full" />}>
            <ResponsiveContainer width="100%" height="100%">
              <BarChart data={data} margin={{ top: 8, right: 8, left: 0, bottom: 0 }}>
                <CartesianGrid
                  stroke="var(--color-chart-grid)"
                  vertical={false}
                  strokeDasharray="3 6"
                />
                <XAxis
                  dataKey="label"
                  tick={{ fill: "var(--color-muted-foreground)", fontSize: 11 }}
                  tickLine={false}
                  axisLine={false}
                  minTickGap={24}
                />
                <YAxis
                  tick={{ fill: "var(--color-muted-foreground)", fontSize: 11 }}
                  tickLine={false}
                  axisLine={false}
                  width={36}
                  tickFormatter={(v: number) => formatAxis(v)}
                />
                <Tooltip
                  cursor={{ fill: "var(--color-accent)" }}
                  content={ChartTooltip}
                />
                <Bar
                  dataKey="income"
                  name="Income"
                  fill="var(--color-income)"
                  radius={[4, 4, 0, 0]}
                  maxBarSize={18}
                />
                <Bar
                  dataKey="expense"
                  name="Expenses"
                  fill="var(--color-expense)"
                  radius={[4, 4, 0, 0]}
                  maxBarSize={18}
                />
              </BarChart>
            </ResponsiveContainer>
          </ClientOnly>
        </div>
        )}
      </CardContent>
    </Card>
  );
}
```

src/components/ledger/income-mix.tsx

```tsx
import { Cell, Pie, PieChart, ResponsiveContainer, Tooltip } from "recharts";
import type { PayeeSlice } from "@/lib/ledger/analytics";
import { formatRon } from "@/lib/ledger/format";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Skeleton } from "@/components/ui/skeleton";
import { cn } from "@/lib/utils";
import { ClientOnly } from "./client-only";
import { ChartTooltip } from "./chart-tooltip";

export function IncomeMix({
  data,
  focus,
  onFocus,
}: {
  data: PayeeSlice[];
  focus: string | null;
  onFocus: (payee: string) => void;
}) {
  const total = data.reduce((s, d) => s + d.value, 0);
  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: "220ms" }}>
      <CardHeader>
        <CardTitle>Income by payee</CardTitle>
      </CardHeader>
      <CardContent>
        {data.length === 0 ? (
          <p className="py-10 text-center text-sm text-muted-foreground">
            No income in this range. Add income to see who paid you.
          </p>
        ) : (
          <div className="flex flex-col gap-4">
            <div
              className="relative mx-auto h-52 w-full max-w-xs overflow-hidden outline-none [&_svg]:outline-none"
              style={{ WebkitTapHighlightColor: "transparent" }}
            >
              <ClientOnly fallback={<Skeleton className="mx-auto size-52 rounded-full" />}>
                <ResponsiveContainer width="100%" height="100%">
                  <PieChart>
                    <Pie
                      data={data}
                      dataKey="value"
                      nameKey="label"
                      innerRadius="62%"
                      outerRadius="88%"
                      paddingAngle={1.5}
                      stroke="none"
                      isAnimationActive={false}
                      activeShape={false}
                      onClick={(_, index) => {
                        const slice = data[index];
                        if (slice) onFocus(slice.payee);
                      }}
                      style={{ cursor: "pointer", outline: "none" }}
                    >
                      {data.map((slice) => (
                        <Cell
                          key={slice.payee}
                          fill={slice.color}
                          stroke="none"
                          opacity={focus && focus.toLowerCase() !== slice.payee.toLowerCase() ? 0.28 : 1}
                        />
                      ))}
                    </Pie>
                    <Tooltip content={ChartTooltip} cursor={false} wrapperStyle={{ outline: "none" }} />
                  </PieChart>
                </ResponsiveContainer>
              </ClientOnly>
              <div className="pointer-events-none absolute inset-0 flex flex-col items-center justify-center px-10 text-center">
                <span className="text-xs text-muted-foreground">Income</span>
                <span className="font-mono text-sm tabular-nums text-foreground">
                  {formatRon(total)}
                </span>
              </div>
            </div>
            <ul className="space-y-1">
              {data.slice(0, 6).map((slice) => (
                <li key={slice.payee}>
```

```
                  <button
                    type="button"
                    onClick={() => onFocus(slice.payee)}
                    className={cn(
                      "flex min-h-11 w-full items-center gap-2 rounded-lg px-2 text-left transition-colors duration-150 hover:bg-accent",
                      focus && focus.toLowerCase() === slice.payee.toLowerCase() && "bg-accent",
                    )}
                  >
                    <span
                      className="size-2 shrink-0 rounded-full"
                      style={{ background: slice.color }}
                    />
                    <span className="min-w-0 flex-1 truncate text-sm">{slice.label}</span>
                    <span className="shrink-0 font-mono text-xs tabular-nums whitespace-nowrap text-muted-foreground">
                      {(slice.share * 100).toFixed(0)}%
                    </span>
                  </button>
                </li>
              ))}
            </ul>
          </div>
        )}
      </CardContent>
    </Card>
  );
}
```

src/components/ledger/kpi-grid.tsx

```tsx
import { ArrowDownRight, ArrowUpRight, Minus } from "lucide-react";
import type { KpiPoint } from "@/lib/ledger/analytics";
import { formatPct, formatPts, formatRon, formatRonSigned } from "@/lib/ledger/format";
import { cn } from "@/lib/utils";
import { Sparkline } from "./sparkline";

function Delta({
  value,
  invert,
  unit = "pct",
}: {
  value: number | null;
  invert?: boolean;
  unit?: "pct" | "pts";
}) {
  if (value === null) {
    return <span className="text-xs text-subtle">vs prior period</span>;
  }
  const Icon = value === 0 ? Minus : value > 0 ? ArrowUpRight : ArrowDownRight;
  const good = invert ? value < 0 : value > 0;
  return (
    <span
      className={cn(
        "inline-flex items-center gap-0.5 font-mono text-xs tabular-nums",
        value === 0 ? "text-muted-foreground" : good ? "text-income" : "text-expense",
      )}
    >
      <Icon className="size-3.5" />
      {unit === "pts" ? formatPts(value) : formatPct(value)}
      <span className="ml-1 font-sans text-subtle">vs prior</span>
    </span>
  );
}

function KpiCard({
  label,
  value,
  display,
  invert,
  delay,
  unit,
}: {
  label: string;
  value: KpiPoint;
  display: string;
  invert?: boolean;
  delay: number;
  unit?: "pct" | "pts";
}) {
  return (
    <article
      className="stagger-in rounded-2xl bg-card p-4 shadow-[var(--shadow-border)] sm:p-5"
      style={{ animationDelay: `${delay}ms` }}
    >
      <p className="text-xs font-medium tracking-wide text-muted-foreground uppercase">
        {label}
      </p>
      <p className="mt-2 font-mono text-xl tracking-tight text-foreground tabular-nums sm:text-2xl">
        {display}
      </p>
      <div className="mt-1">
        <Delta value={value.change} invert={invert} unit={unit} />
      </div>
      <div className="mt-3">
        <Sparkline values={value.series} />
      </div>
    </article>
  );
}

export function KpiGrid({
  net,
  income,
  expenses,
  savingsRate,
}: {
  net: KpiPoint;
  income: KpiPoint;
```

```
  expenses: KpiPoint;
  savingsRate: KpiPoint;
}) {
  return (
    <section className="grid grid-cols-2 gap-3 lg:grid-cols-4">
      <KpiCard
        label="Net cashflow"
        value={net}
        display={formatRonSigned(net.current)}
        delay={0}
      />
      <KpiCard
        label="Income"
        value={income}
        display={formatRon(income.current)}
        delay={40}
      />
      <KpiCard
        label="Expenses"
        value={expenses}
        display={formatRon(expenses.current)}
        invert
        delay={80}
      />
      <KpiCard
        label="Savings rate"
        value={savingsRate}
        display={
          income.current <= 0 && savingsRate.current === 0
            ? "—"
            : formatPct(savingsRate.current)
        }
        delay={120}
        unit="pts"
      />
    </section>
  );
}
```

src/components/ledger/manage-budgets.tsx

```tsx
import { useMemo, useState } from "react";
import { Pencil, Plus, Trash2 } from "lucide-react";
import { categoriesFor } from "@/lib/ledger/categories";
import { formatRon, parseAmount } from "@/lib/ledger/format";
import { useLedger } from "@/lib/ledger/store";
import { Button } from "@/components/ui/button";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import {
  Drawer,
  DrawerContent,
  DrawerDescription,
  DrawerHeader,
  DrawerTitle,
} from "@/components/ui/drawer";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { cn } from "@/lib/utils";
import { toast } from "sonner";

export function ManageBudgets({
  open,
  onOpenChange,
}: {
  open: boolean;
  onOpenChange: (open: boolean) => void;
}) {
  const categories = useLedger((s) => s.categories);
  const budgets = useLedger((s) => s.budgets);
  const addBudget = useLedger((s) => s.addBudget);
  const updateBudget = useLedger((s) => s.updateBudget);
  const deleteBudget = useLedger((s) => s.deleteBudget);
  const expenseCats = useMemo(() => categoriesFor(categories, "expense"), [categories]);
  const used = useMemo(() => new Set(budgets.map((b) => b.category)), [budgets]);
  const available = expenseCats.filter((c) => !used.has(c.id));

  const [category, setCategory] = useState("");
  const [amount, setAmount] = useState("");
  const [editing, setEditing] = useState<string | null>(null);
  const [editValue, setEditValue] = useState("");
  const [pending, setPending] = useState<string | null>(null);

  function resetAdd() {
    setCategory("");
    setAmount("");
  }

  function add() {
    const monthly = parseAmount(amount);
    if (!monthly) {
      toast.error("Enter an amount greater than zero.");
      return;
    }
    if (!category) {
      toast.error("Pick an expense category.");
      return;
    }
    const row = addBudget({ category, monthly });
    if (!row) {
      toast.error("That category already has a budget.");
      return;
    }
    resetAdd();
    toast.success("Budget saved");
  }

  function saveEdit(id: string) {
    const monthly = parseAmount(editValue);
    if (!monthly) {
      setEditing(null);
```

```
      return;
    }
    if (!updateBudget(id, monthly)) {
      toast.error("Enter an amount greater than zero.");
      return;
    }
    setEditing(null);
    toast.success("Budget updated");
  }

  const pendingRow = pending ? budgets.find((b) => b.id === pending) : null;
  const pendingLabel =
    pendingRow && expenseCats.find((c) => c.id === pendingRow.category)?.label;

  return (
    <>
      <Drawer
        open={open}
        onOpenChange={(next) => {
          onOpenChange(next);
          if (!next) {
            resetAdd();
            setEditing(null);
          }
        }}
      >
        <DrawerContent>
          <DrawerHeader>
            <DrawerTitle>Budgets</DrawerTitle>
            <DrawerDescription>
              Monthly caps for expense categories, in RON. Progress is this calendar month.
            </DrawerDescription>
          </DrawerHeader>
          <div className="flex min-h-0 flex-1 flex-col gap-5 overflow-y-auto px-5 pb-6">
            {expenseCats.length === 0 ? (
              <p className="text-sm text-muted-foreground">
                Add an expense category first, then set a cap on it.
              </p>
            ) : null}

            {budgets.length === 0 ? (
              <p className="text-sm text-muted-foreground">No budgets yet.</p>
            ) : (
              <ul className="flex flex-col gap-1">
                {budgets.map((row) => {
                  const cat = expenseCats.find((c) => c.id === row.category);
                  return (
                    <li
                      key={row.id}
                      className="flex min-h-11 items-center gap-2 rounded-lg bg-muted px-2"
                    >
                      <span
                        className="size-2.5 shrink-0 rounded-full"
                        style={{ background: cat?.color }}
                      />
                      <span className="min-w-0 flex-1 truncate text-sm">
                        {cat?.label ?? "Uncategorized"}
                      </span>
                      {editing === row.id ? (
                        <Input
                          value={editValue}
                          onChange={(e) => setEditValue(e.target.value)}
                          inputMode="decimal"
                          autoFocus
                          className="h-9 min-h-9 w-28 bg-background font-mono"
                          onKeyDown={(e) => {
                            if (e.key === "Enter") {
                              e.preventDefault();
                              saveEdit(row.id);
                            }
                            if (e.key === "Escape") setEditing(null);
                          }}
                          onBlur={() => saveEdit(row.id)}
                        />
                      ) : (
                        <span className="font-mono text-xs tabular-nums text-muted-foreground">
                          {formatRon(row.monthly)}
                        </span>
                      )}
                      <Button
```

```
                      type="button"
                      variant="ghost"
                      size="icon-sm"
                      aria-label={`Edit ${cat?.label ?? "budget"}`}
                      onClick={() => {
                        setEditing(row.id);
                        setEditValue(String(row.monthly));
                      }}
                    >
                      <Pencil />
                    </Button>
                    <Button
                      type="button"
                      variant="ghost"
                      size="icon-sm"
                      aria-label={`Delete ${cat?.label ?? "budget"}`}
                      className="text-subtle hover:text-expense"
                      onClick={() => setPending(row.id)}
                    >
                      <Trash2 />
                    </Button>
                  </li>
                );
              })}
            </ul>
          )}

          {available.length > 0 ? (
            <section className="flex flex-col gap-2">
              <p className="text-xs font-medium tracking-wide text-muted-foreground uppercase">
                New budget
              </p>
              <Label>Category</Label>
              <div className="flex flex-wrap gap-1.5">
                {available.map((c) => (
                  <button
                    key={c.id}
                    type="button"
                    onClick={() => setCategory(c.id)}
                    className={cn(
                      "h-9 min-h-9 rounded-full border px-3 text-xs font-medium transition-colors duration-150",
                      category === c.id
                        ? "border-primary bg-primary text-primary-foreground"
                        : "border-border bg-transparent text-muted-foreground hover:text-foreground",
                    )}
                  >
                    {c.label}
                  </button>
                ))}
              </div>
              <div className="flex gap-2 pt-1">
                <Input
                  value={amount}
                  onChange={(e) => setAmount(e.target.value)}
                  inputMode="decimal"
                  placeholder="Monthly cap"
                  className="h-10 min-h-10 font-mono"
                  onKeyDown={(e) => {
                    if (e.key === "Enter") {
                      e.preventDefault();
                      add();
                    }
                  }}
                />
                <Button
                  type="button"
                  size="sm"
                  variant="outline"
                  onClick={add}
                  className="shrink-0"
                  aria-label="Add budget"
                >
                  <Plus />
                  Add
                </Button>
              </div>
            </section>
          ) : expenseCats.length > 0 ? (
            <p className="text-sm text-muted-foreground">
              Every expense category already has a budget.
```

```
              </p>
            ) : null}
          </div>
        </DrawerContent>
      </Drawer>

      <AlertDialog open={pending !== null} onOpenChange={(o) => !o && setPending(null)}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>Remove this budget?</AlertDialogTitle>
            <AlertDialogDescription>
              {pendingLabel
                ? `The monthly cap for ${pendingLabel} will be removed. Transactions stay.`
                : "This monthly cap will be removed."}
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Cancel</AlertDialogCancel>
            <AlertDialogAction
              className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
              onClick={() => {
                if (pending) deleteBudget(pending);
                setPending(null);
                toast.success("Budget removed");
              }}
            >
              Remove
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </>
  );
}
```

src/components/ledger/manage-categories.tsx

```tsx
import { useMemo, useState } from "react";
import { Pencil, Plus, Trash2 } from "lucide-react";
import { categoriesFor, normalizeLabel } from "@/lib/ledger/categories";
import { useLedger } from "@/lib/ledger/store";
import type { Category, TxType } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import {
  Drawer,
  DrawerContent,
  DrawerDescription,
  DrawerHeader,
  DrawerTitle,
} from "@/components/ui/drawer";
import { Input } from "@/components/ui/input";
import { toast } from "sonner";

function CategorySection({
  title,
  type,
  rows,
  onAskDelete,
}: {
  title: string;
  type: TxType;
  rows: Category[];
  onAskDelete: (cat: Category) => void;
}) {
  const addCategory = useLedger((s) => s.addCategory);
  const updateCategory = useLedger((s) => s.updateCategory);
  const [draft, setDraft] = useState("");
  const [editing, setEditing] = useState<string | null>(null);
  const [editValue, setEditValue] = useState("");

  function add() {
    const cleaned = normalizeLabel(draft);
    if (!cleaned) return;
    const row = addCategory({ label: cleaned, type });
    if (!row) {
      toast.error("Use a unique name.");
      return;
    }
    setDraft("");
    toast.success("Category added");
  }

  function saveEdit(id: string) {
    const cleaned = normalizeLabel(editValue);
    if (!cleaned) {
      setEditing(null);
      return;
    }
    if (!updateCategory(id, cleaned)) {
      toast.error("That name is already used.");
      return;
    }
    setEditing(null);
    toast.success("Category renamed");
  }

  return (
    <section className="flex flex-col gap-2">
      <p className="text-xs font-medium tracking-wide text-muted-foreground uppercase">
        {title}
      </p>
      {rows.length === 0 ? (
        <p className="text-sm text-muted-foreground">None yet.</p>
      ) : (
        <ul className="flex flex-col gap-1">
          {rows.map((cat) => (
            <li
              key={cat.id}
              className="flex min-h-11 items-center gap-2 rounded-lg bg-muted px-2"
            >
              <span
                className="size-2.5 shrink-0 rounded-full"
                style={{ background: cat.color }}
              />
              {editing === cat.id ? (
```

```
                <Input
                  value={editValue}
                  onChange={(e) => setEditValue(e.target.value)}
                  maxLength={40}
                  autoFocus
                  className="h-9 min-h-9 bg-background"
                  onKeyDown={(e) => {
                    if (e.key === "Enter") {
                      e.preventDefault();
                      saveEdit(cat.id);
                    }
                    if (e.key === "Escape") setEditing(null);
                  }}
                  onBlur={() => saveEdit(cat.id)}
                />
              ) : (
                <span className="min-w-0 flex-1 truncate text-sm">{cat.label}</span>
              )}
              <Button
                type="button"
                variant="ghost"
                size="icon-sm"
                aria-label={`Rename ${cat.label}`}
                onClick={() => {
                  setEditing(cat.id);
                  setEditValue(cat.label);
                }}
              >
                <Pencil />
              </Button>
              <Button
                type="button"
                variant="ghost"
                size="icon-sm"
                aria-label={`Delete ${cat.label}`}
                className="text-expense hover:text-expense"
                onClick={() => onAskDelete(cat)}
              >
                <Trash2 />
              </Button>
            </li>
          ))}
        </ul>
      )}
      <div className="flex gap-2">
        <Input
          value={draft}
          onChange={(e) => setDraft(e.target.value)}
          placeholder={`New ${type} category`}
          maxLength={40}
          className="h-10 min-h-10"
          onKeyDown={(e) => {
            if (e.key === "Enter") {
              e.preventDefault();
              add();
            }
          }}
        />
        <Button type="button" size="sm" variant="outline" onClick={add} className="shrink-0" aria-label={`Add ${type}
    category`}>
          <Plus />
          Add
        </Button>
      </div>
    </section>
  );
}

export function ManageCategories({
  open,
  onOpenChange,
}: {
  open: boolean;
  onOpenChange: (open: boolean) => void;
}) {
  const categories = useLedger((s) => s.categories);
  const transactions = useLedger((s) => s.transactions);
  const budgets = useLedger((s) => s.budgets);
  const recurrences = useLedger((s) => s.recurrences);
  const deleteCategory = useLedger((s) => s.deleteCategory);
```

```
  const income = useMemo(() => categoriesFor(categories, "income"), [categories]);
  const expense = useMemo(() => categoriesFor(categories, "expense"), [categories]);
  const [pending, setPending] = useState<Category | null>(null);
  const used = pending
    ? transactions.filter((t) => t.category === pending.id).length
    : 0;
  const usedBills = pending
    ? recurrences.filter((r) => r.category === pending.id).length
    : 0;
  const usedBudget = pending
    ? budgets.some((b) => b.category === pending.id)
    : false;

  const deleteCopy = (() => {
    const bits: string[] = [];
    if (used > 0) {
      bits.push(
        `${used} transaction${used === 1 ? "" : "s"} will stay, marked uncategorized.`,
      );
    }
    if (usedBills > 0) {
      bits.push(
        `${usedBills} recurring bill${usedBills === 1 ? "" : "s"} using this category will be deleted.`,
      );
    }
    if (usedBudget) bits.push("Its monthly budget will be removed.");
    if (bits.length === 0) return "This category will be removed from this device.";
    return bits.join(" ");
  })();

  return (
    <Drawer
      open={open}
      onOpenChange={(next) => {
        if (!next) setPending(null);
        onOpenChange(next);
      }}
    >
      <DrawerContent>
        <DrawerHeader>
          <DrawerTitle>Categories</DrawerTitle>
          <DrawerDescription>
            Add, rename, or remove income and expense categories.
          </DrawerDescription>
        </DrawerHeader>
        <div className="flex min-h-0 flex-1 flex-col gap-6 overflow-y-auto px-5 pb-6">
          {pending ? (
            <div className="rounded-xl bg-muted p-4" data-vaul-no-drag>
              <p className="text-sm font-medium">Delete {pending.label}?</p>
              <p className="mt-1 text-sm text-muted-foreground">{deleteCopy}</p>
              <div className="mt-4 flex gap-2">
                <Button type="button" variant="outline" className="flex-1" onClick={() => setPending(null)}>
                  Cancel
                </Button>
                <Button
                  type="button"
                  variant="destructive"
                  className="flex-1"
                  onClick={() => {
                    deleteCategory(pending.id);
                    setPending(null);
                    toast.success("Category deleted");
                  }}
                >
                  Delete
                </Button>
              </div>
            </div>
          ) : null}
          <CategorySection
            title="Income"
            type="income"
            rows={income}
            onAskDelete={setPending}
          />
          <CategorySection
            title="Expenses"
            type="expense"
            rows={expense}
            onAskDelete={setPending}
```

```
            />
          </div>
        </DrawerContent>
      </Drawer>
    );
  }
```

src/components/ledger/manage-payees.tsx

```tsx
import { useMemo, useState } from "react";
import { Pencil, Plus, Trash2 } from "lucide-react";
import { categoriesFor, normalizeLabel, payeesFor } from "@/lib/ledger/categories";
import { useLedger } from "@/lib/ledger/store";
import type { CategoryId, Payee, TxType } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import {
  Drawer,
  DrawerContent,
  DrawerDescription,
  DrawerHeader,
  DrawerTitle,
} from "@/components/ui/drawer";
import { Input } from "@/components/ui/input";
import { toast } from "sonner";

function PayeeSection({
  title,
  type,
  category,
  rows,
  onAskDelete,
}: {
  title: string;
  type: TxType;
  category: CategoryId;
  rows: Payee[];
  onAskDelete: (payee: Payee) => void;
}) {
  const addPayee = useLedger((s) => s.addPayee);
  const renamePayee = useLedger((s) => s.renamePayee);
  const [draft, setDraft] = useState("");
  const [editing, setEditing] = useState<string | null>(null);
  const [editValue, setEditValue] = useState("");

  function add() {
    const cleaned = normalizeLabel(draft);
    if (!cleaned) return;
    const existing = rows.some((p) => p.name.toLowerCase() === cleaned.toLowerCase());
    const added = addPayee(cleaned, type, category);
    if (!added) {
      toast.error("Enter a payee name.");
      return;
    }
    setDraft("");
    toast.success(existing ? "Already in this list" : "Payee added");
  }

  function saveEdit(from: string) {
    const cleaned = normalizeLabel(editValue);
    if (!cleaned || cleaned === from) {
      setEditing(null);
      return;
    }
    if (!renamePayee(from, cleaned, type, category)) {
      toast.error("That payee already exists.");
      return;
    }
    setEditing(null);
    toast.success("Payee renamed");
  }

  return (
    <section className="flex flex-col gap-2" data-vaul-no-drag>
      <p className="text-xs font-medium tracking-wide text-muted-foreground uppercase">
        {title}
      </p>
      {rows.length === 0 ? (
        <p className="text-sm text-muted-foreground">None yet.</p>
      ) : (
        <ul className="flex flex-col gap-2">
          {rows.map((row) => (
            <li
              key={`${row.type}:${row.category}:${row.name}`}
              className="flex flex-col gap-2 rounded-xl bg-muted p-3"
            >
              {editing === row.name ? (
                <Input
```

```
                  value={editValue}
                  onChange={(e) => setEditValue(e.target.value)}
                  maxLength={80}
                  autoFocus
                  className="h-11 min-h-11 bg-background"
                  onKeyDown={(e) => {
                    if (e.key === "Enter") {
                      e.preventDefault();
                      saveEdit(row.name);
                    }
                    if (e.key === "Escape") setEditing(null);
                  }}
                />
              ) : (
                <p className="text-sm font-medium">{row.name}</p>
              )}
              <div className="grid grid-cols-2 gap-2">
                <Button
                  type="button"
                  variant="outline"
                  size="sm"
                  onClick={() => {
                    if (editing === row.name) {
                      saveEdit(row.name);
                      return;
                    }
                    setEditing(row.name);
                    setEditValue(row.name);
                  }}
                >
                  <Pencil />
                  {editing === row.name ? "Save" : "Edit"}
                </Button>
                <Button
                  type="button"
                  variant="destructive"
                  size="sm"
                  onClick={() => onAskDelete(row)}
                >
                  <Trash2 />
                  Delete
                </Button>
              </div>
            </li>
          ))}
        </ul>
      )}
      {category ? (
        <div className="flex gap-2">
          <Input
            value={draft}
            onChange={(e) => setDraft(e.target.value)}
            placeholder={type === "income" ? "Employer, client…" : "Shop, landlord…"}
            maxLength={80}
            className="h-10 min-h-10"
            onKeyDown={(e) => {
              if (e.key === "Enter") {
                e.preventDefault();
                add();
              }
            }}
          />
          <Button type="button" size="sm" variant="outline" onClick={add} className="shrink-0" aria-label={`Add ${titl
e} payee`}>
            <Plus />
            Add
          </Button>
        </div>
      ) : null}
    </section>
  );
}

function TypeBlock({
  title,
  type,
  onAskDelete,
}: {
  title: string;
  type: TxType;
```

```
  onAskDelete: (payee: Payee) => void;
}) {
  const categories = useLedger((s) => s.categories);
  const payees = useLedger((s) => s.payees);
  const cats = useMemo(() => categoriesFor(categories, type), [categories, type]);
  const loose = useMemo(
    () => payeesFor(payees, type).filter((p) => !p.category),
    [payees, type],
  );

  return (
    <section className="flex flex-col gap-4">
      <p className="text-sm font-medium">{title}</p>
      {cats.length === 0 ? (
        <p className="text-sm text-muted-foreground">Add {type} categories first, then payees for each.</p>
      ) : (
        cats.map((cat) => (
          <PayeeSection
            key={cat.id}
            title={cat.label}
            type={type}
            category={cat.id}
            rows={payeesFor(payees, type, cat.id).filter((p) => p.category === cat.id)}
            onAskDelete={onAskDelete}
          />
        ))
      )}
      {loose.length > 0 ? (
        <PayeeSection
          title="Unassigned"
          type={type}
          category=""
          rows={loose}
          onAskDelete={onAskDelete}
        />
      ) : null}
    </section>
  );
}

export function ManagePayees({
  open,
  onOpenChange,
}: {
  open: boolean;
  onOpenChange: (open: boolean) => void;
}) {
  const transactions = useLedger((s) => s.transactions);
  const deletePayee = useLedger((s) => s.deletePayee);
  const [pending, setPending] = useState<Payee | null>(null);
  const used = pending
    ? transactions.filter(
        (t) =>
          t.type === pending.type &&
          t.payee === pending.name &&
          (pending.category ? t.category === pending.category : true),
      ).length
    : 0;

  function confirmRemove() {
    if (!pending) return;
    deletePayee(pending.name, pending.type, pending.category);
    toast.success("Payee removed");
    setPending(null);
  }

  return (
    <Drawer
      open={open}
      onOpenChange={(next) => {
        if (!next) setPending(null);
        onOpenChange(next);
      }}
    >
      <DrawerContent>
        <DrawerHeader>
          <DrawerTitle>Payees</DrawerTitle>
          <DrawerDescription>
            Each category has its own payee list. Transport shops stay under Transport.
          </DrawerDescription>
```

```
        </DrawerHeader>
        <div className="flex min-h-0 flex-1 flex-col gap-8 overflow-y-auto px-5 pb-6">
          {pending ? (
            <div className="rounded-xl bg-muted p-4" data-vaul-no-drag>
              <p className="text-sm font-medium">Remove {pending.name}?</p>
              <p className="mt-1 text-sm text-muted-foreground">
                {used > 0
                  ? `${used} existing transaction${used === 1 ? "" : "s"} keep this name. It just won’t appear in this category’s picklist.`
                  : "This payee will be removed from this category’s picklist."}
              </p>
              <div className="mt-4 flex gap-2">
                <Button type="button" variant="outline" className="flex-1" onClick={() => setPending(null)}>
                  Cancel
                </Button>
                <Button type="button" variant="destructive" className="flex-1" onClick={confirmRemove}>
                  Remove
                </Button>
              </div>
            </div>
          ) : null}
          <TypeBlock title="Income" type="income" onAskDelete={setPending} />
          <TypeBlock title="Expenses" type="expense" onAskDelete={setPending} />
        </div>
      </DrawerContent>
    </Drawer>
  );
}
```

src/components/ledger/manage-products.tsx

```tsx
import { useMemo, useState } from "react";
import { Pencil, Plus, Trash2 } from "lucide-react";
import { categoriesFor, normalizeLabel, productsFor } from "@/lib/ledger/categories";
import { useLedger } from "@/lib/ledger/store";
import type { CategoryId, Product } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import {
  Drawer,
  DrawerContent,
  DrawerDescription,
  DrawerHeader,
  DrawerTitle,
} from "@/components/ui/drawer";
import { Input } from "@/components/ui/input";
import { toast } from "sonner";

function ProductSection({
  title,
  category,
  rows,
  onAskDelete,
}: {
  title: string;
  category: CategoryId;
  rows: Product[];
  onAskDelete: (product: Product) => void;
}) {
  const addProduct = useLedger((s) => s.addProduct);
  const renameProduct = useLedger((s) => s.renameProduct);
  const [draft, setDraft] = useState("");
  const [editing, setEditing] = useState<string | null>(null);
  const [editValue, setEditValue] = useState("");

  function add() {
    const cleaned = normalizeLabel(draft);
    if (!cleaned) return;
    const existing = rows.some((p) => p.name.toLowerCase() === cleaned.toLowerCase());
    const added = addProduct(cleaned, category);
    if (!added) {
      toast.error("Enter a product name.");
      return;
    }
    setDraft("");
    toast.success(existing ? "Already in this list" : "Product added");
  }

  function saveEdit(from: string) {
    const cleaned = normalizeLabel(editValue);
    if (!cleaned || cleaned === from) {
      setEditing(null);
      return;
    }
    if (!renameProduct(from, cleaned, category)) {
      toast.error("That product already exists.");
      return;
    }
    setEditing(null);
    toast.success("Product renamed");
  }

  return (
    <section className="flex flex-col gap-2" data-vaul-no-drag>
      <p className="text-xs font-medium tracking-wide text-muted-foreground uppercase">
        {title}
      </p>
      {rows.length === 0 ? (
        <p className="text-sm text-muted-foreground">None yet.</p>
      ) : (
        <ul className="flex flex-col gap-2">
          {rows.map((row) => (
            <li
              key={`${row.category}:${row.name}`}
              className="flex flex-col gap-2 rounded-xl bg-muted p-3"
            >
              {editing === row.name ? (
                <Input
                  value={editValue}
                  onChange={(e) => setEditValue(e.target.value)}
```

```
                  maxLength={80}
                  autoFocus
                  className="h-11 min-h-11 bg-background"
                  onKeyDown={(e) => {
                    if (e.key === "Enter") {
                      e.preventDefault();
                      saveEdit(row.name);
                    }
                    if (e.key === "Escape") setEditing(null);
                  }}
                />
              ) : (
                <p className="text-sm font-medium">{row.name}</p>
              )}
              <div className="grid grid-cols-2 gap-2">
                <Button
                  type="button"
                  variant="outline"
                  size="sm"
                  onClick={() => {
                    if (editing === row.name) {
                      saveEdit(row.name);
                      return;
                    }
                    setEditing(row.name);
                    setEditValue(row.name);
                  }}
                >
                  <Pencil />
                  {editing === row.name ? "Save" : "Edit"}
                </Button>
                <Button
                  type="button"
                  variant="destructive"
                  size="sm"
                  onClick={() => onAskDelete(row)}
                >
                  <Trash2 />
                  Delete
                </Button>
              </div>
            </li>
          ))}
        </ul>
      )}
      {category ? (
        <div className="flex gap-2">
          <Input
            value={draft}
            onChange={(e) => setDraft(e.target.value)}
            placeholder="Eggs, honey, firewood…"
            maxLength={80}
            className="h-10 min-h-10"
            onKeyDown={(e) => {
              if (e.key === "Enter") {
                e.preventDefault();
                add();
              }
            }}
          />
          <Button type="button" size="sm" variant="outline" onClick={add} className="shrink-0" aria-label={`Add ${titl
e} product`}>
            <Plus />
            Add
          </Button>
        </div>
      ) : null}
    </section>
  );
}

export function ManageProducts({
  open,
  onOpenChange,
}: {
  open: boolean;
  onOpenChange: (open: boolean) => void;
}) {
  const categories = useLedger((s) => s.categories);
  const products = useLedger((s) => s.products);
```

```
  const transactions = useLedger((s) => s.transactions);
  const deleteProduct = useLedger((s) => s.deleteProduct);
  const cats = useMemo(() => categoriesFor(categories, "income"), [categories]);
  const loose = useMemo(
    () => productsFor(products).filter((p) => !p.category),
    [products],
  );
  const [pending, setPending] = useState<Product | null>(null);
  const used = pending
    ? transactions.filter(
        (t) =>
          t.type === "income" &&
          t.product === pending.name &&
          (pending.category ? t.category === pending.category : true),
      ).length
    : 0;

  function confirmRemove() {
    if (!pending) return;
    deleteProduct(pending.name, pending.category);
    toast.success("Product removed");
    setPending(null);
  }

  return (
    <Drawer
      open={open}
      onOpenChange={(next) => {
        if (!next) setPending(null);
        onOpenChange(next);
      }}
    >
      <DrawerContent>
        <DrawerHeader>
          <DrawerTitle>Products</DrawerTitle>
          <DrawerDescription>
            Income only. Each income category has its own product list.
          </DrawerDescription>
        </DrawerHeader>
        <div className="flex min-h-0 flex-1 flex-col gap-6 overflow-y-auto px-5 pb-6">
          {pending ? (
            <div className="rounded-xl bg-muted p-4" data-vaul-no-drag>
              <p className="text-sm font-medium">Remove {pending.name}?</p>
              <p className="mt-1 text-sm text-muted-foreground">
                {used > 0
                  ? `${used} existing transaction${used === 1 ? "" : "s"} keep this name. It just won’t appear in the picklist.`
                  : "This product will be removed from this category’s picklist."}
              </p>
              <div className="mt-4 flex gap-2">
                <Button type="button" variant="outline" className="flex-1" onClick={() => setPending(null)}>
                  Cancel
                </Button>
                <Button type="button" variant="destructive" className="flex-1" onClick={confirmRemove}>
                  Remove
                </Button>
              </div>
            </div>
          ) : null}
          {cats.length === 0 ? (
            <p className="text-sm text-muted-foreground">Add income categories first, then products for each.</p>
          ) : (
            cats.map((cat) => (
              <ProductSection
                key={cat.id}
                title={cat.label}
                category={cat.id}
                rows={productsFor(products, cat.id).filter((p) => p.category === cat.id)}
                onAskDelete={setPending}
              />
            ))
          )}
          {loose.length > 0 ? (
            <ProductSection
              title="Unassigned"
              category=""
              rows={loose}
              onAskDelete={setPending}
            />
          ) : null}
```

```
          </div>
        </DrawerContent>
      </Drawer>
    );
  }
```

src/components/ledger/manage-recurring.tsx

```tsx
import { useMemo, useState } from "react";
import { Pause, Pencil, Play, Plus, Trash2 } from "lucide-react";
import { categoriesFor, labelsEqual, payeeNames } from "@/lib/ledger/categories";
import { formatDayYear, formatRon, parseAmount } from "@/lib/ledger/format";
import { FREQ_LABELS } from "@/lib/ledger/recurrence";
import { useLedger } from "@/lib/ledger/store";
import type { Frequency, RecurringBill, TxType } from "@/lib/ledger/types";
import { FREQUENCIES } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import {
  Drawer,
  DrawerContent,
  DrawerDescription,
  DrawerFooter,
  DrawerHeader,
  DrawerTitle,
} from "@/components/ui/drawer";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { cn } from "@/lib/utils";
import { toast } from "sonner";
import { PayeePicker } from "./payee-picker";

type Draft = {
  type: TxType;
  amount: string;
  category: string;
  payee: string;
  note: string;
  frequency: Frequency;
  nextDate: string;
  autoPost: boolean;
};

function emptyDraft(date: string): Draft {
  return {
    type: "expense",
    amount: "",
    category: "",
    payee: "",
    note: "",
    frequency: "monthly",
    nextDate: date,
    autoPost: true,
  };
}

function fromBill(bill: RecurringBill): Draft {
  return {
    type: bill.type,
    amount: String(bill.amount),
    category: bill.category,
    payee: bill.payee,
    note: bill.note ?? "",
    frequency: bill.frequency,
    nextDate: bill.nextDate,
    autoPost: bill.autoPost,
  };
}

function Toggle({
  checked,
  onChange,
  label,
}: {
  checked: boolean;
  onChange: (value: boolean) => void;
  label: string;
```

```
}) {
  return (
    <button
      type="button"
      role="switch"
      aria-checked={checked}
      onClick={() => onChange(!checked)}
      className="flex min-h-11 w-full items-center justify-between gap-3 rounded-lg bg-muted px-3 text-sm"
    >
      <span>{label}</span>
      <span
        className={cn(
          "relative h-6 w-10 shrink-0 rounded-full transition-colors duration-150",
          checked ? "bg-primary" : "bg-border",
        )}
      >
        <span
          className={cn(
            "absolute top-0.5 size-5 rounded-full bg-background transition-transform duration-150",
            checked ? "translate-x-4" : "translate-x-0.5",
          )}
        />
      </span>
    </button>
  );
}

export function ManageRecurring({
  open,
  onOpenChange,
  defaultDate,
}: {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  defaultDate: string;
}) {
  const categories = useLedger((s) => s.categories);
  const payees = useLedger((s) => s.payees);
  const recurrences = useLedger((s) => s.recurrences);
  const addPayee = useLedger((s) => s.addPayee);
  const deletePayee = useLedger((s) => s.deletePayee);
  const addRecurrence = useLedger((s) => s.addRecurrence);
  const updateRecurrence = useLedger((s) => s.updateRecurrence);
  const deleteRecurrence = useLedger((s) => s.deleteRecurrence);
  const toggleRecurrencePaused = useLedger((s) => s.toggleRecurrencePaused);

  const [mode, setMode] = useState<"list" | "form">("list");
  const [editingId, setEditingId] = useState<string | null>(null);
  const [draft, setDraft] = useState<Draft>(() => emptyDraft(defaultDate));
  const [error, setError] = useState<string | null>(null);
  const [pending, setPending] = useState<RecurringBill | null>(null);

  const cats = useMemo(
    () => categoriesFor(categories, draft.type),
    [categories, draft.type],
  );

  function reset() {
    setMode("list");
    setEditingId(null);
    setDraft(emptyDraft(defaultDate));
    setError(null);
  }

  function startNew() {
    setEditingId(null);
    setDraft(emptyDraft(defaultDate));
    setError(null);
    setMode("form");
  }

  function startEdit(bill: RecurringBill) {
    setEditingId(bill.id);
    setDraft(fromBill(bill));
    setError(null);
    setMode("form");
  }

  function submit() {
    const amount = parseAmount(draft.amount);
```

```
    if (!amount) {
      setError("Enter an amount greater than zero.");
      return;
    }
    if (!draft.payee.trim()) {
      setError("Pick a payee from your list.");
      return;
    }
    if (!draft.category) {
      setError("Pick a category.");
      return;
    }
    if (!/^\d{4}-\d{2}-\d{2}$/.test(draft.nextDate)) {
      setError("Pick a valid next date.");
      return;
    }
    const payload = {
      type: draft.type,
      amount,
      category: draft.category,
      payee: draft.payee.trim(),
      note: draft.note.trim() || undefined,
      frequency: draft.frequency,
      nextDate: draft.nextDate,
      autoPost: draft.autoPost,
      paused: false,
    };
    if (editingId) {
      const existing = recurrences.find((r) => r.id === editingId);
      updateRecurrence(editingId, { ...payload, paused: existing?.paused ?? false });
      toast.success("Recurring bill updated");
    } else {
      const { posted } = addRecurrence(payload);
      toast.success(
        posted.length ? `Saved and posted ${payload.payee}` : "Recurring bill saved",
      );
    }
    reset();
  }

  return (
    <>
      <Drawer
        open={open}
        onOpenChange={(next) => {
          onOpenChange(next);
          if (!next) reset();
        }}
      >
        <DrawerContent>
          <DrawerHeader>
            <DrawerTitle>{mode === "form" ? (editingId ? "Edit recurring" : "New recurring") : "Recurring"}</DrawerTit
le>
            <DrawerDescription>
              {mode === "form"
                ? "Repeats until you pause or delete it."
                : "Salary, rent, and other repeating rows."}
            </DrawerDescription>
          </DrawerHeader>

          {mode === "list" ? (
            <div className="flex min-h-0 flex-1 flex-col gap-3 overflow-y-auto px-5 pb-6">
              {recurrences.length === 0 ? (
                <p className="text-sm text-muted-foreground">
                  Nothing repeating yet. Add income or a bill, pick how often, and choose whether
                  Ledger posts it automatically.
                </p>
              ) : (
                <ul className="flex flex-col gap-1">
                  {recurrences.map((bill) => (
                    <li
                      key={bill.id}
                      className="flex min-h-14 items-center gap-2 rounded-lg bg-muted px-2 py-1.5"
                    >
                      <div className="min-w-0 flex-1">
                        <p className="truncate text-sm">
                          {bill.payee}
                          {bill.paused ? (
                            <span className="ml-2 text-xs text-subtle">Paused</span>
                          ) : null}
```

```
                      </p>
                      <p className="truncate text-xs text-muted-foreground">
                        {formatDayYear(bill.nextDate)} · {FREQ_LABELS[bill.frequency]} ·{" "}
                        {bill.autoPost ? "Auto" : "Manual"}
                      </p>
                    </div>
                    <span
                      className={cn(
                        "shrink-0 font-mono text-xs tabular-nums",
                        bill.type === "income" ? "text-income" : "text-foreground",
                      )}
                    >
                      {formatRon(bill.amount)}
                    </span>
                    <Button
                      type="button"
                      variant="ghost"
                      size="icon-sm"
                      aria-label={bill.paused ? `Resume ${bill.payee}` : `Pause ${bill.payee}`}
                      onClick={() => toggleRecurrencePaused(bill.id)}
                    >
                      {bill.paused ? <Play /> : <Pause />}
                    </Button>
                    <Button
                      type="button"
                      variant="ghost"
                      size="icon-sm"
                      aria-label={`Edit ${bill.payee}`}
                      onClick={() => startEdit(bill)}
                    >
                      <Pencil />
                    </Button>
                    <Button
                      type="button"
                      variant="ghost"
                      size="icon-sm"
                      aria-label={`Delete ${bill.payee}`}
                      className="text-subtle hover:text-expense"
                      onClick={() => setPending(bill)}
                    >
                      <Trash2 />
                    </Button>
                  </li>
                ))}
              </ul>
            )}
            <Button type="button" variant="outline" onClick={startNew}>
              <Plus />
              Add recurring
            </Button>
          </div>
        ) : (
          <>
            <form
              className="flex min-h-0 flex-1 flex-col gap-4 overflow-y-auto px-5"
              onSubmit={(e) => {
                e.preventDefault();
                submit();
              }}
            >
              <div className="flex rounded-xl bg-muted p-1">
                {(["expense", "income"] as const).map((type) => (
                  <button
                    key={type}
                    type="button"
                    onClick={() =>
                      setDraft((d) => ({
                        ...d,
                        type,
                        category: categoriesFor(categories, type).some((c) => c.id === d.category)
                          ? d.category
                          : "",
                        payee: "",
                      }))
                    }
                    className={cn(
                      "h-11 min-h-11 flex-1 rounded-lg text-sm font-medium capitalize transition-colors duration-150
",
                      draft.type === type
                        ? "bg-card text-foreground shadow-[var(--shadow-border)]"
```

```
                          : "text-muted-foreground",
                      )}
                    >
                      {type}
                    </button>
                  ))}
                </div>

                <div className="grid gap-1.5">
                  <Label htmlFor="rec-amount">Amount (RON)</Label>
                  <Input
                    id="rec-amount"
                    inputMode="decimal"
                    placeholder="0.00"
                    value={draft.amount}
                    onChange={(e) => setDraft((d) => ({ ...d, amount: e.target.value }))}
                    className="font-mono tabular-nums"
                    autoComplete="off"
                  />
                </div>

                <div className="grid gap-1.5">
                  <Label>Category</Label>
                  {cats.length === 0 ? (
                    <p className="text-sm text-muted-foreground">
                      Add a {draft.type} category first.
                    </p>
                  ) : (
                    <div className="flex flex-wrap gap-1.5">
                      {cats.map((c) => (
                        <button
                          key={c.id}
                          type="button"
                          onClick={() =>
                            setDraft((d) => {
                              const names = payeeNames(payees, d.type, c.id);
                              return {
                                ...d,
                                category: c.id,
                                payee: names.some((n) => labelsEqual(n, d.payee)) ? d.payee : "",
                              };
                            })
                          }
                          className={cn(
                            "h-9 min-h-9 rounded-full border px-3 text-xs font-medium transition-colors duration-150",
                            draft.category === c.id
                              ? "border-primary bg-primary text-primary-foreground"
                              : "border-border bg-transparent text-muted-foreground hover:text-foreground",
                          )}
                        >
                          {c.label}
                        </button>
                      ))}
                    </div>
                  )}
                </div>

                <div className="grid gap-1.5">
                  <Label>Payee</Label>
                  {draft.category ? (
                    <PayeePicker
                      payees={payeeNames(payees, draft.type, draft.category)}
                      value={draft.payee}
                      onChange={(name) => setDraft((d) => ({ ...d, payee: name }))}
                      onAdd={(name) => addPayee(name, draft.type, draft.category)}
                      onDelete={(name) => deletePayee(name, draft.type, draft.category)}
                    />
                  ) : (
                    <p className="text-sm text-muted-foreground">Pick a category first. Payees are per category.</p>
                  )}
                </div>

                <div className="grid gap-1.5">
                  <Label>Repeats</Label>
                  <div className="flex flex-wrap gap-1.5">
                    {FREQUENCIES.map((freq) => (
                      <button
                        key={freq}
                        type="button"
                        onClick={() => setDraft((d) => ({ ...d, frequency: freq }))}
```

```
                        className={cn(
                          "h-9 min-h-9 rounded-full border px-3 text-xs font-medium transition-colors duration-150",
                          draft.frequency === freq
                            ? "border-primary bg-primary text-primary-foreground"
                            : "border-border bg-transparent text-muted-foreground hover:text-foreground",
                        )}
                      >
                        {FREQ_LABELS[freq]}
                      </button>
                    ))}
                  </div>
                </div>

                <div className="grid gap-1.5">
                  <Label htmlFor="rec-next">Next date</Label>
                  <Input
                    id="rec-next"
                    type="date"
                    value={draft.nextDate}
                    onChange={(e) => setDraft((d) => ({ ...d, nextDate: e.target.value }))}
                  />
                </div>

                <Toggle
                  checked={draft.autoPost}
                  onChange={(autoPost) => setDraft((d) => ({ ...d, autoPost }))}
                  label="Post automatically when due"
                />

                <div className="grid gap-1.5">
                  <Label htmlFor="rec-note">Note</Label>
                  <Input
                    id="rec-note"
                    placeholder="Optional"
                    value={draft.note}
                    onChange={(e) => setDraft((d) => ({ ...d, note: e.target.value }))}
                    maxLength={160}
                  />
                </div>

                {error ? <p className="text-sm text-expense">{error}</p> : null}
              </form>
              <DrawerFooter>
                <Button type="button" onClick={submit}>
                  {editingId ? "Save changes" : "Save recurring"}
                </Button>
                <Button type="button" variant="ghost" onClick={reset}>
                  Back
                </Button>
              </DrawerFooter>
            </>
          )}
        </DrawerContent>
      </Drawer>

      <AlertDialog open={pending !== null} onOpenChange={(o) => !o && setPending(null)}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>Delete {pending?.payee}?</AlertDialogTitle>
            <AlertDialogDescription>
              Future repeats stop. Transactions already posted stay in the ledger.
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Cancel</AlertDialogCancel>
            <AlertDialogAction
              className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
              onClick={() => {
                if (pending) deleteRecurrence(pending.id);
                setPending(null);
                toast.success("Recurring bill deleted");
              }}
            >
              Delete
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </>
  );
```

```
477   }
```

src/components/ledger/payee-picker.tsx

```tsx
import { useMemo, useState } from "react";
import { Check, ChevronDown, Plus, Trash2 } from "lucide-react";
import { labelsEqual, normalizeLabel } from "@/lib/ledger/categories";
import { cn } from "@/lib/utils";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";

export function PayeePicker({
  payees,
  value,
  onChange,
  onAdd,
  onDelete,
  noun = "payee",
  selectPlaceholder = "Select a payee",
}: {
  payees: string[];
  value: string;
  onChange: (name: string) => void;
  onAdd: (name: string) => string | null;
  onDelete?: (name: string) => void;
  noun?: string;
  selectPlaceholder?: string;
}) {
  const [open, setOpen] = useState(false);
  const [query, setQuery] = useState("");
  const [error, setError] = useState<string | null>(null);

  const filtered = useMemo(() => {
    const q = query.trim().toLowerCase();
    if (!q) return payees;
    return payees.filter((p) => p.toLowerCase().includes(q));
  }, [payees, query]);

  const canAdd = useMemo(() => {
    const cleaned = normalizeLabel(query);
    if (!cleaned) return false;
    return !payees.some((p) => labelsEqual(p, cleaned));
  }, [payees, query]);

  function select(name: string) {
    onChange(name);
    setQuery("");
    setError(null);
    setOpen(false);
  }

  function add() {
    const added = onAdd(query);
    if (!added) {
      setError("Enter a name.");
      return;
    }
    select(added);
  }

  return (
    <div className="grid gap-1.5">
      <button
        type="button"
        onClick={() => {
          setOpen((v) => !v);
          setError(null);
        }}
        className={cn(
          "flex h-11 min-h-11 w-full items-center justify-between rounded-lg border border-input bg-background px-3 te
xt-left text-sm shadow-[var(--shadow-border)] outline-none transition-[box-shadow,border-color] duration-150 focus-vis
ible:border-ring focus-visible:ring-2 focus-visible:ring-ring/40",
          value ? "text-foreground" : "text-subtle",
        )}
        aria-expanded={open}
        aria-haspopup="listbox"
      >
        <span className="truncate">{value || selectPlaceholder}</span>
        <ChevronDown className="size-4 shrink-0 text-subtle" />
      </button>

      {open ? (
```

```
        <div className="overflow-hidden rounded-xl bg-muted shadow-[var(--shadow-border)]">
          <div className="flex gap-2 p-2">
            <Input
              value={query}
              onChange={(e) => {
                setQuery(e.target.value);
                setError(null);
              }}
              placeholder={payees.length ? "Search or add" : `Type a ${noun} name`}
              autoComplete="off"
              className="h-10 min-h-10 bg-background"
              onKeyDown={(e) => {
                if (e.key === "Enter") {
                  e.preventDefault();
                  if (filtered.length === 1 && !canAdd) select(filtered[0]!);
                  else if (canAdd) add();
                }
                if (e.key === "Escape") setOpen(false);
              }}
            />
            <Button
              type="button"
              size="sm"
              variant={canAdd ? "default" : "outline"}
              disabled={!normalizeLabel(query)}
              onClick={add}
              className="shrink-0"
            >
              <Plus />
              Add
            </Button>
          </div>
          <ul className="max-h-48 overflow-y-auto px-1 pb-2" role="listbox">
            {filtered.length === 0 ? (
              <li className="px-3 py-3 text-sm text-muted-foreground">
                {payees.length === 0
                  ? `No ${noun}s yet. Type a name and tap Add.`
                  : canAdd
                    ? `Add “${normalizeLabel(query)}” to your list.`
                    : `No matching ${noun}.`}
              </li>
            ) : (
              filtered.map((name) => (
                <li key={name} className="flex items-center gap-1">
                  <button
                    type="button"
                    role="option"
                    aria-selected={value === name}
                    onClick={() => select(name)}
                    className={cn(
                      "flex min-h-11 min-w-0 flex-1 items-center justify-between rounded-lg px-3 text-left text-sm tra
nsition-colors duration-150 hover:bg-accent",
                      value === name && "bg-accent",
                    )}
                  >
                    <span className="truncate">{name}</span>
                    {value === name ? <Check className="size-4 shrink-0 text-foreground" /> : null}
                  </button>
                  {onDelete ? (
                    <Button
                      type="button"
                      variant="ghost"
                      size="sm"
                      className="shrink-0 text-expense"
                      aria-label={`Delete ${name}`}
                      onClick={(e) => {
                        e.preventDefault();
                        e.stopPropagation();
                        onDelete(name);
                        if (value === name) onChange("");
                      }}
                    >
                      <Trash2 />
                      Delete
                    </Button>
                  ) : null}
                </li>
              ))
            )}
          </ul>
```

```
          </div>
        ) : null}
        {error ? <p className="text-sm text-expense">{error}</p> : null}
      </div>
    );
  }
```

src/components/ledger/recurring-panel.tsx

```tsx
import { FREQ_LABELS } from "@/lib/ledger/recurrence";
import { formatDayYear, formatRon } from "@/lib/ledger/format";
import { categoryLabel } from "@/lib/ledger/categories";
import type { Category, RecurringBill } from "@/lib/ledger/types";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { cn } from "@/lib/utils";

function RowMeta({
  bill,
  categories,
}: {
  bill: RecurringBill;
  categories: Category[];
}) {
  return (
    <p className="truncate text-xs text-muted-foreground">
      {categoryLabel(bill.category, categories)} · {FREQ_LABELS[bill.frequency]}
    </p>
  );
}

export function RecurringPanel({
  due,
  upcoming,
  paused,
  categories,
  onManage,
  onPost,
  onSkip,
}: {
  due: RecurringBill[];
  upcoming: RecurringBill[];
  paused: RecurringBill[];
  categories: Category[];
  onManage: () => void;
  onPost: (id: string) => void;
  onSkip: (id: string) => void;
}) {
  const empty = due.length + upcoming.length + paused.length === 0;
  const rest = [...upcoming, ...paused].slice(0, 6);

  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: "180ms" }}>
      <CardHeader>
        <div className="flex items-start justify-between gap-3">
          <div className="min-w-0">
            <CardTitle>Recurring</CardTitle>
            <p className="mt-0.5 text-xs text-muted-foreground">
              {due.length > 0
                ? `${due.length} due`
                : upcoming.length > 0
                  ? `Next ${formatDayYear(upcoming[0]!.nextDate)}`
                  : "Salary, rent, bills"}
            </p>
          </div>
          <Button type="button" variant="ghost" size="sm" onClick={onManage}>
            Manage
          </Button>
        </div>
      </CardHeader>
      <CardContent className="pt-1">
        {empty ? (
          <div className="flex flex-col items-start gap-3 py-4">
            <p className="text-sm text-muted-foreground">
              Save a repeating income or bill. Ledger can post it when the date arrives, or wait
              for you to tap Post.
            </p>
            <Button type="button" size="sm" variant="outline" onClick={onManage}>
              Add recurring
            </Button>
          </div>
        ) : (
          <ul className="flex flex-col gap-1">
            {due.map((bill) => (
              <li key={bill.id} className="rounded-lg bg-muted px-3 py-2">
                <div className="flex items-start justify-between gap-3">
                  <div className="min-w-0">
```

```
                    <p className="truncate text-sm">{bill.payee}</p>
                    <RowMeta bill={bill} categories={categories} />
                  </div>
                  <span
                    className={cn(
                      "shrink-0 font-mono text-xs tabular-nums",
                      bill.type === "income" ? "text-income" : "text-foreground",
                    )}
                  >
                    {bill.type === "income" ? "+" : "−"}
                    {formatRon(bill.amount)}
                  </span>
                </div>
                <div className="mt-2 flex gap-2">
                  <Button
                    type="button"
                    size="sm"
                    className="h-9 min-h-9 flex-1"
                    onClick={() => onPost(bill.id)}
                  >
                    Post
                  </Button>
                  <Button
                    type="button"
                    size="sm"
                    variant="outline"
                    className="h-9 min-h-9 flex-1"
                    onClick={() => onSkip(bill.id)}
                  >
                    Skip
                  </Button>
                </div>
              </li>
            ))}
            {rest.map((bill) => (
              <li
                key={bill.id}
                className="flex min-h-11 items-center justify-between gap-3 rounded-lg px-2 py-1.5"
              >
                <div className="min-w-0">
                  <p className="truncate text-sm">
                    {bill.payee}
                    {bill.paused ? (
                      <span className="ml-2 text-xs text-subtle">Paused</span>
                    ) : null}
                  </p>
                  <p className="truncate text-xs text-muted-foreground">
                    {formatDayYear(bill.nextDate)} · {FREQ_LABELS[bill.frequency]}
                  </p>
                </div>
                <span
                  className={cn(
                    "shrink-0 font-mono text-xs tabular-nums",
                    bill.type === "income" ? "text-income" : "text-foreground",
                  )}
                >
                  {bill.type === "income" ? "+" : "−"}
                  {formatRon(bill.amount)}
                </span>
              </li>
            ))}
          </ul>
        )}
      </CardContent>
    </Card>
  );
}
```

src/components/ledger/sparkline.tsx

```tsx
import { cn } from "@/lib/utils";

export function Sparkline({
  values,
  className,
}: {
  values: number[];
  className?: string;
}) {
  if (values.length < 2) {
    return <div className={cn("h-9", className)} />;
  }
  const min = Math.min(...values);
  const max = Math.max(...values);
  const span = max - min || 1;
  const w = 120;
  const h = 36;
  const pts = values.map((v, i) => {
    const x = (i / (values.length - 1)) * w;
    const y = h - 3 - ((v - min) / span) * (h - 6);
    return [x, y] as const;
  });
  const line = pts
    .map(([x, y], i) => `${i === 0 ? "M" : "L"}${x.toFixed(1)} ${y.toFixed(1)}`)
    .join(" ");
  const area = `${line} L${w} ${h} L0 ${h} Z`;
  return (
    <svg
      viewBox={`0 0 ${w} ${h}`}
      className={cn("h-9 w-full text-primary", className)}
      aria-hidden="true"
    >
      <path d={area} fill="currentColor" opacity={0.12} />
      <path d={line} fill="none" stroke="currentColor" strokeWidth={1.5} />
    </svg>
  );
}
```

src/components/ledger/spend-mix.tsx

```tsx
import { Cell, Pie, PieChart, ResponsiveContainer, Tooltip } from "recharts";
import type { MixSlice } from "@/lib/ledger/analytics";
import { formatRon } from "@/lib/ledger/format";
import type { CategoryId } from "@/lib/ledger/types";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Skeleton } from "@/components/ui/skeleton";
import { cn } from "@/lib/utils";
import { ClientOnly } from "./client-only";
import { ChartTooltip } from "./chart-tooltip";

export function SpendMix({
  data,
  focus,
  onFocus,
}: {
  data: MixSlice[];
  focus: CategoryId | null;
  onFocus: (id: CategoryId) => void;
}) {
  const total = data.reduce((s, d) => s + d.value, 0);
  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: "200ms" }}>
      <CardHeader>
        <CardTitle>Spend mix</CardTitle>
      </CardHeader>
      <CardContent>
        {data.length === 0 ? (
          <p className="py-10 text-center text-sm text-muted-foreground">
            No expenses in this range. Add an expense to see the mix.
          </p>
        ) : (
          <div className="flex flex-col gap-4">
            <div
              className="relative mx-auto h-52 w-full max-w-xs overflow-hidden outline-none [&_svg]:outline-none"
              style={{ WebkitTapHighlightColor: "transparent" }}
            >
              <ClientOnly fallback={<Skeleton className="mx-auto size-52 rounded-full" />}>
                <ResponsiveContainer width="100%" height="100%">
                  <PieChart>
                    <Pie
                      data={data}
                      dataKey="value"
                      nameKey="label"
                      innerRadius="62%"
                      outerRadius="88%"
                      paddingAngle={1.5}
                      stroke="none"
                      isAnimationActive={false}
                      activeShape={false}
                      onClick={(_, index) => {
                        const slice = data[index];
                        if (slice) onFocus(slice.category);
                      }}
                      style={{ cursor: "pointer", outline: "none" }}
                    >
                      {data.map((slice) => (
                        <Cell
                          key={slice.category}
                          fill={slice.color}
                          stroke="none"
                          opacity={focus && focus !== slice.category ? 0.28 : 1}
                        />
                      ))}
                    </Pie>
                    <Tooltip content={ChartTooltip} cursor={false} wrapperStyle={{ outline: "none" }} />
                  </PieChart>
                </ResponsiveContainer>
              </ClientOnly>
              <div className="pointer-events-none absolute inset-0 flex flex-col items-center justify-center px-10 text-center">
                <span className="text-xs text-muted-foreground">Spend</span>
                <span className="font-mono text-sm tabular-nums text-foreground">
                  {formatRon(total)}
                </span>
              </div>
            </div>
            <ul className="space-y-1">
              {data.slice(0, 6).map((slice) => (
```

```
                <li key={slice.category}>
                  <button
                    type="button"
                    onClick={() => onFocus(slice.category)}
                    className={cn(
                      "flex min-h-11 w-full items-center gap-2 rounded-lg px-2 text-left transition-colors duration-150 hover:bg-accent",
                      focus === slice.category && "bg-accent",
                    )}
                  >
                    <span
                      className="size-2 shrink-0 rounded-full"
                      style={{ background: slice.color }}
                    />
                    <span className="min-w-0 flex-1 truncate text-sm">{slice.label}</span>
                    <span className="shrink-0 font-mono text-xs tabular-nums whitespace-nowrap text-muted-foreground">
                      {(slice.share * 100).toFixed(0)}%
                    </span>
                  </button>
                </li>
              ))}
            </ul>
          </div>
        )}
      </CardContent>
    </Card>
  );
}
```

src/components/ledger/transactions-table.tsx

```tsx
import { useMemo, useState } from "react";
import { FileSpreadsheet, FileUp, Search, Trash2 } from "lucide-react";
import { categoryLabel } from "@/lib/ledger/categories";
import { searchTransactions } from "@/lib/ledger/analytics";
import { formatDayYear, formatRonSigned } from "@/lib/ledger/format";
import type { Category, Transaction, TxType } from "@/lib/ledger/types";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Input } from "@/components/ui/input";
import { cn } from "@/lib/utils";

const TYPE_FILTERS: { id: "all" | TxType; label: string }[] = [
  { id: "all", label: "All" },
  { id: "income", label: "In" },
  { id: "expense", label: "Out" },
];

export function TransactionsTable({
  transactions,
  categories,
  onDelete,
  emptyHint,
  onExportCsv,
  onImportCsv,
  canExport,
}: {
  transactions: Transaction[];
  categories: Category[];
  onDelete: (id: string) => void;
  emptyHint?: string;
  onExportCsv?: () => void;
  onImportCsv?: () => void;
  canExport?: boolean;
}) {
  const [query, setQuery] = useState("");
  const [typeFilter, setTypeFilter] = useState<"all" | TxType>("all");
  const [pending, setPending] = useState<Transaction | null>(null);

  const rows = useMemo(
    () => searchTransactions(transactions, query, typeFilter, categories),
    [transactions, query, typeFilter, categories],
  );

  const noSource = transactions.length === 0;

  return (
    <Card className="stagger-in min-w-0" style={{ animationDelay: "320ms" }}>
      <CardHeader className="gap-3">
        <div className="flex items-center justify-between gap-3">
          <CardTitle>Transactions</CardTitle>
          <div className="flex items-center gap-1">
            {onImportCsv ? (
              <Button
                type="button"
                variant="ghost"
                size="sm"
                onClick={onImportCsv}
                aria-label="Import CSV"
              >
                <FileUp />
                Import
              </Button>
            ) : null}
            {onExportCsv ? (
              <Button
                type="button"
                variant="ghost"
                size="sm"
```

```
                onClick={onExportCsv}
                disabled={!canExport}
                aria-label="Export CSV"
              >
                <FileSpreadsheet />
                CSV
              </Button>
            ) : null}
            <span className="font-mono text-xs tabular-nums text-subtle">{rows.length}</span>
          </div>
        </div>
        <div className="flex flex-col gap-2 sm:flex-row">
          <div className="relative min-w-0 flex-1">
            <Search className="pointer-events-none absolute top-1/2 left-3 size-4 -translate-y-1/2 text-subtle" />
            <Input
              value={query}
              onChange={(e) => setQuery(e.target.value)}
              placeholder="Search payee, product, note, category"
              className="pl-9"
              aria-label="Search transactions"
            />
          </div>
          <div className="flex rounded-xl bg-muted p-1">
            {TYPE_FILTERS.map((f) => (
              <button
                key={f.id}
                type="button"
                onClick={() => setTypeFilter(f.id)}
                className={cn(
                  "h-9 min-h-9 min-w-11 flex-1 rounded-lg px-3 text-xs font-medium transition-colors duration-150 sm:flex-none",
                  typeFilter === f.id
                    ? "bg-card text-foreground shadow-[var(--shadow-border)]"
                    : "text-muted-foreground hover:text-foreground",
                )}
              >
                {f.label}
              </button>
            ))}
          </div>
        </div>
      </CardHeader>
      <CardContent className="pt-1">
        {rows.length === 0 ? (
          <p className="py-6 text-center text-sm text-muted-foreground">
            {noSource
              ? emptyHint ?? "No transactions yet."
              : "No matching transactions."}
          </p>
        ) : (
          <ul className="divide-y divide-border">
            {rows.slice(0, 80).map((tx) => (
              <li key={tx.id} className="flex items-center gap-2 py-2">
                <div className="min-w-0 flex-1">
                  <div className="flex items-baseline justify-between gap-3">
                    <p className="truncate text-sm">{tx.payee}</p>
                    <p
                      className={cn(
                        "shrink-0 font-mono text-sm tabular-nums",
                        tx.type === "income" ? "text-income" : "text-foreground",
                      )}
                    >
                      {formatRonSigned(tx.type === "income" ? tx.amount : -tx.amount)}
                    </p>
                  </div>
                  <p className="mt-0.5 truncate text-xs text-muted-foreground">
                    {categoryLabel(tx.category, categories)}
                    {tx.type === "income" && tx.product ? (
                      <span>
                        {" "}
                        · {tx.quantity != null ? `${tx.quantity} × ` : ""}
                        {tx.product}
                      </span>
                    ) : null}
                    <span className="text-subtle"> · {formatDayYear(tx.date)}</span>
                    {tx.note ? <span className="text-subtle"> · {tx.note}</span> : null}
                  </p>
                </div>
                <Button
                  type="button"
```

```
                  variant="ghost"
                  size="icon-sm"
                  aria-label={`Delete ${tx.payee}`}
                  onClick={() => setPending(tx)}
                  className="shrink-0 text-subtle hover:text-expense"
                >
                  <Trash2 />
                </Button>
              </li>
            ))}
          </ul>
        )}
        {rows.length > 80 ? (
          <p className="pt-3 text-center text-xs text-subtle">
            Showing 80 of {rows.length}. Narrow the search to see the rest.
          </p>
        ) : null}
      </CardContent>

      <AlertDialog open={pending !== null} onOpenChange={(o) => !o && setPending(null)}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>Delete transaction?</AlertDialogTitle>
            <AlertDialogDescription>
              {pending
                ? `${pending.payee} · ${categoryLabel(pending.category, categories)} will be removed from this device.
`
                : null}
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Cancel</AlertDialogCancel>
            <AlertDialogAction
              className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
              onClick={() => {
                if (pending) onDelete(pending.id);
                setPending(null);
              }}
            >
              Delete
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </Card>
  );
}
```

src/components/preview-host-bridge.tsx

```tsx
/**
 * Mount once in `__root.tsx` so the Grok preview chrome can drive navigation
 * (and later receive registered routes). Noops when the app is not embedded.
 */

import { useEffect } from "react";
import { useRouter } from "@tanstack/react-router";
import {
  collectRoutePathsFromTree,
  installPreviewHostBridge,
} from "@/lib/preview-host-bridge";

export function PreviewHostBridge() {
  const router = useRouter();

  useEffect(() => {
    return installPreviewHostBridge({
      navigate: (path) => {
        router.history.push(path);
      },
      getRoutePaths: () => collectRoutePathsFromTree(router.routeTree),
    });
  }, [router]);

  return null;
}
```

src/components/ui/alert-dialog.tsx

```tsx
import * as React from "react";
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog";
import { cn } from "@/lib/utils";
import { buttonVariants } from "./button";

function AlertDialog({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Root>) {
  return <AlertDialogPrimitive.Root data-slot="alert-dialog" {...props} />;
}

function AlertDialogTrigger({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Trigger>) {
  return <AlertDialogPrimitive.Trigger {...props} />;
}

function AlertDialogOverlay({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Overlay>) {
  return (
    <AlertDialogPrimitive.Overlay
      className={cn("fixed inset-0 z-[80] bg-background/80", className)}
      {...props}
    />
  );
}

function AlertDialogContent({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Content>) {
  return (
    <AlertDialogPrimitive.Portal>
      <AlertDialogOverlay />
      <AlertDialogPrimitive.Content
        className={cn(
          "fixed top-1/2 left-4 right-4 z-[80] max-w-md -translate-y-1/2 rounded-2xl bg-card p-5 text-card-foreground shadow-[var(--shadow-border)] sm:left-1/2 sm:right-auto sm:w-full sm:-translate-x-1/2",
          className,
        )}
        {...props}
      />
    </AlertDialogPrimitive.Portal>
  );
}

function AlertDialogHeader({ className, ...props }: React.ComponentProps<"div">) {
  return <div className={cn("flex flex-col gap-2", className)} {...props} />;
}

function AlertDialogFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      className={cn("mt-5 flex flex-col-reverse gap-2 sm:flex-row sm:justify-end", className)}
      {...props}
    />
  );
}

function AlertDialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Title>) {
  return (
    <AlertDialogPrimitive.Title
      className={cn("text-base font-medium", className)}
      {...props}
    />
  );
}

function AlertDialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Description>) {
  return (
```

```
    <AlertDialogPrimitive.Description
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  );
}

function AlertDialogAction({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Action>) {
  return (
    <AlertDialogPrimitive.Action className={cn(buttonVariants(), className)} {...props} />
  );
}

function AlertDialogCancel({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Cancel>) {
  return (
    <AlertDialogPrimitive.Cancel
      className={cn(buttonVariants({ variant: "outline" }), className)}
      {...props}
    />
  );
}

export {
  AlertDialog,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
  AlertDialogAction,
  AlertDialogCancel,
};
```

src/components/ui/badge.tsx

```tsx
import * as React from "react";
import { cva, type VariantProps } from "class-variance-authority";
import { cn } from "@/lib/utils";

const badgeVariants = cva(
  "inline-flex items-center gap-1 rounded-full border px-2.5 py-1 text-xs font-medium whitespace-nowrap",
  {
    variants: {
      variant: {
        default: "border-transparent bg-primary text-primary-foreground",
        steel: "border-border bg-secondary text-foreground",
        outline: "border-border text-foreground",
        income: "border-transparent bg-income/15 text-income",
        expense: "border-transparent bg-expense/15 text-expense",
      },
    },
    defaultVariants: { variant: "steel" },
  },
);

function Badge({
  className,
  variant,
  ...props
}: React.ComponentProps<"span"> & VariantProps<typeof badgeVariants>) {
  return (
    <span className={cn(badgeVariants({ variant }), className)} {...props} />
  );
}

export { Badge, badgeVariants };
```

src/components/ui/button.tsx

```tsx
import * as React from "react";
import { Slot } from "@radix-ui/react-slot";
import { cva, type VariantProps } from "class-variance-authority";
import { cn } from "@/lib/utils";

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium transition-[color,background-color,box-shadow,transform,opacity] duration-150 ease-out outline-none focus-visible:ring-2 focus-visible:ring-ring/60 focus-visible:ring-offset-2 focus-visible:ring-offset-background disabled:pointer-events-none disabled:opacity-50 [&_svg]:pointer-events-none [&_svg:not([class*='size-'])]:size-4 [&_svg]:shrink-0 active:not-disabled:scale-[0.96]",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/90",
        destructive:
          "bg-destructive text-destructive-foreground hover:bg-destructive/90",
        outline:
          "border border-border bg-transparent text-foreground hover:bg-accent",
        secondary: "bg-secondary text-secondary-foreground hover:bg-secondary/80",
        ghost: "text-foreground hover:bg-accent",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-11 min-h-11 px-4",
        sm: "h-9 min-h-9 rounded-md px-3 text-sm",
        lg: "h-12 min-h-12 rounded-lg px-5",
        icon: "size-11 min-h-11 min-w-11",
        "icon-sm": "size-9 min-h-9 min-w-9",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

type ButtonProps = React.ComponentProps<"button"> &
  VariantProps<typeof buttonVariants> & {
    asChild?: boolean;
  };

function Button({ className, variant, size, asChild = false, ...props }: ButtonProps) {
  const Comp = asChild ? Slot : "button";
  return (
    <Comp
      data-slot="button"
      className={cn(buttonVariants({ variant, size, className }))}
      {...props}
    />
  );
}

export { Button, buttonVariants };
```

src/components/ui/card.tsx

```tsx
import * as React from "react";
import { cn } from "@/lib/utils";

function Card({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card"
      className={cn(
        "rounded-2xl bg-card text-card-foreground shadow-[var(--shadow-border)]",
        className,
      )}
      {...props}
    />
  );
}

function CardHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-header"
      className={cn("flex flex-col gap-1 px-4 pt-4 sm:px-5 sm:pt-5", className)}
      {...props}
    />
  );
}

function CardTitle({ className, ...props }: React.ComponentProps<"h3">) {
  return (
    <h3
      data-slot="card-title"
      className={cn("text-sm font-medium tracking-tight text-foreground", className)}
      {...props}
    />
  );
}

function CardDescription({ className, ...props }: React.ComponentProps<"p">) {
  return (
    <p
      data-slot="card-description"
      className={cn("text-xs text-muted-foreground", className)}
      {...props}
    />
  );
}

function CardContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-content"
      className={cn("px-4 pb-4 sm:px-5 sm:pb-5", className)}
      {...props}
    />
  );
}

export { Card, CardHeader, CardTitle, CardDescription, CardContent };
```

src/components/ui/drawer.tsx

```tsx
import * as React from "react";
import { Drawer as DrawerPrimitive } from "vaul";
import { cn } from "@/lib/utils";

function Drawer({
  shouldScaleBackground = false,
  repositionInputs = false,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Root>) {
  return (
    <DrawerPrimitive.Root
      shouldScaleBackground={shouldScaleBackground}
      repositionInputs={repositionInputs}
      {...props}
    />
  );
}

function DrawerTrigger({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Trigger>) {
  return <DrawerPrimitive.Trigger {...props} />;
}

function DrawerPortal({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Portal>) {
  return <DrawerPrimitive.Portal {...props} />;
}

function DrawerClose({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Close>) {
  return <DrawerPrimitive.Close {...props} />;
}

function DrawerOverlay({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Overlay>) {
  return (
    <DrawerPrimitive.Overlay
      className={cn("fixed inset-0 z-50 bg-background/80", className)}
      {...props}
    />
  );
}

function useLiftForKeyboard(ref: React.RefObject<HTMLElement | null>) {
  React.useEffect(() => {
    const vv = window.visualViewport;
    if (!vv) return;

    const apply = () => {
      const el = ref.current;
      if (!el) return;
      const inset = Math.max(0, Math.round(window.innerHeight - vv.height - vv.offsetTop));
      const keyboard = inset > 80 ? inset : 0;
      el.style.setProperty("--keyboard-inset", `${keyboard}px`);
      if (keyboard > 0) {
        el.style.maxHeight = `${Math.round(vv.height)}px`;
        el.style.marginTop = "0px";
      } else {
        el.style.maxHeight = "";
        el.style.marginTop = "";
      }
    };

    apply();
    vv.addEventListener("resize", apply);
    vv.addEventListener("scroll", apply);
    window.addEventListener("focusin", apply);
    return () => {
      vv.removeEventListener("resize", apply);
      vv.removeEventListener("scroll", apply);
      window.removeEventListener("focusin", apply);
    };
  }, [ref]);
```

```
}

function scrollFocusedField(target: EventTarget | null) {
  if (!(target instanceof HTMLElement)) return;
  if (target.tagName !== "INPUT" && target.tagName !== "TEXTAREA") return;
  window.setTimeout(() => {
    target.scrollIntoView({ block: "center", inline: "nearest", behavior: "smooth" });
  }, 280);
}

function DrawerContent({
  className,
  children,
  onFocusCapture,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Content>) {
  const sheetRef = React.useRef<HTMLDivElement>(null);
  useLiftForKeyboard(sheetRef);

  return (
    <DrawerPortal>
      <DrawerOverlay />
      <DrawerPrimitive.Content
        ref={sheetRef}
        className={cn(
          "drawer-sheet fixed inset-x-0 bottom-0 z-50 mt-16 flex flex-col overflow-hidden rounded-t-2xl bg-card text-card-foreground shadow-[var(--shadow-border)] outline-none",
          className,
        )}
        {...props}
        onFocusCapture={(e) => {
          onFocusCapture?.(e);
          scrollFocusedField(e.target);
        }}
      >
        <div className="mx-auto mt-3 h-1 w-10 shrink-0 rounded-full bg-border" />
        {children}
      </DrawerPrimitive.Content>
    </DrawerPortal>
  );
}

function DrawerHeader({ className, ...props }: React.ComponentProps<"div">) {
  return <div className={cn("flex shrink-0 flex-col gap-1 px-5 pt-4 pb-2", className)} {...props} />;
}

function DrawerTitle({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Title>) {
  return (
    <DrawerPrimitive.Title
      className={cn("text-base font-medium", className)}
      {...props}
    />
  );
}

function DrawerDescription({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Description>) {
  return (
    <DrawerPrimitive.Description
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  );
}

function DrawerFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      className={cn(
        "flex shrink-0 flex-col gap-2 px-5 pt-2 safe-bottom",
        className,
      )}
      {...props}
    />
  );
```

```
}

export {
  Drawer,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,
  DrawerTitle,
  DrawerDescription,
};
```

src/components/ui/dropdown-menu.tsx

```tsx
import * as React from "react";
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu";
import { Check } from "lucide-react";
import { cn } from "@/lib/utils";

function DropdownMenu({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Root>) {
  return <DropdownMenuPrimitive.Root data-slot="dropdown-menu" {...props} />;
}

function DropdownMenuTrigger({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Trigger>) {
  return (
    <DropdownMenuPrimitive.Trigger data-slot="dropdown-menu-trigger" {...props} />
  );
}

function DropdownMenuContent({
  className,
  sideOffset = 6,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Content>) {
  return (
    <DropdownMenuPrimitive.Portal>
      <DropdownMenuPrimitive.Content
        sideOffset={sideOffset}
        className={cn(
          "z-50 min-w-48 overflow-hidden rounded-xl bg-popover p-1 text-popover-foreground shadow-[var(--shadow-border)]",
          className,
        )}
        {...props}
      />
    </DropdownMenuPrimitive.Portal>
  );
}

function DropdownMenuItem({
  className,
  variant = "default",
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Item> & {
  variant?: "default" | "destructive";
}) {
  return (
    <DropdownMenuPrimitive.Item
      className={cn(
        "relative flex min-h-11 cursor-pointer items-center gap-2 rounded-lg px-3 text-sm outline-none select-none focus:bg-accent data-disabled:pointer-events-none data-disabled:opacity-50 [&_svg]:size-4",
        variant === "destructive" && "text-expense focus:bg-expense/10",
        className,
      )}
      {...props}
    />
  );
}

function DropdownMenuSeparator({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Separator>) {
  return (
    <DropdownMenuPrimitive.Separator
      className={cn("-mx-1 my-1 h-px bg-border", className)}
      {...props}
    />
  );
}

function DropdownMenuLabel({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Label>) {
  return (
    <DropdownMenuPrimitive.Label
      className={cn("px-3 py-2 text-xs font-medium text-muted-foreground", className)}
```

```
      {...props}
    />
  );
}

function DropdownMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.CheckboxItem>) {
  return (
    <DropdownMenuPrimitive.CheckboxItem
      className={cn(
        "relative flex min-h-11 cursor-pointer items-center rounded-lg py-1.5 pr-3 pl-8 text-sm outline-none select-none focus:bg-accent",
        className,
      )}
      checked={checked}
      {...props}
    >
      <span className="absolute left-2 flex size-4 items-center justify-center">
        <DropdownMenuPrimitive.ItemIndicator>
          <Check className="size-4" />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </DropdownMenuPrimitive.CheckboxItem>
  );
}

export {
  DropdownMenu,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuSeparator,
  DropdownMenuLabel,
  DropdownMenuCheckboxItem,
};
```

src/components/ui/input.tsx

```tsx
import * as React from "react";
import { cn } from "@/lib/utils";

function Input({ className, type, ...props }: React.ComponentProps<"input">) {
  return (
    <input
      type={type}
      data-slot="input"
      className={cn(
        "flex h-12 w-full min-w-0 rounded-lg border border-input bg-background px-3 text-base text-foreground shadow-[var(--shadow-border)] outline-none transition-[box-shadow,border-color] duration-150 placeholder:text-subtle file:border-0 file:bg-transparent file:text-sm file:font-medium focus-visible:border-ring focus-visible:ring-2 focus-visible:ring-ring/40 disabled:cursor-not-allowed disabled:opacity-50 md:h-11 md:text-base",
        className,
      )}
      {...props}
    />
  );
}

export { Input };
```

src/components/ui/label.tsx

```tsx
import * as React from "react";
import * as LabelPrimitive from "@radix-ui/react-label";
import { cn } from "@/lib/utils";

function Label({ className, ...props }: React.ComponentProps<typeof LabelPrimitive.Root>) {
  return (
    <LabelPrimitive.Root
      data-slot="label"
      className={cn("text-xs font-medium text-muted-foreground", className)}
      {...props}
    />
  );
}

export { Label };
```

src/components/ui/separator.tsx

```tsx
import * as React from "react";
import * as SeparatorPrimitive from "@radix-ui/react-separator";
import { cn } from "@/lib/utils";

function Separator({
  className,
  orientation = "horizontal",
  decorative = true,
  ...props
}: React.ComponentProps<typeof SeparatorPrimitive.Root>) {
  return (
    <SeparatorPrimitive.Root
      decorative={decorative}
      orientation={orientation}
      className={cn(
        "shrink-0 bg-border",
        orientation === "horizontal" ? "h-px w-full" : "h-full w-px",
        className,
      )}
      {...props}
    />
  );
}

export { Separator };
```

src/components/ui/skeleton.tsx

```tsx
import { cn } from "@/lib/utils";

function Skeleton({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      className={cn("animate-pulse rounded-lg bg-muted", className)}
      {...props}
    />
  );
}

export { Skeleton };
```

src/components/ui/tooltip.tsx

```tsx
import * as React from "react";
import * as TooltipPrimitive from "@radix-ui/react-tooltip";
import { cn } from "@/lib/utils";

function TooltipProvider({
  delayDuration = 200,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Provider>) {
  return <TooltipPrimitive.Provider delayDuration={delayDuration} {...props} />;
}

function Tooltip({ ...props }: React.ComponentProps<typeof TooltipPrimitive.Root>) {
  return <TooltipPrimitive.Root data-slot="tooltip" {...props} />;
}

function TooltipTrigger({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Trigger>) {
  return <TooltipPrimitive.Trigger data-slot="tooltip-trigger" {...props} />;
}

function TooltipContent({
  className,
  sideOffset = 6,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Content>) {
  return (
    <TooltipPrimitive.Portal>
      <TooltipPrimitive.Content
        sideOffset={sideOffset}
        className={cn(
          "z-50 max-w-xs rounded-lg bg-popover px-2.5 py-1.5 text-xs text-popover-foreground shadow-[var(--shadow-bord
er)]",
          className,
        )}
        {...props}
      />
    </TooltipPrimitive.Portal>
  );
}

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider };
```

src/lib/auth/client.ts

```ts
import { genericOAuthClient } from "better-auth/client/plugins";
import { createAuthClient } from "better-auth/react";
import { runPreSignInSignOut, runSignOut } from "../../../scripts/sign-out-plan.mjs";
import { GROK_PROVIDERS } from "./providers";

/**
 * Better Auth client for this React SPA (browser-side).
 *
 * Talks to this app's OWN Better Auth at same-origin `/api/auth/*`. In the live
 * preview the app is an embedded iframe with PARTITIONED cookies, so after a
 * popup sign-in it can't read the session cookie — it authenticates with a
 * bearer token instead (captured from the popup, see `signIn`). The `onRequest`
 * hook attaches that token when present; when deployed (cookie auth) no token
 * is stored, so nothing changes.
 *
 * To sign out call `signOut()` below, NOT `authClient.signOut()`: the raw call
 * leaves the bearer token in place, and `onRequest` keeps re-attaching it, so
 * the visitor stays signed in.
 */
export const authClient = createAuthClient({
  plugins: [genericOAuthClient()],
  fetchOptions: {
    onRequest(ctx) {
      const token = getBearerToken();
      if (token) ctx.headers.set("Authorization", `Bearer ${token}`);
      return ctx;
    },
  },
});

/**
 * True when sign-in UI should be shown — i.e. whenever `VITE_AUTH_ENABLED` is
 * not `"false"`. The shipped template sets it to `"false"`
 * (`.grok/app-env.json`), which selects the dev user (see `use-current-user`);
 * with the key removed, sign-in is real in preview (baked preview client) and
 * when deployed (injected per-app client).
 */
export const authEnabled = import.meta.env.VITE_AUTH_ENABLED !== "false";

/** The upstream providers to render sign-in buttons for. */
export { GROK_PROVIDERS };

// ── Live-preview bearer token ──────────────────────────────────────────
// The embedded preview iframe has partitioned cookies, so we keep the session's
// bearer token in sessionStorage and attach it to every Better Auth request (and
// to server functions, via `@/lib/auth/middleware`). Empty everywhere except the
// preview after a popup sign-in, so the cookie path is untouched elsewhere.
const BEARER_KEY = "grok-auth.bearer-token";

/** The stored preview bearer token, or null. */
export function getBearerToken(): string | null {
  if (typeof window === "undefined") return null;
  try {
    return window.sessionStorage.getItem(BEARER_KEY);
  } catch {
    return null;
  }
}

function setBearerToken(token: string | null): void {
  if (typeof window === "undefined") return;
  try {
    if (token) window.sessionStorage.setItem(BEARER_KEY, token);
    else window.sessionStorage.removeItem(BEARER_KEY);
  } catch {
    /* storage unavailable — ignore */
  }
}

/**
 * The sandbox live preview runs this app inside an iframe on a `*.grok-sandbox.com`
 * host, where a full-page redirect to the broker can't work — so sign-in uses a
 * popup there and a normal redirect everywhere else.
 */
function inLivePreview(): boolean {
  return (
    typeof window !== "undefined" &&
    window.location.hostname.endsWith(".grok-sandbox.com")
```

```
  );
}

/** Message the popup posts back to the opener once sign-in completes. */
type PopupMessage = { source: "grok-auth-popup"; token: string | null; error?: string };

/**
 * Start sign-in with one upstream provider (`providerId` from `GROK_PROVIDERS`),
 * federating through the Grok auth broker.
 *
 * - **Live preview** (`*.grok-sandbox.com` iframe): opens a POPUP to
 *   `/auth/popup`, served by the template Vite plugin (see `vite.config.ts` +
 *   `popup.server.ts`) — 302s to the broker/upstream login (no app chrome) and,
 *   on return, posts the session bearer token back. We store it and refresh the
 *   session; no top-level navigation of the iframe to the broker.
 * - **Deployed** (and local non-iframe): a normal full-page redirect into the broker.
 *
 * Either way it clears any existing local session FIRST so switching providers
 * actually switches identity.
 */
export async function signIn(
  providerId: string,
  opts: { callbackURL?: string; errorCallbackURL?: string } = {},
): Promise<void> {
  const callbackURL = opts.callbackURL ?? "/";
  const errorCallbackURL = opts.errorCallbackURL ?? "/";

  // Open the popup SYNCHRONOUSLY on the user gesture — before any await
  // (including signOut). Awaiting first drops user-gesture privilege in some
  // browsers when the opener is a cross-origin live-preview iframe.
  const popup = inLivePreview() ? openSignInPopup(providerId) : null;

  // Clear any prior session so switching providers actually switches identity.
  // Bounded because the popup is already open — a request that never settles
  // would leave it hanging — but bounded PER ENVIRONMENT: only the server can
  // end a deployed session, so cutting it short at the preview's 1.5s would
  // start OAuth with the old session still live.
  await runPreSignInSignOut({
    livePreview: inLivePreview(),
    hasBearer: Boolean(getBearerToken()),
    requestSignOut: () => authClient.signOut(),
    clearToken: () => setBearerToken(null),
  });

  if (inLivePreview()) {
    if (!popup) throw new Error("Pop-up blocked — allow pop-ups for sign-in");
    const token = await waitForPopupToken(popup);
    if (!token) throw new Error("Sign-in was cancelled or failed");
    setBearerToken(token);
    // Refresh the client session store with the bearer attached (onRequest).
    // Avoid a full iframe reload when we're already on the destination — that
    // reload was the slow "still loading after the popup closed" feeling.
    try {
      await authClient.getSession();
    } catch {
      /* session store will recover on next useSession fetch */
    }
    if (typeof window !== "undefined") {
      const dest = new URL(callbackURL, window.location.origin);
      const here = window.location;
      if (dest.origin !== here.origin || dest.pathname !== here.pathname || dest.search !== here.search) {
        window.location.href = callbackURL;
      }
    }
    return;
  }

  const { data, error } = await authClient.signIn.oauth2({
    providerId,
    callbackURL,
    errorCallbackURL,
  });
  if (error) throw new Error(error.message ?? "Sign-in failed");
  if (data?.url) window.location.href = data.url;
}

/**
 * Open `/auth/popup` in a new window. Must run synchronously inside the click
 * handler (no await before this). The path is served by the template Vite
 * plugin (`authPopupPlugin` in vite.config.ts) — NOT by a React route.
```

```
 *
 * Opens the real URL directly (not about:blank → assign). From a cross-origin
 * iframe the about:blank dance often fails on the first click and the window
 * ends up showing the app shell.
 */
function openSignInPopup(providerId: string): Window | null {
  const origin = window.location.origin;
  const url = `${origin}/auth/popup?providerId=${encodeURIComponent(providerId)}`;
  // Unique name per attempt so a prior attempt stuck on the SPA is not reused.
  const name = `grok-signin-${Date.now()}`;
  return window.open(url, name, "popup,width=500,height=650");
}

/**
 * Wait for the popup's completion page to postMessage the session bearer (or
 * for the user to dismiss the popup).
 */
function waitForPopupToken(popup: Window): Promise<string | null> {
  return new Promise((resolve) => {
    const origin = window.location.origin;
    let settled = false;
    let closeTimer: number | undefined;
    const settle = (token: string | null) => {
      if (settled) return;
      settled = true;
      cleanup();
      resolve(token);
    };
    const onMessage = (event: MessageEvent) => {
      if (event.origin !== origin) return;
      const data = event.data as PopupMessage | undefined;
      if (!data || data.source !== "grok-auth-popup") return;
      settle(data.token ?? null);
    };
    // Fallback when the user dismisses the popup. Grace period lets the
    // completion page's postMessage win over a racing `popup.closed`.
    const pollTimer = window.setInterval(() => {
      if (!popup.closed) return;
      window.clearInterval(pollTimer);
      closeTimer = window.setTimeout(() => settle(null), 400);
    }, 300);
    function cleanup() {
      window.clearInterval(pollTimer);
      if (closeTimer !== undefined) window.clearTimeout(closeTimer);
      window.removeEventListener("message", onMessage);
    }
    window.addEventListener("message", onMessage);
  });
}

/**
 * Sign out of THIS app's local session, clear the preview token, then redirect.
 *
 * Use this, never `authClient.signOut()` — see the note on `authClient`.
 * Sequencing lives in `scripts/sign-out-plan.mjs` so it can be unit-tested.
 *
 * **Rejects when deployed if the server never confirms.** There the session is
 * an HttpOnly cookie only the server can clear, so redirecting anyway would
 * report a sign-out that did not happen. `<UserButton />` handles that for you;
 * a hand-rolled control must catch it and let the visitor retry. In the live
 * preview the local clear is sufficient, so it always resolves.
 */
export async function signOut(redirectTo = "/"): Promise<void> {
  await runSignOut({
    livePreview: inLivePreview(),
    hasBearer: Boolean(getBearerToken()),
    // Better Auth resolves with `{ error }` instead of rejecting, so surface a
    // failed response as a rejection for the sequence to act on.
    requestSignOut: async () => {
      const { error } = await authClient.signOut();
      if (error) throw new Error(error.message ?? "Sign-out failed");
    },
    clearToken: () => setBearerToken(null),
    redirect: () => {
      window.location.href = redirectTo;
    },
  });
}
```

src/lib/auth/email-password.ts

```
/**
 * Local email/password sign-in (this app's Better Auth DB — not the broker).
 *
 * Off by default. To enable: set `emailAndPasswordEnabled` to `true` below,
 * then build sign-up / sign-in forms with `authClient.signUp.email` /
 * `authClient.signIn.email` from `@/lib/auth/client` (see the auth skill).
 *
 * Do NOT edit `server.ts` for this — that file is frozen pre-wired config.
 */
export const emailAndPasswordEnabled = false;
```

src/lib/auth/gates.tsx

```tsx
import { useState, type ReactNode } from "react";
import { Navigate } from "@tanstack/react-router";
import { authEnabled, signOut } from "./client";
import { useCurrentUser, useCurrentUserState } from "./use-current-user";

/**
 * Auth state components — plain wrappers around `useCurrentUserState()`.
 *
 * With auth on, visitors are signed out until they authenticate — in the sandbox
 * live preview too, which does real sign-in. The shared dev user appears only
 * when auth is disabled (`VITE_AUTH_ENABLED=false`, the shipped default).
 * While the session is still resolving, gates that care about signed-out state
 * render nothing so there's no signed-out flash on hard reload.
 */

/** Where `RedirectToSignIn` sends signed-out visitors. Create this route. */
export const SIGN_IN_PATH = "/login";

/** Render children only when a user is present (real session, or the disabled-auth dev user). */
export function SignedIn({ children }: { children: ReactNode }) {
  const { user } = useCurrentUserState();
  return user ? <>{children}</> : null;
}

/**
 * Render children only once we KNOW the visitor is signed out (`isPending` has
 * cleared and there is no user). Hidden while the session is still loading.
 */
export function SignedOut({ children }: { children: ReactNode }) {
  const { user, isPending } = useCurrentUserState();
  if (isPending || user) return null;
  return <>{children}</>;
}

/**
 * Client-side redirect to the sign-in route (TanStack `<Navigate>` — NOT a full
 * `window.location` reload). A hard navigation re-bootstraps the SPA and re-runs
 * session loading, which feels like a second "Loading…" on /login.
 *
 * Guard routes by waiting out `isPending` first (see `use-current-user`), then
 * render this.
 */
export function RedirectToSignIn({ to = SIGN_IN_PATH }: { to?: string }) {
  return <Navigate to={to} />;
}

/**
 * Minimal signed-in identity chip + sign-out. Restyle freely (see the
 * `design-ui` skill). Sign-out is only shown when auth is enabled (the
 * disabled-auth dev user has nothing to sign out of).
 */
export function UserButton() {
  const user = useCurrentUser();
  // Sign-out can take a moment (and can fail when deployed), so the control
  // shows it is working and cannot be fired twice.
  const [signingOut, setSigningOut] = useState(false);
  if (!user) return null;
  const label = user.displayName ?? user.primaryEmail ?? "Account";
  return (
    <div className="flex items-center gap-2">
      {user.profileImageUrl ? (
        <img
          src={user.profileImageUrl}
          alt=""
          className="h-8 w-8 rounded-full object-cover"
        />
      ) : (
        <span className="grid h-8 w-8 place-items-center rounded-full bg-black/10 text-sm font-medium dark:bg-white/20
">
          {label.charAt(0).toUpperCase()}
        </span>
      )}
      <span className="text-sm font-medium">{label}</span>
      {authEnabled && (
        <button
          type="button"
          disabled={signingOut}
          onClick={() => {
```

```
78              setSigningOut(true);
79              // Success navigates away; on failure re-enable so it can be retried.
80              void signOut().catch(() => setSigningOut(false));
81            }}
82            className="cursor-pointer text-sm underline-offset-4 opacity-70 hover:underline disabled:cursor-wait disable
    d:no-underline"
83          >
84            {signingOut ? "Signing out…" : "Sign out"}
85          </button>
86        )}
87      </div>
88    );
89  }
```

src/lib/auth/isolation.server.ts

```ts
import { getRequest } from "@tanstack/react-start/server";

/**
 * Fetch-Metadata sibling isolation — **server-only** (`.server.ts` suffix).
 *
 * MUST keep the `.server` suffix: this file imports `@tanstack/react-start/server`
 * (`getRequest` → Node `AsyncLocalStorage`). If it is imported from a dual
 * client/server module under a non-`.server` name, Vite ships it to the browser
 * and the app dies with: `AsyncLocalStorage is not a constructor`.
 *
 * Apps deployed on `*.grok.me` are "same-site" to each other but MUTUALLY
 * UNTRUSTED, and a `SameSite=Lax` session cookie IS sent on same-site
 * subrequests — so without this, a malicious sibling could make a SCRIPTED
 * (fetch/XHR/form-POST) request to this app's server functions and ride this
 * app's session cookie.
 *
 * We allow only: same-origin requests (this app's own client), non-browser
 * requests (SSR / server-to-server, which send no `Sec-Fetch-Site`), and
 * top-level GET navigations (how the OAuth callback and normal page loads
 * arrive). Every cross-site / same-site *scripted* request is rejected.
 * Together with `__Host-` cookies and Better Auth's `trustedOrigins`, this
 * closes the sibling-tenant attack surface. Enforced at the `authMiddleware`
 * chokepoint (see `middleware.ts`).
 */
export class CrossSiteRequestError extends Error {
  readonly status = 403;
  constructor() {
    super("Forbidden: cross-site request blocked");
    this.name = "CrossSiteRequestError";
  }
}

/** Throw `CrossSiteRequestError` for a scripted cross-site/sibling request. */
export function assertSameSiteRequest(): void {
  const request = getRequest();
  if (!request) return; // no request context (e.g. build) — nothing to guard
  const h = request.headers;
  const site = h.get("sec-fetch-site");
  // Non-browser client (no header), the app's own origin, or a direct
  // (address-bar/bookmark) load are all fine.
  if (!site || site === "same-origin" || site === "none") return;
  // A top-level GET navigation (e.g. the broker's OAuth callback redirect) is
  // fine even when it's cross-site; scripted requests never set navigate mode.
  const dest = h.get("sec-fetch-dest");
  const isTopLevelGet =
    h.get("sec-fetch-mode") === "navigate" &&
    request.method === "GET" &&
    dest !== "object" &&
    dest !== "embed";
  if (isTopLevelGet) return;
  throw new CrossSiteRequestError();
}
```

src/lib/auth/middleware.ts

```ts
import { createMiddleware } from "@tanstack/react-start";

/**
 * Auth middleware for server functions — the standard way to get the caller's
 * verified user id. When deployed the session cookie is same-origin and rides
 * along automatically. In the live preview the client also forwards the bearer
 * token (partitioned cookies) via the `.client` hook below — call sites do not
 * thread it themselves.
 *
 *   import { createServerFn } from "@tanstack/react-start";
 *   import { getSql } from "@/lib/db";
 *   import { authMiddleware } from "@/lib/auth/middleware";
 *
 *   export const listTodos = createServerFn({ method: "GET" })
 *     .middleware([authMiddleware])
 *     .handler(async ({ context }) => {
 *       const sql = await getSql();
 *       return sql`select * from todos where user_id = ${context.userId}`;
 *     });
 *
 * Signed out with auth on (live preview included) -> throws `UnauthorizedError`
 * (see `verify.server.ts`). With auth disabled (`VITE_AUTH_ENABLED=false`, the
 * shipped default) it resolves the shared dev user — but throws instead when a
 * `DATABASE_URL` is also set, so an app without sign-in must not use this at
 * all. On the auth-on path, use it on every server function that touches
 * per-user data and scope every query by `context.userId`.
 */
export const authMiddleware = createMiddleware({ type: "function" })
  .client(async ({ next }) => {
    // Live preview (partitioned iframe): the session rides a bearer token, not a
    // cookie, so forward it to the server. Null when deployed (cookie auth), so
    // this is a no-op there.
    const { getBearerToken } = await import("./client");
    return next({ sendContext: { bearerToken: getBearerToken() ?? undefined } });
  })
  .server(async ({ next, context }) => {
    // ONLY import `*.server` modules here. This file is dual client/server
    // (bearer hook on the client). A plain `./isolation` path was renamed to
    // `isolation.server.ts` — keep this import in sync so image `tsc` resolves
    // it, and so Vite does not ship `@tanstack/react-start/server` to the browser.
    const { assertSameSiteRequest } = await import("./isolation.server");
    const { requireUserId } = await import("./verify.server");
    // Reject scripted cross-site/sibling requests before touching per-user data.
    assertSameSiteRequest();
    const userId = await requireUserId(context.bearerToken);
    return next({ context: { userId } });
  });
```

src/lib/auth/pglite-dialect.ts

```ts
/**
 * Kysely dialect for Better Auth over the app's embedded PGLite instance.
 * Lazy: resolves `getClient` on first connection so migrations can finish first.
 */
import type { PGlite } from "@electric-sql/pglite";
import {
  CompiledQuery,
  type DatabaseConnection,
  type DatabaseIntrospector,
  type Dialect,
  type Driver,
  type Kysely,
  PostgresAdapter,
  PostgresIntrospector,
  PostgresQueryCompiler,
  type QueryCompiler,
  type QueryResult,
  type TransactionSettings,
} from "kysely";

type Client = PGlite;

/** Factory used by `auth/server.ts`: `pgliteDialect(() => getPglite())`. */
export function pgliteDialect(
  getClient: () => Promise<Client> | Client,
): Dialect {
  return {
    createAdapter: () => new PostgresAdapter(),
    createDriver: () => new LazyPGliteDriver(getClient),
    createQueryCompiler: (): QueryCompiler => new PostgresQueryCompiler(),
    createIntrospector: (db: Kysely<unknown>): DatabaseIntrospector =>
      new PostgresIntrospector(db),
  };
}

class LazyPGliteDriver implements Driver {
  private client: Client | undefined;
  private connection: PGliteConnection | undefined;
  private queue: Array<(con: PGliteConnection) => void> = [];

  constructor(private readonly getClient: () => Promise<Client> | Client) {}

  async init(): Promise<void> {
    this.client = await this.getClient();
  }

  async acquireConnection(): Promise<DatabaseConnection> {
    if (this.client === undefined) {
      this.client = await this.getClient();
    }
    if (this.connection !== undefined) {
      return new Promise((resolve) => {
        this.queue.push(resolve);
      });
    }
    this.connection = new PGliteConnection(this.client);
    return this.connection;
  }

  async releaseConnection(connection: DatabaseConnection): Promise<void> {
    if (connection !== this.connection) {
      throw new Error("Invalid connection");
    }
    const next = this.queue.shift();
    if (next === undefined) {
      this.connection = undefined;
      return;
    }
    next(this.connection);
  }

  async beginTransaction(
    conn: DatabaseConnection,
    settings: TransactionSettings,
  ): Promise<void> {
    const c = conn as PGliteConnection;
    if (settings.isolationLevel) {
      await c.executeQuery(
```

```
        CompiledQuery.raw(
          `start transaction isolation level ${settings.isolationLevel}`,
        ),
      );
    } else {
      await c.executeQuery(CompiledQuery.raw("begin"));
    }
  }

  async commitTransaction(conn: DatabaseConnection): Promise<void> {
    await (conn as PGliteConnection).executeQuery(CompiledQuery.raw("commit"));
  }

  async rollbackTransaction(conn: DatabaseConnection): Promise<void> {
    await (conn as PGliteConnection).executeQuery(
      CompiledQuery.raw("rollback"),
    );
  }

  async destroy(): Promise<void> {
    // Do not close the client: it is the shared getPglite() singleton used by
    // app SQL (getSql). Only drop our local handle so auth teardown cannot
    // poison the rest of the process.
    this.client = undefined;
    this.connection = undefined;
    this.queue = [];
  }
}

class PGliteConnection implements DatabaseConnection {
  constructor(private readonly client: Client) {}

  async executeQuery<O>(compiledQuery: CompiledQuery): Promise<QueryResult<O>> {
    const result = await this.client.query(compiledQuery.sql, [
      ...compiledQuery.parameters,
    ]);
    if (result.affectedRows) {
      return {
        numAffectedRows: BigInt(result.affectedRows),
        rows: result.rows as O[],
      };
    }
    return { rows: result.rows as O[] };
  }

  async *streamQuery<O>(
    compiledQuery: CompiledQuery,
    chunkSize: number,
  ): AsyncIterableIterator<QueryResult<O>> {
    if (!Number.isInteger(chunkSize) || chunkSize <= 0) {
      throw new Error("chunkSize must be a positive integer");
    }
    const result = await this.client.query(compiledQuery.sql, [
      ...compiledQuery.parameters,
    ]);
    for (let i = 0; i < result.rows.length; i += chunkSize) {
      yield { rows: result.rows.slice(i, i + chunkSize) as O[] };
    }
  }
}
```

src/lib/auth/popup.server.ts

```ts
/**
 * Live-preview sign-in popup — server-only (NEVER import from the client).
 *
 * The sandbox preview runs the app in a partitioned iframe, so OAuth must happen
 * in a top-level popup (first-party cookies). This handler is the ENTIRE popup
 * document — no React shell:
 *
 *   Phase 1 (`?providerId=…`): start OAuth server-side and 302 straight to the
 *     broker / upstream login page. The popup never paints the app.
 *   Phase 2 (`?done=1`): after the broker round-trip, emit a tiny HTML page that
 *     posts the session token to the opener and closes. No SPA hydrate, no
 *     server-fn round-trip.
 *
 * Wired automatically by the Vite `authPopupPlugin` in `vite.config.ts` during
 * `npm run dev` (live preview). Do NOT create `src/routes/auth/popup.tsx` — a
 * React route here paints the full app shell in the popup. The opener lives in
 * `client.ts` (`signIn` → `openSignInPopup`).
 */
import { auth, SESSION_TOKEN_COOKIE } from "./server";

/** Message shape the popup posts to the opener (must match `client.ts`). */
type PopupMessage = {
  source: "grok-auth-popup";
  token: string | null;
  error?: string;
};

/**
 * Handle `GET /auth/popup`. Invoked by the Vite `authPopupPlugin` (dev / live
 * preview). Do not re-export this from a React route file.
 */
export async function handleAuthPopupRequest(request: Request): Promise<Response> {
  const url = new URL(request.url);
  const done = url.searchParams.get("done") === "1";

  if (done) {
    const errored = url.searchParams.has("error");
    const token = errored ? null : readCookie(request, SESSION_TOKEN_COOKIE);
    const message: PopupMessage = {
      source: "grok-auth-popup",
      token,
      ...(errored ? { error: url.searchParams.get("error") ?? "sign_in_failed" } : {}),
    };
    return new Response(completionHtml(message), {
      status: 200,
      headers: {
        "content-type": "text/html; charset=utf-8",
        // Never cache a page that embeds a session token.
        "cache-control": "no-store",
      },
    });
  }

  const providerId = url.searchParams.get("providerId")?.trim();
  if (!providerId) {
    return new Response("Missing providerId", {
      status: 400,
      headers: { "content-type": "text/plain; charset=utf-8" },
    });
  }

  // Stay first-party for the callback so the session cookie lands in THIS popup.
  const back = `${url.origin}/auth/popup?done=1`;
  try {
    const apiRes = await auth.api.signInWithOAuth2({
      body: {
        providerId,
        callbackURL: back,
        errorCallbackURL: `${back}&error=1`,
      },
      // Forward the preview host so Better Auth derives the correct baseURL /
      // redirect_uri for the dynamic `*.grok-sandbox.com` origin.
      headers: request.headers,
      asResponse: true,
    });

    if (!apiRes.ok) {
      const detail = await apiRes.text().catch(() => "");
```

```
      return completionResponse({
        source: "grok-auth-popup",
        token: null,
        error: detail || `oauth_init_failed_${apiRes.status}`,
      });
    }

    const body = (await apiRes.json().catch(() => null)) as {
      url?: string;
    } | null;
    const location = body?.url;
    if (!location) {
      return completionResponse({
        source: "grok-auth-popup",
        token: null,
        error: "oauth_init_missing_url",
      });
    }

    // 302 to the broker (which headlessly forwards to Google/X). Forward any
    // Set-Cookie (OAuth state / PKCE) so the callback can complete in this popup.
    const headers = new Headers({ location, "cache-control": "no-store" });
    for (const cookie of apiRes.headers.getSetCookie()) {
      headers.append("set-cookie", cookie);
    }
    return new Response(null, { status: 302, headers });
  } catch (err) {
    const message = err instanceof Error ? err.message : "oauth_init_threw";
    return completionResponse({
      source: "grok-auth-popup",
      token: null,
      error: message,
    });
  }
}

function completionResponse(message: PopupMessage): Response {
  return new Response(completionHtml(message), {
    status: 200,
    headers: {
      "content-type": "text/html; charset=utf-8",
      "cache-control": "no-store",
    },
  });
}

/** Minimal HTML: postMessage the token to the opener and close. No React. */
function completionHtml(message: PopupMessage): string {
  // JSON is safe inside a <script type="application/json"> block; the inline
  // script only reads it. Avoids escaping pitfalls of embedding in JS source.
  const payload = JSON.stringify(message).replace(/</g, "\\u003c");
  return `<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Signing in…</title>
<style>
  html,body{margin:0;min-height:100%;background:#0b0b0c;color:#a1a1aa;
    font:14px/1.5 -apple-system,BlinkMacSystemFont,"Segoe UI",Roboto,Helvetica,Arial,sans-serif}
  main{min-height:100vh;display:grid;place-items:center;padding:1.5rem;text-align:center}
</style>
</head>
<body>
<main><p>Signing you in…</p></main>
<script type="application/json" id="grok-auth-popup-msg">${payload}</script>
<script>
(function () {
  var el = document.getElementById("grok-auth-popup-msg");
  var msg = { source: "grok-auth-popup", token: null };
  try { if (el && el.textContent) msg = JSON.parse(el.textContent); } catch (e) {}
  try {
    if (window.opener) window.opener.postMessage(msg, window.location.origin);
  } catch (e) {}
  try { window.close(); } catch (e) {}
})();
</script>
</body>
</html>`;
}
```

```
159
160 /** Read a single cookie value from the request (handles `=` inside values). */
161 function readCookie(request: Request, name: string): string | null {
162   const header = request.headers.get("cookie");
163   if (!header) return null;
164   for (const part of header.split(";")) {
165     const trimmed = part.trim();
166     if (!trimmed) continue;
167     const eq = trimmed.indexOf("=");
168     if (eq <= 0) continue;
169     if (trimmed.slice(0, eq) !== name) continue;
170     const raw = trimmed.slice(eq + 1);
171     try {
172       return decodeURIComponent(raw);
173     } catch {
174       return raw;
175     }
176   }
177   return null;
178 }
```

src/lib/auth/preview.ts

```ts
/**
 * Shared LIVE-PREVIEW OAuth client (server-only — NEVER import from the client).
 *
 * The sandbox serves each live preview on a dynamic `https://*.grok-sandbox.com`
 * URL, which can't be pre-registered per app. The broker instead exposes ONE
 * shared "preview" client that accepts any
 * `https://*.grok-sandbox.com/api/auth/oauth2/callback/*`
 * (broker: `app-builder-deployer/auth/src/preview-oauth.ts`). Baking it here lets
 * the live preview do REAL sign-in — no demo/mock users — with no platform
 * injection. When deployed the deployer injects a per-app
 * `GROK_AUTH_*` that overrides these (see `server.ts`).
 *
 * These MUST equal the broker's `GROK_PREVIEW_CLIENT_ID` /
 * `GROK_PREVIEW_CLIENT_SECRET` (set in the broker's Vercel env; the broker stores
 * only the secret's `base64url(SHA-256)` hash). This is a dedicated, low-privilege
 * client (preview-only, `*.grok-sandbox.com`) — rotate it by regenerating the
 * broker env var and this constant together.
 */
export const PREVIEW_CLIENT_ID = "grok_preview";
export const PREVIEW_CLIENT_SECRET =
  "8bcdb7fc5a33874ad933ca568918d5790388a0795e44c4d1dea691f801b17ec5";

/** The shared auth broker issuer (OIDC discovery lives under it). */
export const GROK_ISSUER_DEFAULT = "https://auth.grok.me";

/**
 * Host patterns whose callbacks the preview client accepts. Better Auth derives
 * the live preview's real origin from the request host and validates it against
 * this list (wildcard-matched), so the OAuth `redirect_uri` becomes the concrete
 * `https://<preview-host>/api/auth/oauth2/callback/...` the broker allows.
 */
export const PREVIEW_ALLOWED_HOSTS = ["*.grok-sandbox.com"] as const;
```

src/lib/auth/provider.tsx

```tsx
import type { ReactNode } from "react";

/**
 * App-wide client provider mounted once near the root (in `src/routes/__root.tsx`):
 *
 *   <AuthProvider><Outlet /></AuthProvider>
 *
 * Better Auth's React client (`@/lib/auth/client`) needs NO context provider —
 * its `useSession()` works standalone — so this is a passthrough today. It's
 * kept as the single, stable mount point for any future client-side providers
 * (e.g. a toast or theme provider) without churning the root shell.
 */
export function AuthProvider({ children }: { children: ReactNode }) {
  return <>{children}</>;
}
```

src/lib/auth/providers.ts

```ts
/**
 * The upstream identity providers this app offers for sign-in (via the broker).
 *
 * Source of truth for BOTH the server (`server.ts`, one `genericOAuth` provider
 * per entry) and the client (`client.ts` / sign-in buttons). Kept in its own
 * dependency-free module so the client can import it without pulling the
 * server-only Better Auth instance (and `pg`) into the browser bundle.
 *
 * Each app federates to the shared **auth broker** (`GROK_AUTH_ISSUER`), which
 * holds the real Google/X secrets. The app never sees them — it only knows its
 * own per-app client id/secret and which upstream to ask the broker for (`idp`).
 *
 * To add an upstream (e.g. GitHub) once the broker supports it: add one entry
 * here (`{ providerId: "grok-github", idp: "github", label: "GitHub" }`). The
 * `providerId` is this app's local id and the OAuth callback path segment
 * (`/api/auth/oauth2/callback/<providerId>`); `idp` is the hint the broker reads
 * to pick the upstream (Better Auth's id for X is still `twitter`).
 */
export type GrokProvider = {
  /** This app's local provider id; also the callback path segment. */
  providerId: string;
  /** Upstream hint the broker forwards to (Better Auth social id). */
  idp: string;
  /** Human label for the sign-in button. */
  label: string;
};

export const GROK_PROVIDERS: readonly GrokProvider[] = [
  { providerId: "grok-google", idp: "google", label: "Google" },
  { providerId: "grok-x", idp: "twitter", label: "X" },
];
```

src/lib/auth/server.ts

```ts
/**
 * Self-hosted Better Auth for THIS app (server-only).
 *
 * Pre-wired for live preview + deploy — do not rewrite this file. To enable
 * local email/password, flip the flag in `./email-password` only (see auth skill).
 *
 * The app runs its own Better Auth at `/api/auth/*`, so the session cookie stays
 * on this app's own origin. Sign-in federates to the shared **Grok auth broker**
 * (`GROK_AUTH_ISSUER`) via the `genericOAuth` plugin — the broker brokers the
 * upstream sign-in methods (Google, X, …) and holds their shared secrets; this
 * app only holds its own client id/secret and names the upstream it wants via
 * each provider's `idp` hint.
 *
 * Tri-mode:
 *   - Deployed: the deployer injects a per-app `GROK_AUTH_*` + `BETTER_AUTH_URL`
 *     + `DATABASE_URL`, so real federated auth is persisted in Postgres.
 *   - Sandbox live preview: no injection -> falls back to the shared **preview
 *     client** (`./preview`) and derives the preview's `https://*.grok-sandbox.com`
 *     origin from the request, so real sign-in works (no demo users). Sessions
 *     and identities persist in the embedded PGLite DB (same DB as app data);
 *     the process restart wipes both. Live-preview iframe clients use a bearer
 *     token (partitioned cookies) — see `client.ts`.
 *   - Off (`VITE_AUTH_ENABLED=false`, the shipped default): no providers;
 *     `requireUserId` resolves a dev user with no database configured, and
 *     throws fail-closed once `DATABASE_URL` is set (see `verify.server.ts`).
 *
 * NEVER import this from client code — it pulls in `pg` + the preview secret +
 * server-only Better Auth internals. The client uses `@/lib/auth/client`;
 * components read the user via `@/lib/auth/use-current-user`; server functions get
 * a verified id via `@/lib/auth/middleware`.
 */
import { betterAuth } from "better-auth";
import { bearer, genericOAuth } from "better-auth/plugins";
import { tanstackStartCookies } from "better-auth/tanstack-start";
import { getCookie } from "@tanstack/react-start/server";
import { randomBytes } from "node:crypto";
import { Pool } from "pg";
import { ensureDbReady, getPglite } from "../db";
import { emailAndPasswordEnabled } from "./email-password";
import { GROK_PROVIDERS } from "./providers";
import { pgliteDialect } from "./pglite-dialect";
import {
  GROK_ISSUER_DEFAULT,
  PREVIEW_ALLOWED_HOSTS,
  PREVIEW_CLIENT_ID,
  PREVIEW_CLIENT_SECRET,
} from "./preview";

// Kick (and share) PGLite bootstrap as soon as the auth server module loads.
void ensureDbReady();

/**
 * Preview secret must outlive module reloads: PGLite (and its session rows) is
 * stored on `globalThis`, so an HMR re-eval of this file must NOT mint a new
 * signing secret or every existing session becomes invalid mid-dev. Process
 * restart clears both the secret and PGLite together.
 */
const globalAuthRef = globalThis as typeof globalThis & {
  __grokAuthPreviewSecret__?: string;
};
function previewAuthSecret(): string {
  globalAuthRef.__grokAuthPreviewSecret__ ??= randomBytes(32).toString("hex");
  return globalAuthRef.__grokAuthPreviewSecret__;
}

/** Read an env var, treating empty/whitespace as unset. */
const env = (key: string): string | undefined => {
  const value = process.env[key]?.trim();
  return value ? value : undefined;
};

// Explicit off-switch. The deployer sets `VITE_AUTH_ENABLED=true` when it
// provisions auth; set it to "false" to force auth off everywhere (dev user).
const authDisabled = env("VITE_AUTH_ENABLED") === "false";

// Broker federation creds: the deployer injects a per-app client when deployed;
// otherwise fall back to the shared live-preview client, which the broker accepts
// for any `*.grok-sandbox.com` callback (see `./preview`).
```

```
const grokIssuer = env("GROK_AUTH_ISSUER") ?? GROK_ISSUER_DEFAULT;
const grokClientId = env("GROK_AUTH_CLIENT_ID") ?? PREVIEW_CLIENT_ID;
const grokClientSecret = env("GROK_AUTH_CLIENT_SECRET") ?? PREVIEW_CLIENT_SECRET;

/** True when federated sign-in is active (real auth is enforced). */
export const authConfigured =
  !authDisabled && Boolean(grokClientId && grokClientSecret);

// This app's own Better Auth origin. When deployed the deployer injects the
// public URL. In the sandbox live preview there's no fixed URL (each preview gets
// a dynamic `*.grok-sandbox.com` host), so we hand Better Auth a dynamic baseURL:
// it derives the origin per-request from the (proxied) host, validated against the
// preview allowlist, which makes the OAuth `redirect_uri` the concrete preview URL
// the broker's preview client accepts.
const explicitBaseURL = env("BETTER_AUTH_URL");
// Explicit `string[]` (not a readonly tuple) — Better Auth's DynamicBaseURLConfig
// requires a mutable `allowedHosts: string[]`.
const previewAllowedHosts: string[] = [...PREVIEW_ALLOWED_HOSTS];
// Local `npm run dev` (port 8080 contract). Browsers may send Origin as any of
// these for the same server — trusting only `localhost` rejects `127.0.0.1` and
// breaks email/password with "Invalid origin".
const LOCAL_DEV_ORIGINS: string[] = [
  "http://localhost:8080",
  "http://127.0.0.1:8080",
  "http://[::1]:8080",
];
const baseURL = explicitBaseURL ?? {
  // Include loopback hosts so dynamic baseURL resolves for local email/password
  // (not only the preview wildcard).
  allowedHosts: [...previewAllowedHosts, "localhost", "127.0.0.1", "[::1]"],
  // `auto` → trust both http:// and https:// expansions of allowedHosts
  // (preview is https; local dev is http).
  protocol: "auto" as const,
  fallback: "http://localhost:8080",
};

// Origins Better Auth accepts on credentialed POSTs (sign-up/sign-in, etc.).
// Missing entries here surface as FORBIDDEN "Invalid origin".
const trustedOrigins: string[] = explicitBaseURL
  ? [explicitBaseURL, ...LOCAL_DEV_ORIGINS]
  : [
      // Host wildcards (matched against Origin's host)
      ...previewAllowedHosts,
      // Full-origin wildcards (matched against Origin)
      ...previewAllowedHosts.flatMap((host) => [`https://${host}`, `http://${host}`]),
      ...LOCAL_DEV_ORIGINS,
    ];

const databaseUrl = env("DATABASE_URL");

// Static broker OAuth endpoints (skip OIDC discovery on every sign-in / callback).
// Discovery would cost an extra network hop to the broker before the popup can
// even redirect to Google/X — the live-preview popup felt stuck on the app for
// that whole round-trip. These paths match the broker's discovery document.
const issuerBase = grokIssuer.replace(/\/+$/, "");
const grokAuthorizationUrl = `${issuerBase}/api/auth/oauth2/authorize`;
const grokTokenUrl = `${issuerBase}/api/auth/oauth2/token`;
const grokUserInfoUrl = `${issuerBase}/api/auth/oauth2/userinfo`;

// Real Postgres when `DATABASE_URL` is set (deployed apps), else the app's
// embedded PGLite (preview) via a Kysely dialect — so Better Auth persists to the
// SAME DB as app data, including email/password users. Both use the Better Auth
// schema from `migrations/auth/0001_auth.sql`, copied into `migrations/` when
// the app turns sign-in on.
const database = databaseUrl
  ? new Pool({ connectionString: databaseUrl })
  : { dialect: pgliteDialect(() => getPglite()), type: "postgres" as const };

/** Session token cookie name — also read by the live-preview popup completion page. */
export const SESSION_TOKEN_COOKIE = "__Host-grok-auth.session_token";

// Built separately so the `betterAuth({...})` call stays easy to edit without
// breaking brackets (models often trip on the conditional plugin spread).
const grokOAuthPlugin = authConfigured
  ? genericOAuth({
      config: GROK_PROVIDERS.map(({ providerId, idp }) => ({
        providerId,
        clientId: grokClientId as string,
        clientSecret: grokClientSecret as string,
        // Prefer static endpoints over `discoveryUrl` so initiating (and
```

```
        // completing) OAuth does not wait on a broker discovery fetch.
        authorizationUrl: grokAuthorizationUrl,
        tokenUrl: grokTokenUrl,
        userInfoUrl: grokUserInfoUrl,
        scopes: ["openid", "profile", "email"],
        // `prompt: "login"` forces the broker to re-authenticate against the
        // upstream on every sign-in instead of silently reusing an existing
        // broker session. Combined with the broker sending Google
        // `prompt=select_account`, the user always gets the account chooser
        // and can pick (or switch) which account to sign in with.
        authorizationUrlParams: { idp, prompt: "login" },
      })),
    })
  : null;

export const auth = betterAuth({
  baseURL,
  // Deployed apps inject BETTER_AUTH_SECRET. Preview: process-stable secret on
  // globalThis so HMR doesn't invalidate PGLite-backed sessions (see above).
  secret: env("BETTER_AUTH_SECRET") ?? previewAuthSecret(),
  database,

  // CSRF / origin check for credentialed auth POSTs (email sign-up/sign-in, …).
  // See `trustedOrigins` construction above — must cover live preview hosts AND
  // local loopback variants, or clients get "Invalid origin".
  trustedOrigins,

  // Encrypt broker-issued OAuth tokens at rest, and treat the broker's upstreams
  // as trusted first-party identities. The broker owns identity and X emails are
  // synthetic/unverified, so WITHOUT this a login can fail with
  // `account_not_linked` (Better Auth refuses to attach an untrusted, unverified
  // identity to an existing user). Google and X carry DISTINCT emails, so this
  // never merges them into one user — they stay separate identities.
  account: {
    encryptOAuthTokens: true,
    accountLinking: {
      enabled: true,
      trustedProviders: GROK_PROVIDERS.map((p) => p.providerId),
      // X's synthetic email is never "verified", so don't gate linking on the
      // local user's email-verified state.
      requireLocalEmailVerified: false,
    },
  },

  // Cache the session in the short-lived signed `session_data` cookie so reads
  // (incl. the client's `/get-session`) skip the DB — this shrinks the "loading"
  // window and reduces auth flicker. See the `auth` skill for the full
  // flicker-prevention guidance (gate on `isPending`; SSR the session).
  session: { cookieCache: { enabled: true, maxAge: 300 } },

  // Local email/password — toggled only via `./email-password` (not a plugin).
  ...(emailAndPasswordEnabled ? { emailAndPassword: { enabled: true } } : {}),

  // `__Host-` prefixed cookies: the browser REFUSES any same-named cookie that
  // carries a `Domain` attribute, so a sibling `*.grok.me` app cannot "toss" a
  // `Domain=.grok.me` session cookie onto this app. `__Host-` requires Secure +
  // Path=/ + no Domain; Better Auth otherwise uses `__Secure-` (which permits
  // Domain), so we drop its auto prefix (`useSecureCookies: false`) and set
  // Secure + the names ourselves. (Browsers allow Secure cookies on
  // `http://localhost`, so local dev still works.)
  advanced: {
    useSecureCookies: false,
    defaultCookieAttributes: { secure: true, sameSite: "lax", path: "/" },
    cookies: {
      session_token: { name: SESSION_TOKEN_COOKIE },
      session_data: { name: "__Host-grok-auth.session_data" },
      account_data: { name: "__Host-grok-auth.account_data" },
      dont_remember: { name: "__Host-grok-auth.dont_remember" },
    },
  },

  plugins: [
    // One genericOAuth provider per upstream (when auth is on), all federating
    // to the broker with the SAME client and differing only by the `idp` hint.
    ...(grokOAuthPlugin ? [grokOAuthPlugin] : []),

    // Accept `Authorization: Bearer <session-token>` as an alternative to the
    // cookie. Needed for the LIVE PREVIEW: the app runs in an embedded iframe
    // where cookies are partitioned, so after popup sign-in it authenticates with
    // a bearer token instead (see `client.ts` / the `auth` skill). The hook only
```

```
    // fires when an Authorization header is present, so the cookie path
    // (deployed apps) is unaffected.
    bearer(),

    // Bridges Better Auth's Set-Cookie into TanStack Start responses. MUST be
    // last so it runs after every other plugin's hooks.
    tanstackStartCookies(),
  ],
});

export function readSessionToken(): string | null {
  return getCookie(SESSION_TOKEN_COOKIE) ?? null;
}

// Re-exported for convenience; the array lives in the dependency-free
// `providers.ts` so the client can import it too.
export { GROK_PROVIDERS } from "./providers";
```

src/lib/auth/use-current-user.ts

```ts
import { authClient, authEnabled } from "./client";

/** Normalized user shape used across the app, auth on or off. */
export type AppUser = {
  id: string;
  displayName: string | null;
  primaryEmail: string | null;
  profileImageUrl: string | null;
  /** True when this is the sandbox/dev fallback (auth not configured). */
  isDevFallback: boolean;
};

/**
 * Stable fallback user, used ONLY when auth is disabled
 * (`VITE_AUTH_ENABLED=false`, the shipped default). With auth on, the sandbox
 * live preview does real sign-in via the baked preview client. Its id is
 * `"dev-user"` — the SAME id `verify.server.ts` returns server-side — so per-user
 * rows written in that mode belong to one consistent owner.
 */
export const DEV_USER: AppUser = {
  id: "dev-user",
  displayName: "Dev User",
  primaryEmail: "dev@example.com",
  profileImageUrl: null,
  isDevFallback: true,
};

/** `useCurrentUserState()` result: the user plus the session-loading flag. */
export type CurrentUserState = {
  /** The user — `null` BOTH while the session loads and when signed out. */
  user: AppUser | null;
  /** True while the session is still resolving — don't treat `user: null` as signed out yet. */
  isPending: boolean;
};

/**
 * Current user + loading state. Same behavior in live preview and when deployed:
 *   - Auth enabled -> the real signed-in user; `user` is `null` while
 *                             the session resolves (`isPending: true`) and when
 *                             signed out (`isPending: false`). Session comes from
 *                             Better Auth `useSession()` → `/api/auth/get-session`
 *                             (cookie when deployed; bearer in live preview).
 *   - Auth disabled (`VITE_AUTH_ENABLED=false`) -> `DEV_USER`, never pending.
 *
 * Protect a route by waiting out `isPending` before acting on `user` —
 * redirecting on `user: null` alone bounces signed-in visitors to sign-in on
 * every hard reload:
 *
 *   import { RedirectToSignIn } from "@/lib/auth/gates";
 *   const { user, isPending } = useCurrentUserState();
 *   if (isPending) return null;              // still resolving — don't redirect yet
 *   if (!user) return <RedirectToSignIn />;  // definitely signed out
 *
 * `authEnabled` is a module-level constant fixed at load, so the guarded hook
 * call keeps a stable hook order across every render of a given component.
 */
export function useCurrentUserState(): CurrentUserState {
  if (!authEnabled) return { user: DEV_USER, isPending: false };
  // eslint-disable-next-line react-hooks/rules-of-hooks -- authEnabled is constant for the app's lifetime
  const { data, isPending } = authClient.useSession();
  const user = data?.user;
  return {
    user: user
      ? {
          id: user.id,
          displayName: user.name ?? null,
          primaryEmail: user.email ?? null,
          profileImageUrl: user.image ?? null,
          isDevFallback: false,
        }
      : null,
    isPending,
  };
}

/**
 * Convenience view of `useCurrentUserState().user` for display (e.g.
 * `user?.displayName ?? "Guest"`). NOTE: `null` means *loading OR signed out* —
```

```
 * for redirects/guards use `useCurrentUserState()` and check `isPending`.
 */
export function useCurrentUser(): AppUser | null {
  return useCurrentUserState().user;
}
```

src/lib/auth/verify.server.ts

```ts
import { getRequest } from "@tanstack/react-start/server";
import { auth, authConfigured } from "./server";

/**
 * Server-side session resolution (server-only).
 *
 * Because this app runs its OWN Better Auth at same-origin `/api/auth/*`, the
 * session cookie is sent with every request to this app — server functions AND
 * SSR loaders included. So we resolve the user straight from the request cookies
 * via `auth.api.getSession` (no client-minted JWT needed). Never trust a
 * client-supplied user id — only the result of this verification.
 */

/** True when a real database is configured server-side. */
const databaseConfigured = Boolean(process.env.DATABASE_URL?.trim());

/** Re-export so callers can branch on it without importing `server.ts`. */
export { authConfigured };

if (databaseConfigured && !authConfigured) {
  console.error(
    "[auth] DATABASE_URL is set but auth is disabled (VITE_AUTH_ENABLED=false) " +
      "— requireUserId() will reject every request (fail closed) rather than " +
      "share one dev user on a real database.",
  );
}

/** Dev fallback user id, used only when auth is disabled (VITE_AUTH_ENABLED=false). */
export const DEV_USER_ID = "dev-user";

/**
 * Thrown by `requireUserId` when the caller has no valid session. Carries
 * `status: 401`; the message is a stable contract — match
 * `err.message === "Unauthorized"` client-side to send the visitor to sign-in.
 */
export class UnauthorizedError extends Error {
  readonly status = 401;
  constructor() {
    super("Unauthorized");
    this.name = "UnauthorizedError";
  }
}

export type VerifiedUser = { id: string; email: string | null };

/**
 * Resolve the signed-in user from the current request, or `null` when auth isn't
 * configured / nobody is signed in. Safe to call from server functions and SSR
 * loaders.
 *
 * `bearerToken` is for the LIVE PREVIEW: the app runs in a partitioned iframe
 * whose cookies don't reach the server, so `authMiddleware` forwards the session
 * as a bearer token, which we present as `Authorization: Bearer …` (the `bearer`
 * plugin resolves it). When deployed no token is passed and the cookie is used.
 */
export async function getSessionUser(
  bearerToken?: string,
): Promise<VerifiedUser | null> {
  if (!authConfigured) return null;
  const request = getRequest();
  if (!request) return null;
  let headers = request.headers;
  if (bearerToken) {
    headers = new Headers(request.headers);
    headers.set("Authorization", `Bearer ${bearerToken}`);
  }
  const session = await auth.api.getSession({ headers });
  if (!session?.user) return null;
  return { id: session.user.id, email: session.user.email ?? null };
}

/**
 * Resolve the current user id for a server function, or throw when unauthorized.
 * Prefer `authMiddleware` (`./middleware`), which calls this for you.
 * - Auth enabled -> the verified session user id; throws
 *   `UnauthorizedError` when signed out. Works in the sandbox preview too (real
 *   sign-in via the baked preview client).
 * - Auth disabled (`VITE_AUTH_ENABLED=false`) + `DATABASE_URL` set -> throw (fail
```

```
 *   closed): one shared dev user on a real database would let every visitor
 *   read/write everyone's rows.
 * - Auth disabled + no database -> the shared dev user id.
 */
export async function requireUserId(bearerToken?: string): Promise<string> {
  if (!authConfigured) {
    if (databaseConfigured) {
      throw new Error(
        "Auth is disabled (VITE_AUTH_ENABLED=false) but DATABASE_URL is set — " +
          "refusing to fall back to the shared dev user against a real database.",
      );
    }
    return DEV_USER_ID;
  }
  const user = await getSessionUser(bearerToken);
  if (!user) throw new UnauthorizedError();
  return user.id;
}
```

src/lib/db.ts

```ts
import { pendingMigrations } from "../../scripts/migration-plan.mjs";

/** Which database backend is active. */
export type DbSource = "neon" | "pglite";

// An empty/whitespace DATABASE_URL (an easy misconfig in deploy UIs) must mean
// "unset" — otherwise production would silently run on the PGLite fallback.
const rawDatabaseUrl =
  typeof process !== "undefined" ? process.env.DATABASE_URL : undefined;
const databaseUrl =
  rawDatabaseUrl && rawDatabaseUrl.trim() ? rawDatabaseUrl : undefined;

/**
 * Active backend: real **Neon** when `DATABASE_URL` is set (deployed / configured
 * sandbox), otherwise a local embedded **PGLite** (Postgres compiled to WASM) so
 * the app has a working database even with nothing configured — the live preview
 * included. Swap in Neon later by just setting `DATABASE_URL`; no code changes.
 */
export const dbSource: DbSource = databaseUrl ? "neon" : "pglite";

/**
 * Minimal shared SQL surface, satisfied by both Neon and PGLite. Both the
 * tagged-template and `.query()` forms resolve to an array of row objects:
 *
 *   const sql = await getSql();
 *   const rows = await sql`select * from todos where id = ${id}`; // parameterized
 *   const rows2 = await sql.query("select * from todos where id = $1", [id]);
 */
export interface Sql {
  <T = Record<string, unknown>>(
    strings: TemplateStringsArray,
    ...values: unknown[]
  ): Promise<T[]>;
  query<T = Record<string, unknown>>(
    text: string,
    params?: unknown[],
  ): Promise<T[]>;
}

/**
 * Init state lives on globalThis as promises: dev HMR creates new instances of
 * this module, and two instances racing module-level state would open a second
 * pool or run two concurrent PGLite migration passes (whose duplicate
 * `_migrations` insert rejects — and would get memoized, poisoning every later
 * `getSql()`). A failed init clears its slot so the next call retries.
 */
const globalRef = globalThis as typeof globalThis & {
  __pgSqlPromise__?: Promise<Sql>;
  __pgliteInstance__?: Promise<import("@electric-sql/pglite").PGlite>;
  __pgliteMigrateChain__?: Promise<void>;
};

/**
 * Result-type parity: Postgres sends every value as text plus a type OID — the
 * JS value is the DRIVER's parsing choice, and pg and PGLite disagree (pg:
 * int8 -> string, date -> local-midnight Date; PGLite: int8 -> BigInt, which
 * JSON.stringify rejects, date -> UTC Date). Normalize both so preview and
 * production return identical, JSON-safe shapes:
 *   int8/bigint (incl. count(*)) -> number (past 2^53 loses precision — cast
 *                                   `::text` if you ever need huge integers)
 *   date                         -> 'YYYY-MM-DD' string
 *   interval                     -> Postgres interval text
 * numeric already comes back as a string on both (arbitrary precision).
 */
const OID_INT8 = 20;
const OID_DATE = 1082;
const OID_INTERVAL = 1186;
const identity = (v: string) => v;

type Run = <T>(text: string, params: unknown[]) => Promise<T[]>;

/** Wrap a query runner in the tagged-template + `.query()` `Sql` surface. */
function toSql(run: Run): Sql {
  const sql = (async <T = Record<string, unknown>>(
    strings: TemplateStringsArray,
    ...values: unknown[]
  ): Promise<T[]> => {
    // Rebuild with $1, $2, … placeholders so values stay parameterized.
```

```
    let text = strings[0];
    for (let i = 0; i < values.length; i += 1) text += `$${i + 1}${strings[i + 1]}`;
    return run<T>(text, values);
  }) as unknown as Sql;
  sql.query = <T = Record<string, unknown>>(text: string, params: unknown[] = []) =>
    run<T>(text, params);
  return sql;
}

function createNeonSql(): Promise<Sql> {
  globalRef.__pgSqlPromise__ ??= (async () => {
    // Regular Postgres driver: node-postgres (`pg`) — works directly with Neon's
    // pooled endpoint. One pool per process; warm serverless instances reuse it.
    const { Pool, types } = await import("pg");
    types.setTypeParser(OID_INT8, Number);
    types.setTypeParser(OID_DATE, identity);
    types.setTypeParser(OID_INTERVAL, identity);
    const pool = new Pool({ connectionString: databaseUrl });
    return toSql(async <T>(text: string, params: unknown[]) => {
      const res = await pool.query(text, params);
      return res.rows as T[];
    });
  })().catch((err) => {
    globalRef.__pgSqlPromise__ = undefined;
    throw err;
  });
  return globalRef.__pgSqlPromise__;
}

async function createPgliteSql(): Promise<Sql> {
  // Embedded Postgres, imported on demand so it never loads on the Neon path.
  // One in-memory instance per process, shared across HMR module instances, so
  // data survives source edits (it resets on dev-server restart).
  globalRef.__pgliteInstance__ ??= (async () => {
    const { PGlite } = await import("@electric-sql/pglite");
    const pg = new PGlite({
      parsers: {
        [OID_INT8]: Number,
        [OID_DATE]: identity,
        [OID_INTERVAL]: identity,
      },
    });
    await pg.waitReady;
    await pg.exec(
      "create table if not exists _migrations (name text primary key, applied_at timestamptz not null default now())",
    );
    return pg;
  })().catch((err) => {
    globalRef.__pgliteInstance__ = undefined;
    throw err;
  });
  const pg = await globalRef.__pgliteInstance__;

  // Apply migrations/ (the single schema source) so preview matches production.
  // SQL is inlined by the bundler via import.meta.glob (no runtime fs); applied
  // files are tracked in _migrations. The glob does not descend, so the opt-in
  // auth schema under migrations/auth/ stays out. Runs once per module instance
  // — so an HMR reload after adding a migration file applies it live — with
  // passes serialized on a global chain so concurrent callers never
  // double-apply.
  const migrate = async (): Promise<void> => {
    const migrations = import.meta.glob("/migrations/*.sql", {
      query: "?raw",
      import: "default",
      eager: true,
    }) as Record<string, string>;
    const doneRows = await pg.query<{ name: string }>(
      "select name from _migrations",
    );
    const done = doneRows.rows.map((r) => r.name);
    for (const { name, path } of pendingMigrations(Object.keys(migrations), done)) {
      // Apply + record atomically (parity with scripts/migrate.mjs) so a failed
      // statement can't leave a file half-applied but untracked.
      await pg.transaction(async (tx) => {
        await tx.exec(migrations[path]);
        await tx.query("insert into _migrations (name) values ($1)", [name]);
      });
    }
  };
  const pass = (globalRef.__pgliteMigrateChain__ ?? Promise.resolve())
```

```
    .catch(() => undefined) // an earlier failed pass must not wedge the chain
    .then(migrate);
  globalRef.__pgliteMigrateChain__ = pass;
  await pass;

  return toSql(async <T>(text: string, params: unknown[]) => {
    const result = await pg.query<T>(text, params);
    return result.rows;
  });
}

let sqlPromise: Promise<Sql> | null = null;

async function createSql(): Promise<Sql> {
  if (typeof window !== "undefined") {
    throw new Error(
      "@/lib/db is server-only — call getSql() from a createServerFn handler " +
        "or a server route loader, never from client code.",
    );
  }
  return dbSource === "neon" ? createNeonSql() : createPgliteSql();
}

/**
 * Get the shared, **server-only** SQL client. Neon when `DATABASE_URL` is set,
 * otherwise the local PGLite fallback. Memoized — safe to call per request.
 *
 * Schema comes from `migrations/*.sql`, auto-applied before the first query on
 * both backends — define tables there, never inline in server functions.
 */
export function getSql(): Promise<Sql> {
  sqlPromise ??= createSql().catch((err) => {
    sqlPromise = null; // don't memoize failures — let the next call retry
    throw err;
  });
  return sqlPromise;
}

/**
 * The shared PGLite instance (preview only), with `migrations/*.sql` applied.
 * Lets Better Auth persist to the SAME embedded DB as app data in preview (via a
 * Kysely dialect). Throws when `DATABASE_URL` is set (that path uses Neon).
 */
export async function getPglite(): Promise<import("@electric-sql/pglite").PGlite> {
  if (dbSource !== "pglite") {
    throw new Error("getPglite() is only available on the PGLite fallback (no DATABASE_URL)");
  }
  await getSql();
  const pg = await globalRef.__pgliteInstance__;
  if (!pg) throw new Error("PGLite instance failed to initialize");
  return pg;
}

/**
 * Finish DB bootstrap before the server handles traffic.
 *
 * - **PGLite** (preview / no `DATABASE_URL`): open the in-memory DB and apply
 *   `migrations/*.sql`. Idempotent — concurrent callers share one promise.
 * - **Neon**: no-op (pool is created lazily on first query).
 *
 * Vite `configureServer` awaits this at dev startup; production imports of this
 * module kick it off immediately (see bottom of file).
 */
export function ensureDbReady(): Promise<void> {
  if (dbSource !== "pglite") return Promise.resolve();
  return getSql().then(() => undefined);
}

// Server-only eager start: kick PGLite bootstrap as soon as this module loads in
// Node. Client bundles never hit this path (`getSql` throws in the browser).
const globalBoot = globalThis as typeof globalThis & {
  __pgBootstrapPromise__?: Promise<void>;
};
if (typeof window === "undefined" && dbSource === "pglite") {
  globalBoot.__pgBootstrapPromise__ ??= ensureDbReady().catch((err) => {
    globalBoot.__pgBootstrapPromise__ = undefined;
    console.error("[db] PGLite bootstrap failed:", err);
    throw err;
  });
}
```

src/lib/error-component.tsx

```tsx
import type { ErrorComponentProps } from "@tanstack/react-router";
import { TriangleAlert } from "lucide-react";

export function AppErrorComponent({ error }: ErrorComponentProps) {
  return (
    <main
      className={
        "flex min-h-screen flex-col items-center justify-center gap-3 px-6 text-center " +
        "bg-zinc-50 text-zinc-900 dark:bg-zinc-950 dark:text-zinc-50"
      }
    >
      <span className="text-red-500" aria-hidden="true">
        <TriangleAlert className="size-10" strokeWidth={2} />
      </span>
      <h1 className="text-lg font-semibold">Something went wrong</h1>
      <p className="max-w-md text-sm break-words text-zinc-500 dark:text-zinc-400">
        {error.message || "An unexpected error occurred. Try reloading the page."}
      </p>
    </main>
  );
}
```

src/lib/ledger/analytics.ts

```ts
import {
  addDays,
  addMonths,
  addWeeks,
  differenceInCalendarDays,
  eachDayOfInterval,
  endOfDay,
  endOfMonth,
  format,
  startOfDay,
  startOfMonth,
  startOfWeek,
  subDays,
  subMonths,
  subYears,
} from "date-fns";
import {
  categoryColor,
  categoryLabel,
  findCategory,
  labelsEqual,
  normalizeLabel,
  STEEL_COLORS,
} from "./categories";
import { signedAmount } from "./format";
import type {
  Budget,
  Category,
  CategoryId,
  RangePreset,
  Transaction,
  TxType,
} from "./types";

export type DateWindow = {
  start: Date;
  end: Date;
  priorStart: Date;
  priorEnd: Date;
};

export type KpiPoint = {
  current: number;
  previous: number;
  change: number | null;
  series: number[];
};

export type MixSlice = {
  category: CategoryId;
  label: string;
  value: number;
  share: number;
  color: string;
  count: number;
  type: TxType;
};

export type LedgerFocus =
  | { kind: "category"; id: CategoryId }
  | { kind: "payee"; name: string };

export type PayeeSlice = {
  payee: string;
  label: string;
  value: number;
  share: number;
  color: string;
  count: number;
};

export type BarPoint = {
  key: string;
  label: string;
  income: number;
  expense: number;
};

```

```ts
export type BalancePoint = {
  date: string;
  label: string;
  balance: number;
};

export type BudgetProgress = {
  id: string;
  category: CategoryId;
  label: string;
  color: string;
  monthly: number;
  spent: number;
  remaining: number;
  ratio: number;
};

export function windowFor(preset: RangePreset, now = new Date()): DateWindow {
  const end = endOfDay(now);
  let start: Date;
  switch (preset) {
    case "7d":
      start = startOfDay(subDays(now, 6));
      break;
    case "30d":
      start = startOfDay(subDays(now, 29));
      break;
    case "90d":
      start = startOfDay(subDays(now, 89));
      break;
    case "6m":
      start = startOfDay(subMonths(now, 6));
      break;
    case "1y":
      start = startOfDay(subYears(now, 1));
      break;
  }
  const days = differenceInCalendarDays(end, start);
  const priorEnd = endOfDay(subDays(start, 1));
  const priorStart = startOfDay(subDays(priorEnd, days));
  return { start, end, priorStart, priorEnd };
}

export function monthWindow(now = new Date()): { start: Date; end: Date } {
  return { start: startOfMonth(now), end: endOfMonth(now) };
}

export function inWindow(tx: Transaction, start: Date, end: Date): boolean {
  const t = startOfDay(new Date(`${tx.date}T00:00:00`)).getTime();
  return t >= startOfDay(start).getTime() && t <= startOfDay(end).getTime();
}

function sums(txs: Transaction[]) {
  let income = 0;
  let expenses = 0;
  for (const tx of txs) {
    if (tx.type === "income") income += tx.amount;
    else expenses += tx.amount;
  }
  return { income, expenses, net: income - expenses };
}

function change(current: number, previous: number): number | null {
  if (previous === 0) return current === 0 ? 0 : null;
  return (current - previous) / Math.abs(previous);
}

function dailySeries(
  txs: Transaction[],
  start: Date,
  end: Date,
  pick: (tx: Transaction) => number,
): number[] {
  const days = eachDayOfInterval({ start: startOfDay(start), end: startOfDay(end) });
  const byDay = new Map<string, number>();
  for (const tx of txs) {
    byDay.set(tx.date, (byDay.get(tx.date) ?? 0) + pick(tx));
  }
  return days.map((d) => byDay.get(format(d, "yyyy-MM-dd")) ?? 0);
}
```

```

function running(values: number[]): number[] {
  let acc = 0;
  return values.map((v) => {
    acc += v;
    return acc;
  });
}

export function filterFocused(
  txs: Transaction[],
  focus: LedgerFocus | null,
): Transaction[] {
  if (!focus) return txs;
  if (focus.kind === "category") return txs.filter((t) => t.category === focus.id);
  return txs.filter((t) => labelsEqual(t.payee, focus.name));
}

export function searchTransactions(
  txs: Transaction[],
  query: string,
  typeFilter: "all" | TxType,
  categories: Category[],
): Transaction[] {
  const q = query.trim().toLowerCase();
  return txs.filter((t) => {
    if (typeFilter !== "all" && t.type !== typeFilter) return false;
    if (!q) return true;
    const hay = `${t.payee} ${t.product ?? ""} ${t.note ?? ""} ${categoryLabel(t.category, categories)}`.toLowerCase()
;
    return hay.includes(q);
  });
}

export function kpisFor(
  txs: Transaction[],
  range: DateWindow,
): { net: KpiPoint; income: KpiPoint; expenses: KpiPoint; savingsRate: KpiPoint } {
  const currentTxs = txs.filter((t) => inWindow(t, range.start, range.end));
  const priorTxs = txs.filter((t) => inWindow(t, range.priorStart, range.priorEnd));
  const cur = sums(currentTxs);
  const prev = sums(priorTxs);
  const incomeSeries = dailySeries(currentTxs, range.start, range.end, (t) =>
    t.type === "income" ? t.amount : 0,
  );
  const expenseSeries = dailySeries(currentTxs, range.start, range.end, (t) =>
    t.type === "expense" ? t.amount : 0,
  );
  const netSeries = incomeSeries.map((v, i) => v - (expenseSeries[i] ?? 0));
  const runInc = running(incomeSeries);
  const runExp = running(expenseSeries);
  const savingsSeries = runInc.map((inc, i) => {
    const exp = runExp[i] ?? 0;
    return inc <= 0 ? 0 : (inc - exp) / inc;
  });
  const savingsCurrent = cur.income <= 0 ? 0 : cur.net / cur.income;
  const savingsPrev = prev.income <= 0 ? 0 : prev.net / prev.income;
  const priorEmpty = priorTxs.length === 0;
  return {
    net: {
      current: cur.net,
      previous: prev.net,
      change: priorEmpty ? null : change(cur.net, prev.net),
      series: running(netSeries),
    },
    income: {
      current: cur.income,
      previous: prev.income,
      change: priorEmpty ? null : change(cur.income, prev.income),
      series: running(incomeSeries),
    },
    expenses: {
      current: cur.expenses,
      previous: prev.expenses,
      change: priorEmpty ? null : change(cur.expenses, prev.expenses),
      series: running(expenseSeries),
    },
    savingsRate: {
      current: savingsCurrent,
      previous: savingsPrev,
```

```
      change: priorEmpty ? null : savingsCurrent - savingsPrev,
      series: savingsSeries,
    },
  };
}

export function spendMix(
  txs: Transaction[],
  range: DateWindow,
  categories: Category[],
): MixSlice[] {
  const current = txs.filter(
    (t) => t.type === "expense" && inWindow(t, range.start, range.end),
  );
  const byCat = new Map<CategoryId, { value: number; count: number; type: TxType }>();
  for (const tx of current) {
    const prev = byCat.get(tx.category) ?? { value: 0, count: 0, type: tx.type };
    prev.value += tx.amount;
    prev.count += 1;
    byCat.set(tx.category, prev);
  }
  const total = [...byCat.values()].reduce((s, v) => s + v.value, 0);
  return [...byCat.entries()]
    .map(([category, v]) => ({
      category,
      label: categoryLabel(category, categories),
      value: v.value,
      count: v.count,
      share: total === 0 ? 0 : v.value / total,
      color: categoryColor(category, categories),
      type: findCategory(categories, category)?.type ?? v.type,
    }))
    .sort((a, b) => b.value - a.value);
}

function payeeColor(name: string): string {
  let hash = 0;
  for (let i = 0; i < name.length; i++) {
    hash = (hash * 31 + name.charCodeAt(i)) >>> 0;
  }
  return STEEL_COLORS[hash % STEEL_COLORS.length]!;
}

export function incomePayeeMix(txs: Transaction[], range: DateWindow): PayeeSlice[] {
  const current = txs.filter((t) => t.type === "income" && inWindow(t, range.start, range.end));
  const byPayee = new Map<string, { label: string; value: number; count: number }>();
  for (const tx of current) {
    const label = normalizeLabel(tx.payee) || "Unknown";
    const key = label.toLowerCase();
    const prev = byPayee.get(key) ?? { label, value: 0, count: 0 };
    prev.value += tx.amount;
    prev.count += 1;
    byPayee.set(key, prev);
  }
  const total = [...byPayee.values()].reduce((s, v) => s + v.value, 0);
  return [...byPayee.values()]
    .map((v) => ({
      payee: v.label,
      label: v.label,
      value: v.value,
      count: v.count,
      share: total === 0 ? 0 : v.value / total,
      color: payeeColor(v.label),
    }))
    .sort((a, b) => b.value - a.value);
}

export function categoryRows(
  txs: Transaction[],
  range: DateWindow,
  categories: Category[],
): MixSlice[] {
  const current = txs.filter((t) => inWindow(t, range.start, range.end));
  const byCat = new Map<CategoryId, { value: number; count: number; type: TxType }>();
  for (const tx of current) {
    const prev = byCat.get(tx.category) ?? { value: 0, count: 0, type: tx.type };
    prev.value += tx.amount;
    prev.count += 1;
    byCat.set(tx.category, prev);
  }
```

```
  const total = [...byCat.values()].reduce((s, v) => s + v.value, 0) || 1;
  return [...byCat.entries()]
    .map(([category, v]) => ({
      category,
      label: categoryLabel(category, categories),
      value: v.value,
      count: v.count,
      share: v.value / total,
      color: categoryColor(category, categories),
      type: findCategory(categories, category)?.type ?? v.type,
    }))
    .sort((a, b) => b.value - a.value);
}

export function budgetProgress(
  budgets: Budget[],
  txs: Transaction[],
  categories: Category[],
  now = new Date(),
): BudgetProgress[] {
  const { start, end } = monthWindow(now);
  return budgets
    .map((b) => {
      let spent = 0;
      for (const tx of txs) {
        if (tx.type !== "expense" || tx.category !== b.category) continue;
        if (!inWindow(tx, start, end)) continue;
        spent += tx.amount;
      }
      return {
        id: b.id,
        category: b.category,
        label: categoryLabel(b.category, categories),
        color: categoryColor(b.category, categories),
        monthly: b.monthly,
        spent,
        remaining: b.monthly - spent,
        ratio: b.monthly <= 0 ? 0 : spent / b.monthly,
      };
    })
    .sort((a, b) => b.ratio - a.ratio || a.label.localeCompare(b.label));
}

export function balanceSeries(
  txs: Transaction[],
  range: DateWindow,
  focused: boolean,
): BalancePoint[] {
  const sorted = [...txs].sort(
    (a, b) => a.date.localeCompare(b.date) || a.id.localeCompare(b.id),
  );
  const startIso = format(range.start, "yyyy-MM-dd");
  const endIso = format(range.end, "yyyy-MM-dd");
  let bal = 0;
  if (!focused) {
    for (const tx of sorted) {
      if (tx.date < startIso) bal += signedAmount(tx.type, tx.amount);
    }
  }
  const delta = new Map<string, number>();
  for (const tx of sorted) {
    if (tx.date < startIso || tx.date > endIso) continue;
    delta.set(tx.date, (delta.get(tx.date) ?? 0) + signedAmount(tx.type, tx.amount));
  }
  const days = eachDayOfInterval({
    start: startOfDay(range.start),
    end: startOfDay(range.end),
  });
  const span = days.length;
  return days.map((d) => {
    const iso = format(d, "yyyy-MM-dd");
    bal += delta.get(iso) ?? 0;
    const label =
      span <= 40 ? format(d, "d MMM") : span <= 120 ? format(d, "d MMM") : format(d, "MMM");
    return { date: iso, label, balance: bal };
  });
}

function iterateBuckets(
  start: Date,
```

```
  end: Date,
  step: "day" | "week" | "month",
): { start: Date; end: Date; label: string; key: string }[] {
  const out: { start: Date; end: Date; label: string; key: string }[] = [];
  if (step === "day") {
    for (const d of eachDayOfInterval({ start: startOfDay(start), end: startOfDay(end) })) {
      out.push({
        start: d,
        end: d,
        label: format(d, "d MMM"),
        key: format(d, "yyyy-MM-dd"),
      });
    }
    return out;
  }
  if (step === "week") {
    let cursor = startOfWeek(start, { weekStartsOn: 1 });
    if (cursor < start) cursor = addWeeks(cursor, 1);
    if (cursor > start) {
      cursor = startOfWeek(start, { weekStartsOn: 1 });
    }
    while (cursor <= end) {
      const bucketEnd = addDays(addWeeks(cursor, 1), -1);
      out.push({
        start: cursor,
        end: bucketEnd > end ? end : bucketEnd,
        label: format(cursor, "d MMM"),
        key: format(cursor, "yyyy-MM-dd"),
      });
      cursor = addWeeks(cursor, 1);
    }
    return out;
  }
  let cursor = startOfMonth(start);
  while (cursor <= end) {
    const bucketEnd = addDays(addMonths(cursor, 1), -1);
    out.push({
      start: cursor,
      end: bucketEnd > end ? end : bucketEnd,
      label: format(cursor, "MMM"),
      key: format(cursor, "yyyy-MM"),
    });
    cursor = addMonths(cursor, 1);
  }
  return out;
}

export function incomeExpenseBars(
  txs: Transaction[],
  range: DateWindow,
  preset: RangePreset,
): BarPoint[] {
  const step: "day" | "week" | "month" =
    preset === "7d" || preset === "30d" ? "day" : preset === "90d" ? "week" : "month";
  const buckets = iterateBuckets(range.start, range.end, step);
  const current = txs.filter((t) => inWindow(t, range.start, range.end));
  return buckets.map((b) => {
    let income = 0;
    let expense = 0;
    for (const tx of current) {
      if (!inWindow(tx, b.start, b.end)) continue;
      if (tx.type === "income") income += tx.amount;
      else expense += tx.amount;
    }
    return { key: b.key, label: b.label, income, expense };
  });
}

export const RANGE_LABELS: Record<RangePreset, string> = {
  "7d": "7D",
  "30d": "30D",
  "90d": "90D",
  "6m": "6M",
  "1y": "1Y",
};
```

src/lib/ledger/categories.ts

```ts
import type { Category, CategoryId, Payee, Product, TxType } from "./types";

export type CategoryMeta = Category;

export const STEEL_COLORS = [
  "#d7dce3",
  "#c9d0d8",
  "#b7bfc8",
  "#adb6c0",
  "#c0c8d1",
  "#98a2ad",
  "#88929c",
  "#9aa3ae",
  "#7a848f",
  "#6a7380",
] as const;

export const FALLBACK_COLOR = "#6a7380";

/** Used only when importing a v1 backup. */
export const LEGACY_CATEGORIES: Record<
  string,
  { label: string; type: TxType; color: string }
> = {
  salary: { label: "Salary", type: "income", color: "#d7dce3" },
  freelance: { label: "Freelance", type: "income", color: "#b7bfc8" },
  transfer: { label: "Transfer", type: "income", color: "#98a2ad" },
  other_income: { label: "Other income", type: "income", color: "#818a96" },
  groceries: { label: "Groceries", type: "expense", color: "#d7dce3" },
  dining: { label: "Dining", type: "expense", color: "#b7bfc8" },
  transport: { label: "Transport", type: "expense", color: "#98a2ad" },
  housing: { label: "Housing", type: "expense", color: "#c9d0d8" },
  utilities: { label: "Utilities", type: "expense", color: "#88929c" },
  health: { label: "Health", type: "expense", color: "#adb6c0" },
  shopping: { label: "Shopping", type: "expense", color: "#7a848f" },
  entertainment: { label: "Entertainment", type: "expense", color: "#c0c8d1" },
  subscriptions: { label: "Subscriptions", type: "expense", color: "#6a7380" },
  other: { label: "Other", type: "expense", color: "#9aa3ae" },
};

export function nextColor(existing: Category[]): string {
  const used = new Set(existing.map((c) => c.color.toLowerCase()));
  const free = STEEL_COLORS.find((c) => !used.has(c));
  if (free) return free;
  return STEEL_COLORS[existing.length % STEEL_COLORS.length]!;
}

export function normalizeLabel(value: string): string {
  return value.trim().replace(/\s+/g, " ");
}

export function labelsEqual(a: string, b: string): boolean {
  return normalizeLabel(a).toLowerCase() === normalizeLabel(b).toLowerCase();
}

export function findCategory(
  categories: Category[],
  id: CategoryId,
): Category | undefined {
  return categories.find((c) => c.id === id);
}

export function categoryLabel(id: CategoryId, categories: Category[]): string {
  return findCategory(categories, id)?.label ?? "Uncategorized";
}

export function categoryColor(id: CategoryId, categories: Category[]): string {
  return findCategory(categories, id)?.color ?? FALLBACK_COLOR;
}

export function categoriesFor(categories: Category[], type: TxType): Category[] {
  return categories.filter((c) => c.type === type);
}

export function hasDuplicateLabel(
  categories: Category[],
  type: TxType,
  label: string,
```

```
  exceptId?: string,
): boolean {
  const normalized = normalizeLabel(label);
  if (!normalized) return false;
  return categories.some(
    (c) => c.type === type && c.id !== exceptId && labelsEqual(c.label, normalized),
  );
}

export function findPayee(
  payees: Payee[],
  name: string,
  type: TxType,
  category: CategoryId,
): Payee | undefined {
  return payees.find(
    (p) => p.type === type && p.category === category && labelsEqual(p.name, name),
  );
}

export function payeesFor(
  payees: Payee[],
  type: TxType,
  category?: CategoryId,
): Payee[] {
  return payees
    .filter((p) => {
      if (p.type !== type) return false;
      if (!category) return true;
      return p.category === category || p.category === "";
    })
    .sort((a, b) => a.name.localeCompare(b.name, undefined, { sensitivity: "base" }));
}

export function payeeNames(
  payees: Payee[],
  type: TxType,
  category?: CategoryId,
): string[] {
  const names: string[] = [];
  for (const row of payeesFor(payees, type, category)) {
    if (!names.some((n) => labelsEqual(n, row.name))) names.push(row.name);
  }
  return names;
}

export function findProduct(
  products: Product[],
  name: string,
  category: CategoryId,
): Product | undefined {
  return products.find((p) => p.category === category && labelsEqual(p.name, name));
}

export function productsFor(products: Product[], category?: CategoryId): Product[] {
  return products
    .filter((p) => {
      if (!category) return true;
      return p.category === category || p.category === "";
    })
    .sort((a, b) => a.name.localeCompare(b.name, undefined, { sensitivity: "base" }));
}

export function productNames(products: Product[], category?: CategoryId): string[] {
  const names: string[] = [];
  for (const row of productsFor(products, category)) {
    if (!names.some((n) => labelsEqual(n, row.name))) names.push(row.name);
  }
  return names;
}
```

src/lib/ledger/csv.ts

```ts
import { isValid, parse, parseISO } from "date-fns";
import {
  labelsEqual,
  nextColor,
  normalizeLabel,
} from "./categories";
import { isoDate } from "./format";
import type { Category, Payee, Product, Transaction, TxType } from "./types";

const MAX_TX = 5000;
const DATE_FORMATS = [
  "yyyy-MM-dd",
  "d.M.yyyy",
  "dd.MM.yyyy",
  "d/M/yyyy",
  "dd/MM/yyyy",
  "d-M-yyyy",
  "dd-MM-yyyy",
  "d MMM yyyy",
  "dd MMM yyyy",
  "d MMMM yyyy",
];

function csvField(value: string): string {
  if (/[",\r\n]/.test(value)) return `"${value.replaceAll('"', '""')}"`;
  return value;
}

function money(n: number): string {
  return n.toFixed(2);
}

export function transactionsToCsv(
  transactions: Transaction[],
  categories: Category[],
): string {
  const header = ["Date", "Type", "Payee", "Category", "Product", "Quantity", "Income", "Expense", "Note"];
  const rows = [...transactions]
    .sort((a, b) => a.date.localeCompare(b.date) || a.payee.localeCompare(b.payee))
    .map((tx) => {
      const income = tx.type === "income" ? money(tx.amount) : "";
      const expense = tx.type === "expense" ? money(tx.amount) : "";
      return [
        tx.date,
        tx.type === "income" ? "Income" : "Expense",
        csvField(tx.payee),
        csvField(categoryLabelSafe(tx.category, categories)),
        csvField(tx.type === "income" ? (tx.product ?? "") : ""),
        tx.type === "income" && tx.quantity != null ? String(tx.quantity) : "",
        income,
        expense,
        csvField(tx.note ?? ""),
      ].join(",");
    });
  return [header.join(","), ...rows].join("\r\n");
}

function categoryLabelSafe(id: string, categories: Category[]): string {
  return categories.find((c) => c.id === id)?.label ?? "Uncategorized";
}

function stamp(): string {
  const now = new Date();
  return `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, "0")}-${String(now.getDate()).padStart(2, "0")
}`;
}

function downloadFile(file: File): void {
  const url = URL.createObjectURL(file);
  const a = document.createElement("a");
  a.href = url;
  a.download = file.name;
  a.rel = "noopener";
  document.body.appendChild(a);
  a.click();
  a.remove();
  window.setTimeout(() => URL.revokeObjectURL(url), 1500);
}
```

```

export type CsvExportResult = "empty" | "shared" | "downloaded" | "cancelled";

export async function exportTransactionsCsv(
  transactions: Transaction[],
  categories: Category[],
): Promise<CsvExportResult> {
  if (transactions.length === 0) return "empty";
  const csv = transactionsToCsv(transactions, categories);
  const file = new File([`\uFEFF${csv}`], `ledger-transactions-${stamp()}.csv`, {
    type: "text/csv;charset=utf-8",
  });

  try {
    if (typeof navigator.share === "function" && navigator.canShare?.({ files: [file] })) {
      await navigator.share({ files: [file], title: "Ledger transactions" });
      return "shared";
    }
  } catch (err) {
    if (err instanceof DOMException && err.name === "AbortError") return "cancelled";
  }

  downloadFile(file);
  return "downloaded";
}

function fold(value: string): string {
  return value
    .normalize("NFD")
    .replace(/\p{M}/gu, "")
    .toLowerCase()
    .trim();
}

function detectDelimiter(line: string): "," | ";" {
  let comma = 0;
  let semi = 0;
  let inQuotes = false;
  for (let i = 0; i < line.length; i++) {
    const c = line[i]!;
    if (c === '"') {
      if (inQuotes && line[i + 1] === '"') {
        i += 1;
        continue;
      }
      inQuotes = !inQuotes;
    } else if (!inQuotes) {
      if (c === ",") comma += 1;
      if (c === ";") semi += 1;
    }
  }
  return semi > comma ? ";" : ",";
}

export function parseCsv(text: string): string[][] {
  const src = text.replace(/^\uFEFF/, "");
  const firstNl = src.search(/\r\n|\n|\r/);
  const firstLine = firstNl === -1 ? src : src.slice(0, firstNl);
  const delim = detectDelimiter(firstLine);
  const rows: string[][] = [];
  let row: string[] = [];
  let field = "";
  let inQuotes = false;
  for (let i = 0; i < src.length; i++) {
    const c = src[i]!;
    if (inQuotes) {
      if (c === '"') {
        if (src[i + 1] === '"') {
          field += '"';
          i += 1;
        } else {
          inQuotes = false;
        }
        continue;
      }
      field += c;
      continue;
    }
    if (c === '"') {
      inQuotes = true;
```

```
      continue;
    }
    if (c === delim) {
      row.push(field);
      field = "";
      continue;
    }
    if (c === "\n" || c === "\r") {
      if (c === "\r" && src[i + 1] === "\n") i += 1;
      row.push(field);
      field = "";
      if (row.some((cell) => cell.trim())) rows.push(row);
      row = [];
      continue;
    }
    field += c;
  }
  row.push(field);
  if (row.some((cell) => cell.trim())) rows.push(row);
  return rows;
}

type Col =
  | "date"
  | "type"
  | "payee"
  | "category"
  | "product"
  | "quantity"
  | "income"
  | "expense"
  | "amount"
  | "note";

const HEADER_MAP: Record<string, Col> = {
  date: "date",
  data: "date",
  zi: "date",
  type: "type",
  tip: "type",
  kind: "type",
  payee: "payee",
  merchant: "payee",
  vendor: "payee",
  beneficiar: "payee",
  name: "payee",
  nume: "payee",
  category: "category",
  categorie: "category",
  cat: "category",
  product: "product",
  produs: "product",
  item: "product",
  quantity: "quantity",
  qty: "quantity",
  cantitate: "quantity",
  income: "income",
  venit: "income",
  in: "income",
  expense: "expense",
  cheltuiala: "expense",
  cheltuieli: "expense",
  out: "expense",
  amount: "amount",
  suma: "amount",
  value: "amount",
  valoare: "amount",
  note: "note",
  notes: "note",
  nota: "note",
  description: "note",
  descriere: "note",
  memo: "note",
};

function headerKey(raw: string): Col | null {
  const cleaned = fold(raw)
    .replace(/\(ron\)/g, "")
    .replace(/[^a-z0-9]+/g, " ")
    .trim();
```

```
  if (cleaned === "kind") return "type";
  return HEADER_MAP[cleaned] ?? null;
}

function parseDateCell(raw: string): string | null {
  const s = raw.trim();
  if (!s) return null;
  for (const fmt of DATE_FORMATS) {
    const d = parse(s, fmt, new Date());
    if (isValid(d) && d.getFullYear() >= 1990 && d.getFullYear() <= 2100) {
      return isoDate(d);
    }
  }
  const iso = parseISO(s);
  if (isValid(iso) && iso.getFullYear() >= 1990 && iso.getFullYear() <= 2100) {
    return isoDate(iso);
  }
  return null;
}

function parseMoney(raw: string): { amount: number; negative: boolean } | null {
  let s = raw.trim();
  if (!s) return null;
  s = s.replace(/\s/g, "").replace(/lei|ron/gi, "");
  const parenNeg = s.startsWith("(") && s.endsWith(")");
  s = s.replace(/[()]/g, "");
  const negative = parenNeg || /^[-−–]/.test(s);
  s = s.replace(/^[-+−–]/, "");
  if (!s) return null;
  const lastComma = s.lastIndexOf(",");
  const lastDot = s.lastIndexOf(".");
  if (lastComma !== -1 && lastDot !== -1) {
    s = lastComma > lastDot ? s.replace(/\./g, "").replace(",", ".") : s.replace(/,/g, "");
  } else if (lastComma !== -1) {
    const frac = s.length - lastComma - 1;
    s = frac === 3 && !s.includes(".") ? s.replace(",", "") : s.replace(",", ".");
  } else if (lastDot !== -1) {
    const frac = s.length - lastDot - 1;
    if (frac === 3 && (s.match(/\./g) ?? []).length > 1) s = s.replace(/\./g, "");
  }
  const n = Number(s);
  if (!Number.isFinite(n) || n <= 0) return null;
  return { amount: Math.round(n * 100) / 100, negative };
}

function parseType(raw: string): TxType | null {
  const s = fold(raw);
  if (["income", "in", "venit", "salary", "+"].includes(s)) return "income";
  if (["expense", "out", "cheltuiala", "cheltuieli", "spend", "-"].includes(s)) return "expense";
  return null;
}

function mapHeaders(row: string[]): Partial<Record<Col, number>> | null {
  const map: Partial<Record<Col, number>> = {};
  let hits = 0;
  row.forEach((cell, i) => {
    const key = headerKey(cell);
    if (!key || map[key] !== undefined) return;
    map[key] = i;
    hits += 1;
  });
  if (hits >= 2 && (map.date !== undefined || map.payee !== undefined)) return map;
  return null;
}

function cell(row: string[], index: number | undefined): string {
  if (index === undefined) return "";
  return row[index]?.trim() ?? "";
}

export type CsvLedger = {
  transactions: Transaction[];
  categories: Category[];
  payees: Payee[];
  products: Product[];
  truncated: boolean;
};

export function csvToLedger(text: string): CsvLedger {
  const rows = parseCsv(text);
```

```
  if (rows.length === 0) throw new Error("That spreadsheet is empty.");

  let start = 0;
  let cols = mapHeaders(rows[0]!);
  if (cols) {
    start = 1;
  } else {
    cols = {
      date: 0,
      type: 1,
      payee: 2,
      category: 3,
      income: 4,
      expense: 5,
      note: 6,
    };
    const probe = parseDateCell(rows[0]![0] ?? "");
    if (!probe && fold(rows[0]![0] ?? "") === "date") start = 1;
  }

  if (cols.date === undefined || cols.payee === undefined) {
    throw new Error("Need Date and Payee columns.");
  }
  if (
    cols.income === undefined &&
    cols.expense === undefined &&
    cols.amount === undefined
  ) {
    throw new Error("Need an Income, Expense, or Amount column.");
  }

  const categories: Category[] = [];
  const payees: Payee[] = [];
  const products: Product[] = [];
  const transactions: Transaction[] = [];

  function categoryId(label: string, type: TxType): string {
    const cleaned = normalizeLabel(label).slice(0, 40) || (type === "income" ? "Income" : "Expense");
    const existing = categories.find((c) => c.type === type && labelsEqual(c.label, cleaned));
    if (existing) return existing.id;
    const row: Category = {
      id: crypto.randomUUID(),
      label: cleaned,
      type,
      color: nextColor(categories),
    };
    categories.push(row);
    return row.id;
  }

  function addPayee(name: string, type: TxType, category: string): string {
    const cleaned = normalizeLabel(name).slice(0, 80);
    const found = payees.find((p) => p.type === type && p.category === category && labelsEqual(p.name, cleaned));
    if (found) return found.name;
    payees.push({ name: cleaned, type, category });
    return cleaned;
  }

  function addProduct(name: string, category: string): string {
    const cleaned = normalizeLabel(name).slice(0, 80);
    if (!cleaned) return "";
    const found = products.find((p) => p.category === category && labelsEqual(p.name, cleaned));
    if (found) return found.name;
    products.push({ name: cleaned, category });
    return cleaned;
  }

  for (let i = start; i < rows.length; i++) {
    if (transactions.length >= MAX_TX) break;
    const row = rows[i]!;
    const date = parseDateCell(cell(row, cols.date));
    const payeeRaw = cell(row, cols.payee);
    if (!date || !payeeRaw) continue;

    const incomeVal = parseMoney(cell(row, cols.income));
    const expenseVal = parseMoney(cell(row, cols.expense));
    const amountVal = parseMoney(cell(row, cols.amount));
    let type = parseType(cell(row, cols.type));
    let amount = 0;

```

```
    if (type === "income" && incomeVal) amount = incomeVal.amount;
    else if (type === "expense" && expenseVal) amount = expenseVal.amount;
    else if (incomeVal && !expenseVal) {
      type = "income";
      amount = incomeVal.amount;
    } else if (expenseVal && !incomeVal) {
      type = "expense";
      amount = expenseVal.amount;
    } else if (amountVal) {
      type = type ?? "expense";
      amount = amountVal.amount;
    } else continue;

    const catLabel = cell(row, cols.category);
    const noteRaw = cell(row, cols.note);
    const productRaw = cell(row, cols.product);
    const qtyRaw = cell(row, cols.quantity);
    const category = categoryId(catLabel, type);
    const qtyParsed = type === "income" && qtyRaw ? Number(qtyRaw.replace(",", ".")) : NaN;
    const quantity =
      type === "income" && Number.isFinite(qtyParsed) && qtyParsed > 0 ? Math.round(qtyParsed * 1000) / 1000 : undefin
ed;
    const product = type === "income" && productRaw ? addProduct(productRaw, category) : undefined;
    transactions.push({
      id: crypto.randomUUID(),
      type,
      amount,
      category,
      date,
      payee: addPayee(payeeRaw, type, category),
      note: noteRaw ? normalizeLabel(noteRaw).slice(0, 160) : undefined,
      product: product || undefined,
      quantity,
    });
  }

  if (transactions.length === 0) {
    throw new Error("No transactions found in that spreadsheet.");
  }

  return {
    transactions,
    categories,
    payees,
    products,
    truncated: rows.length - start > transactions.length && transactions.length >= MAX_TX,
  };
}

export async function readCsvFile(file: File): Promise<CsvLedger> {
  const name = file.name.toLowerCase();
  if (name.endsWith(".json")) throw new Error("Use Import backup for JSON files.");
  const text = await file.text();
  if (text.trim().startsWith("{") || text.trim().startsWith("[")) {
    throw new Error("Use Import backup for JSON files.");
  }
  return csvToLedger(text);
}
```

src/lib/ledger/format.ts

```ts
import { format, parseISO } from "date-fns";

function groupThousands(intPart: string): string {
  return intPart.replace(/\B(?=(\d{3})+(?!\d))/g, ".");
}

export function formatRon(n: number): string {
  const sign = n < 0 ? "−" : "";
  const [intPart, frac] = Math.abs(n).toFixed(2).split(".");
  return `${sign}${groupThousands(intPart!)},${frac} RON`;
}

export function formatRonCompact(n: number): string {
  if (Math.abs(n) < 10000) return formatRon(n);
  return formatAxis(n) + " RON";
}

export function formatAxis(n: number): string {
  const sign = n < 0 ? "−" : "";
  const abs = Math.abs(n);
  if (abs >= 1_000_000) return `${sign}${(abs / 1_000_000).toFixed(1)}M`;
  if (abs >= 1000) return `${sign}${Math.round(abs / 1000)}k`;
  return `${sign}${Math.round(abs)}`;
}

export function formatRonSigned(n: number): string {
  if (n > 0) return `+${formatRon(n)}`;
  if (n < 0) return formatRon(n);
  return formatRon(0);
}

export function formatPct(ratio: number): string {
  const sign = ratio > 0 ? "+" : ratio < 0 ? "−" : "";
  return `${sign}${Math.abs(ratio * 100).toFixed(1)}%`;
}

export function formatPts(ratio: number): string {
  const sign = ratio > 0 ? "+" : ratio < 0 ? "−" : "";
  return `${sign}${Math.abs(ratio * 100).toFixed(1)} pts`;
}

export function formatDay(iso: string): string {
  return format(parseISO(iso), "d MMM");
}

export function formatDayYear(iso: string): string {
  return format(parseISO(iso), "d MMM yyyy");
}

export function formatMonthLabel(d: Date): string {
  return format(d, "MMMM yyyy");
}

export function isoDate(d: Date): string {
  return format(d, "yyyy-MM-dd");
}

export function signedAmount(type: "income" | "expense", amount: number): number {
  return type === "income" ? amount : -amount;
}

export function parseAmount(value: string): number | null {
  const amount = Number(value.replace(",", "."));
  if (!Number.isFinite(amount) || amount <= 0) return null;
  return amount;
}
```

src/lib/ledger/persist.ts

```ts
import {
  FALLBACK_COLOR,
  LEGACY_CATEGORIES,
  findPayee,
  findProduct,
  labelsEqual,
  normalizeLabel,
} from "./categories";
import type {
  Budget,
  Category,
  Payee,
  Product,
  RangePreset,
  RecurringBill,
  Snapshot,
  Transaction,
} from "./types";
import { FREQUENCIES, RANGE_PRESETS } from "./types";

export const STORAGE_KEY = "ledger.v2";
const IDB_NAME = "ledger-app";
const IDB_STORE = "kv";
const IDB_TIMEOUT_MS = 400;
const RANGE_SET = new Set<string>(RANGE_PRESETS);
const FREQ_SET = new Set<string>(FREQUENCIES);

function withTimeout<T>(promise: Promise<T>, ms: number): Promise<T | null> {
  return new Promise((resolve) => {
    const timer = window.setTimeout(() => resolve(null), ms);
    promise.then(
      (value) => {
        window.clearTimeout(timer);
        resolve(value);
      },
      () => {
        window.clearTimeout(timer);
        resolve(null);
      },
    );
  });
}

function openDb(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    const req = indexedDB.open(IDB_NAME, 1);
    req.onupgradeneeded = () => {
      if (!req.result.objectStoreNames.contains(IDB_STORE)) {
        req.result.createObjectStore(IDB_STORE);
      }
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error);
  });
}

function isTx(value: unknown): value is Transaction {
  if (!value || typeof value !== "object") return false;
  const t = value as Partial<Transaction>;
  return (
    typeof t.id === "string" &&
    t.id.length > 0 &&
    (t.type === "income" || t.type === "expense") &&
    typeof t.amount === "number" &&
    Number.isFinite(t.amount) &&
    t.amount > 0 &&
    typeof t.category === "string" &&
    t.category.length > 0 &&
    t.category.length <= 80 &&
    typeof t.date === "string" &&
    /^\d{4}-\d{2}-\d{2}$/.test(t.date) &&
    typeof t.payee === "string" &&
    t.payee.length > 0 &&
    t.payee.length <= 80 &&
    (t.note === undefined || typeof t.note === "string") &&
    (t.product === undefined || (typeof t.product === "string" && t.product.length <= 80)) &&
    (t.quantity === undefined ||
      (typeof t.quantity === "number" &&
```

```
        Number.isFinite(t.quantity) &&
        t.quantity > 0 &&
        t.quantity <= 1_000_000))
  );
}

function isCategory(value: unknown): value is Category {
  if (!value || typeof value !== "object") return false;
  const c = value as Partial<Category>;
  return (
    typeof c.id === "string" &&
    c.id.length > 0 &&
    c.id.length <= 80 &&
    typeof c.label === "string" &&
    c.label.trim().length > 0 &&
    c.label.length <= 40 &&
    (c.type === "income" || c.type === "expense") &&
    typeof c.color === "string" &&
    /^#[0-9a-fA-F]{6}$/.test(c.color)
  );
}

function isPayeeName(value: unknown): value is string {
  return typeof value === "string" && value.trim().length > 0 && value.length <= 80;
}

function isPayeeRow(value: unknown): value is Payee {
  if (!value || typeof value !== "object") return false;
  const p = value as Partial<Payee> & { category?: unknown };
  if (!isPayeeName(p.name) || (p.type !== "income" && p.type !== "expense")) return false;
  if (p.category === undefined) return true;
  return typeof p.category === "string" && p.category.length <= 80;
}

function isProductRow(value: unknown): value is Product {
  if (!value || typeof value !== "object") return false;
  const p = value as Partial<Product>;
  return isPayeeName(p.name) && typeof p.category === "string" && p.category.length <= 80;
}

export function isBudget(value: unknown): value is Budget {
  if (!value || typeof value !== "object") return false;
  const b = value as Partial<Budget>;
  return (
    typeof b.id === "string" &&
    b.id.length > 0 &&
    typeof b.category === "string" &&
    b.category.length > 0 &&
    b.category.length <= 80 &&
    typeof b.monthly === "number" &&
    Number.isFinite(b.monthly) &&
    b.monthly > 0 &&
    b.monthly <= 1_000_000_000
  );
}

export function isRecurring(value: unknown): value is RecurringBill {
  if (!value || typeof value !== "object") return false;
  const r = value as Partial<RecurringBill>;
  return (
    typeof r.id === "string" &&
    r.id.length > 0 &&
    (r.type === "income" || r.type === "expense") &&
    typeof r.amount === "number" &&
    Number.isFinite(r.amount) &&
    r.amount > 0 &&
    r.amount <= 1_000_000_000 &&
    typeof r.category === "string" &&
    r.category.length > 0 &&
    r.category.length <= 80 &&
    typeof r.payee === "string" &&
    r.payee.length > 0 &&
    r.payee.length <= 80 &&
    (r.note === undefined || (typeof r.note === "string" && r.note.length <= 160)) &&
    typeof r.frequency === "string" &&
    FREQ_SET.has(r.frequency) &&
    typeof r.nextDate === "string" &&
    /^\d{4}-\d{2}-\d{2}$/.test(r.nextDate) &&
    typeof r.autoPost === "boolean" &&
    typeof r.paused === "boolean"
```

```
  );
}

export function emptySnapshot(now = Date.now()): Snapshot {
  return {
    v: 2,
    updatedAt: now,
    range: "30d",
    transactions: [],
    categories: [],
    payees: [],
    products: [],
    budgets: [],
    recurrences: [],
  };
}

function uniquePayees(rows: Payee[]): Payee[] {
  const out: Payee[] = [];
  for (const row of rows) {
    const name = normalizeLabel(row.name);
    if (!name || (row.type !== "income" && row.type !== "expense")) continue;
    const category = row.category ?? "";
    if (findPayee(out, name, row.type, category)) continue;
    out.push({ name, type: row.type, category });
  }
  return out.sort(
    (a, b) =>
      a.type.localeCompare(b.type) ||
      a.category.localeCompare(b.category) ||
      a.name.localeCompare(b.name, undefined, { sensitivity: "base" }),
  );
}

function readPayees(
  raw: unknown,
  transactions: Transaction[],
  recurrences: RecurringBill[],
): Payee[] {
  const fromActivity: Payee[] = uniquePayees([
    ...transactions.map((t) => ({ name: t.payee, type: t.type, category: t.category })),
    ...recurrences.map((r) => ({ name: r.payee, type: r.type, category: r.category })),
  ]);
  if (!Array.isArray(raw)) return fromActivity.slice(0, 500);

  let rows: Payee[] = [];
  if (raw.every(isPayeeRow)) {
    for (const row of raw as Array<Payee & { category?: string }>) {
      const name = normalizeLabel(row.name);
      const category = row.category ?? "";
      if (category) {
        rows.push({ name, type: row.type, category });
        continue;
      }
      const used = fromActivity.filter((p) => p.type === row.type && labelsEqual(p.name, name) && p.category);
      if (used.length > 0) rows.push(...used);
      else rows.push({ name, type: row.type, category: "" });
    }
  } else if (raw.every(isPayeeName)) {
    for (const name of raw as string[]) {
      const cleaned = normalizeLabel(name);
      const used = fromActivity.filter((p) => labelsEqual(p.name, cleaned) && p.category);
      if (used.length > 0) {
        rows.push(...used);
        continue;
      }
      rows.push({ name: cleaned, type: "income", category: "" }, { name: cleaned, type: "expense", category: "" });
    }
  } else {
    return fromActivity.slice(0, 500);
  }

  return uniquePayees(rows).slice(0, 500);
}

function uniqueProducts(rows: Product[]): Product[] {
  const out: Product[] = [];
  for (const row of rows) {
    const name = normalizeLabel(row.name);
    if (!name) continue;
```

```
    const category = row.category ?? "";
    if (findProduct(out, name, category)) continue;
    out.push({ name, category });
  }
  return out.sort(
    (a, b) =>
      a.category.localeCompare(b.category) ||
      a.name.localeCompare(b.name, undefined, { sensitivity: "base" }),
  );
}

function readQuantity(value: unknown): number | undefined {
  if (typeof value !== "number" || !Number.isFinite(value) || value <= 0 || value > 1_000_000) {
    return undefined;
  }
  return Math.round(value * 1000) / 1000;
}

function readTransactions(rows: Transaction[]): Transaction[] {
  return rows.map((t) => {
    if (t.type !== "income") {
      return {
        id: t.id,
        type: t.type,
        amount: t.amount,
        category: t.category,
        date: t.date,
        payee: t.payee,
        note: t.note,
      };
    }
    const product = t.product ? normalizeLabel(t.product).slice(0, 80) : undefined;
    return {
      ...t,
      product: product || undefined,
      quantity: readQuantity(t.quantity),
    };
  });
}

function readProducts(raw: unknown, transactions: Transaction[]): Product[] {
  const fromActivity = uniqueProducts(
    transactions
      .filter((t) => t.type === "income" && t.product)
      .map((t) => ({ name: t.product!, category: t.category })),
  );
  if (!Array.isArray(raw)) return fromActivity.slice(0, 500);
  if (!raw.every(isProductRow)) return fromActivity.slice(0, 500);
  return uniqueProducts([...fromActivity, ...(raw as Product[])]).slice(0, 500);
}

function uniqueBudgets(rows: Budget[]): Budget[] {
  const seen = new Set<string>();
  const out: Budget[] = [];
  for (const row of rows) {
    if (seen.has(row.category)) continue;
    seen.add(row.category);
    out.push(row);
  }
  return out;
}

export function snapshotFromParts(input: {
  range: RangePreset;
  transactions: Transaction[];
  categories: Category[];
  payees: unknown;
  products?: unknown;
  budgets?: Budget[];
  recurrences?: RecurringBill[];
  updatedAt?: number;
}): Snapshot {
  const recurrences = input.recurrences ?? [];
  const transactions = readTransactions(input.transactions);
  return {
    v: 2,
    updatedAt: input.updatedAt ?? Date.now(),
    range: input.range,
    transactions,
    categories: input.categories,
```

```
    payees: readPayees(input.payees, transactions, recurrences),
    products: readProducts(input.products, transactions),
    budgets: uniqueBudgets(input.budgets ?? []),
    recurrences,
  };
}

function deriveFromTransactions(transactions: Transaction[]): {
  categories: Category[];
  payees: Payee[];
} {
  const cats = new Map<string, Category>();
  for (const tx of transactions) {
    if (cats.has(tx.category)) continue;
    const legacy = LEGACY_CATEGORIES[tx.category];
    cats.set(tx.category, {
      id: tx.category,
      label: legacy?.label ?? tx.category,
      type: legacy?.type ?? tx.type,
      color: legacy?.color ?? FALLBACK_COLOR,
    });
  }
  return {
    categories: [...cats.values()],
    payees: uniquePayees(transactions.map((t) => ({ name: t.payee, type: t.type, category: t.category }))),
  };
}

function readBudgets(raw: unknown): Budget[] {
  if (!Array.isArray(raw)) return [];
  return uniqueBudgets(raw.filter(isBudget)).slice(0, 100);
}

function readRecurrences(raw: unknown): RecurringBill[] {
  if (!Array.isArray(raw)) return [];
  return (raw.filter(isRecurring) as RecurringBill[]).slice(0, 200);
}

export function asSnapshot(raw: unknown): Snapshot | null {
  if (!raw || typeof raw !== "object") return null;
  const o = raw as Record<string, unknown>;
  if (o.v !== 2 || typeof o.updatedAt !== "number" || !Array.isArray(o.transactions)) {
    return null;
  }
  if (typeof o.range !== "string" || !RANGE_SET.has(o.range)) return null;
  if (!Array.isArray(o.categories) || !Array.isArray(o.payees)) return null;
  if (o.transactions.length > 5000 || !o.transactions.every(isTx)) return null;
  if (o.categories.length > 100 || !o.categories.every(isCategory)) return null;
  if (o.payees.length > 500) return null;
  if (Array.isArray(o.products) && o.products.length > 500) return null;
  const recurrences = readRecurrences(o.recurrences);
  return snapshotFromParts({
    range: o.range as RangePreset,
    transactions: o.transactions as Transaction[],
    categories: o.categories as Category[],
    payees: o.payees as Payee[] | string[],
    products: o.products,
    budgets: readBudgets(o.budgets),
    recurrences,
    updatedAt: o.updatedAt,
  });
}

export function readLocalSnapshot(): Snapshot | null {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return null;
    return asSnapshot(JSON.parse(raw));
  } catch {
    return null;
  }
}

export function writeLocalSnapshot(snap: Snapshot): boolean {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(snap));
    return true;
  } catch {
    return false;
  }
```

```
}

export async function readIdbSnapshot(): Promise<Snapshot | null> {
  try {
    const db = await openDb();
    return await new Promise((resolve, reject) => {
      const tx = db.transaction(IDB_STORE, "readonly");
      const req = tx.objectStore(IDB_STORE).get(STORAGE_KEY);
      req.onsuccess = () => resolve(asSnapshot(req.result));
      req.onerror = () => reject(req.error);
    });
  } catch {
    return null;
  }
}

export async function writeIdbSnapshot(snap: Snapshot): Promise<void> {
  try {
    const db = await openDb();
    await new Promise<void>((resolve, reject) => {
      const tx = db.transaction(IDB_STORE, "readwrite");
      tx.objectStore(IDB_STORE).put(snap, STORAGE_KEY);
      tx.oncomplete = () => resolve();
      tx.onerror = () => reject(tx.error);
    });
  } catch {
    /* private mode / disabled IDB */
  }
}

export function pickNewest(a: Snapshot | null, b: Snapshot | null): Snapshot | null {
  if (a && b) return a.updatedAt >= b.updatedAt ? a : b;
  return a ?? b;
}

export function persistSnapshot(snap: Snapshot): boolean {
  const ok = writeLocalSnapshot(snap);
  void writeIdbSnapshot(snap);
  return ok;
}

export async function loadSnapshot(): Promise<Snapshot | null> {
  const local = readLocalSnapshot();
  if (typeof indexedDB === "undefined") return local;
  const idb = await withTimeout(readIdbSnapshot(), IDB_TIMEOUT_MS);
  return pickNewest(local, idb);
}

export async function parseImport(raw: unknown): Promise<Snapshot> {
  const { importSchema } = await import("./schema");
  const parsed = importSchema.safeParse(raw);
  if (!parsed.success) {
    throw new Error("That file is not a Ledger backup.");
  }
  const data = parsed.data;
  if (Array.isArray(data)) {
    const derived = deriveFromTransactions(data);
    return snapshotFromParts({
      range: "30d",
      transactions: data,
      categories: derived.categories,
      payees: derived.payees,
    });
  }
  if ("v" in data && data.v === 2) {
    return snapshotFromParts({
      ...data,
      budgets: readBudgets("budgets" in data ? data.budgets : []),
      recurrences: readRecurrences("recurrences" in data ? data.recurrences : []),
    });
  }
  const transactions = "transactions" in data ? data.transactions : [];
  const derived = deriveFromTransactions(transactions);
  const range = "range" in data && data.range ? data.range : "30d";
  return snapshotFromParts({
    range,
    transactions,
    categories: derived.categories,
    payees: derived.payees,
    updatedAt: "updatedAt" in data && typeof data.updatedAt === "number" ? data.updatedAt : Date.now(),
```

```
  });
}

export function downloadBackup(snap: Snapshot): void {
  const blob = new Blob([JSON.stringify(snap, null, 2)], {
    type: "application/json",
  });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  const now = new Date();
  const stamp = `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, "0")}-${String(now.getDate()).padStart(2, "0")}`;
  a.href = url;
  a.download = `ledger-${stamp}.json`;
  document.body.appendChild(a);
  a.click();
  a.remove();
  URL.revokeObjectURL(url);
}

export function readFileAsJson(file: File): Promise<unknown> {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onerror = () => reject(new Error("Could not read file."));
    reader.onload = () => {
      try {
        resolve(JSON.parse(String(reader.result)));
      } catch {
        reject(new Error("That file is not valid JSON."));
      }
    };
    reader.readAsText(file);
  });
}
```

src/lib/ledger/recurrence.ts

```ts
import { addMonths, addWeeks, addYears, format, parseISO } from "date-fns";
import type { Frequency, RecurringBill } from "./types";

export const FREQ_LABELS: Record<Frequency, string> = {
  weekly: "Weekly",
  monthly: "Monthly",
  yearly: "Yearly",
};

export function advanceDate(iso: string, frequency: Frequency): string {
  const d = parseISO(`${iso}T00:00:00`);
  const next =
    frequency === "weekly"
      ? addWeeks(d, 1)
      : frequency === "yearly"
        ? addYears(d, 1)
        : addMonths(d, 1);
  return format(next, "yyyy-MM-dd");
}

export type DuePlan = {
  dates: string[];
  nextDate: string;
};

export function planDue(bill: RecurringBill, today: string, max = 24): DuePlan {
  const dates: string[] = [];
  let cursor = bill.nextDate;
  let guard = 0;
  while (cursor <= today && guard < max) {
    dates.push(cursor);
    cursor = advanceDate(cursor, bill.frequency);
    guard += 1;
  }
  return { dates, nextDate: cursor };
}

export function splitRecurrences(
  rows: RecurringBill[],
  today: string,
): { due: RecurringBill[]; upcoming: RecurringBill[]; paused: RecurringBill[] } {
  const due: RecurringBill[] = [];
  const upcoming: RecurringBill[] = [];
  const paused: RecurringBill[] = [];
  for (const row of rows) {
    if (row.paused) paused.push(row);
    else if (row.nextDate <= today) due.push(row);
    else upcoming.push(row);
  }
  const byDate = (a: RecurringBill, b: RecurringBill) =>
    a.nextDate.localeCompare(b.nextDate) || a.payee.localeCompare(b.payee);
  due.sort(byDate);
  upcoming.sort(byDate);
  paused.sort(byDate);
  return { due, upcoming, paused };
}
```

src/lib/ledger/schema.ts

```ts
import { z } from "zod";
import { FREQUENCIES, RANGE_PRESETS } from "./types";

export const transactionSchema = z.object({
  id: z.string().min(1).max(80),
  type: z.enum(["income", "expense"]),
  amount: z.number().finite().positive().max(1_000_000_000),
  category: z.string().trim().min(1).max(80),
  date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  payee: z.string().trim().min(1).max(80),
  note: z.string().trim().max(160).optional(),
  product: z.string().trim().min(1).max(80).optional(),
  quantity: z.number().finite().positive().max(1_000_000).optional(),
});

export const categorySchema = z.object({
  id: z.string().min(1).max(80),
  label: z.string().trim().min(1).max(40),
  type: z.enum(["income", "expense"]),
  color: z.string().regex(/^#[0-9a-fA-F]{6}$/),
});

export const budgetSchema = z.object({
  id: z.string().min(1).max(80),
  category: z.string().trim().min(1).max(80),
  monthly: z.number().finite().positive().max(1_000_000_000),
});

export const recurringSchema = z.object({
  id: z.string().min(1).max(80),
  type: z.enum(["income", "expense"]),
  amount: z.number().finite().positive().max(1_000_000_000),
  category: z.string().trim().min(1).max(80),
  payee: z.string().trim().min(1).max(80),
  note: z.string().trim().max(160).optional(),
  frequency: z.enum(FREQUENCIES),
  nextDate: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  autoPost: z.boolean(),
  paused: z.boolean(),
});

export const payeeSchema = z.object({
  name: z.string().trim().min(1).max(80),
  type: z.enum(["income", "expense"]),
  category: z.string().trim().max(80).optional(),
});

export const productSchema = z.object({
  name: z.string().trim().min(1).max(80),
  category: z.string().trim().max(80),
});

export const snapshotSchema = z.object({
  v: z.literal(2),
  updatedAt: z.number().finite(),
  range: z.enum(RANGE_PRESETS),
  transactions: z.array(transactionSchema).max(5000),
  categories: z.array(categorySchema).max(100),
  payees: z
    .array(z.union([z.string().trim().min(1).max(80), payeeSchema]))
    .max(500),
  products: z.array(productSchema).max(500).optional(),
  budgets: z.array(budgetSchema).max(100).optional(),
  recurrences: z.array(recurringSchema).max(200).optional(),
});

export const legacySnapshotSchema = z.object({
  v: z.literal(1),
  updatedAt: z.number().finite().optional(),
  range: z.enum(RANGE_PRESETS).optional(),
  transactions: z.array(transactionSchema).max(5000),
});

export const importSchema = z.union([
  snapshotSchema,
  legacySnapshotSchema,
  z.array(transactionSchema).max(5000),
  z.object({ transactions: z.array(transactionSchema).max(5000) }),
```

```
79  ]);
```

src/lib/ledger/store.ts

```ts
import { create } from "zustand";
import {
  findCategory,
  findPayee,
  findProduct,
  hasDuplicateLabel,
  labelsEqual,
  nextColor,
  normalizeLabel,
} from "./categories";
import { isoDate } from "./format";
import { csvToLedger } from "./csv";
import {
  downloadBackup,
  emptySnapshot,
  loadSnapshot,
  parseImport,
  persistSnapshot,
  readFileAsJson,
  readLocalSnapshot,
  snapshotFromParts,
} from "./persist";
import { advanceDate, planDue } from "./recurrence";
import type {
  Budget,
  Category,
  CategoryId,
  Payee,
  PostedItem,
  Product,
  RangePreset,
  RecurringBill,
  Snapshot,
  Transaction,
  TxType,
} from "./types";

type RecurringInput = Omit<RecurringBill, "id">;

type LedgerState = {
  hydrated: boolean;
  transactions: Transaction[];
  categories: Category[];
  payees: Payee[];
  products: Product[];
  budgets: Budget[];
  recurrences: RecurringBill[];
  range: RangePreset;
  savedAt: number | null;
  persistOk: boolean;
  hydrate: () => Promise<PostedItem[]>;
  setRange: (range: RangePreset) => void;
  addTransaction: (input: Omit<Transaction, "id">) => void;
  deleteTransaction: (id: string) => void;
  addCategory: (input: { label: string; type: TxType }) => Category | null;
  updateCategory: (id: string, label: string) => boolean;
  deleteCategory: (id: string) => void;
  addPayee: (name: string, type: TxType, category: CategoryId) => string | null;
  renamePayee: (from: string, to: string, type: TxType, category: CategoryId) => boolean;
  deletePayee: (name: string, type: TxType, category: CategoryId) => void;
  addProduct: (name: string, category: CategoryId) => string | null;
  renameProduct: (from: string, to: string, category: CategoryId) => boolean;
  deleteProduct: (name: string, category: CategoryId) => void;
  addBudget: (input: { category: string; monthly: number }) => Budget | null;
  updateBudget: (id: string, monthly: number) => boolean;
  deleteBudget: (id: string) => void;
  addRecurrence: (input: RecurringInput) => { row: RecurringBill; posted: PostedItem[] };
  updateRecurrence: (id: string, patch: Partial<RecurringInput>) => boolean;
  deleteRecurrence: (id: string) => void;
  toggleRecurrencePaused: (id: string) => void;
  postRecurrence: (id: string, today?: string) => PostedItem[];
  skipRecurrence: (id: string) => void;
  postDueRecurrences: (today?: string) => PostedItem[];
  clearAll: () => void;
  exportBackup: () => void;
  importBackup: (file: File) => Promise<void>;
  importCsv: (file: File) => Promise<{ imported: number; truncated: boolean }>;
};
```

```

type Persistable = Pick<
  LedgerState,
  "transactions" | "categories" | "payees" | "products" | "range" | "budgets" | "recurrences"
>;

function snapshotFrom(state: Persistable): Snapshot {
  return snapshotFromParts(state);
}

function commit(patch: Partial<Persistable>) {
  const current = useLedger.getState();
  const next: Persistable = {
    transactions: patch.transactions ?? current.transactions,
    categories: patch.categories ?? current.categories,
    payees: patch.payees ?? current.payees,
    products: patch.products ?? current.products,
    range: patch.range ?? current.range,
    budgets: patch.budgets ?? current.budgets,
    recurrences: patch.recurrences ?? current.recurrences,
  };
  const snap = snapshotFrom(next);
  const persistOk = persistSnapshot(snap);
  useLedger.setState({
    ...next,
    payees: snap.payees,
    products: snap.products,
    budgets: snap.budgets,
    recurrences: snap.recurrences,
    savedAt: snap.updatedAt,
    persistOk,
  });
}

function applySnapshot(snap: Snapshot, persistOk = true) {
  useLedger.setState({
    hydrated: true,
    transactions: snap.transactions,
    categories: snap.categories,
    payees: snap.payees,
    products: snap.products,
    budgets: snap.budgets,
    recurrences: snap.recurrences,
    range: snap.range,
    savedAt: snap.updatedAt,
    persistOk,
  });
}

function todayIso(override?: string): string {
  return override ?? isoDate(new Date());
}

function withPayee(list: Payee[], name: string, type: TxType, category: CategoryId): Payee[] {
  const cleaned = normalizeLabel(name);
  if (!cleaned || !category) return list;
  if (findPayee(list, cleaned, type, category)) return list;
  const loose = list.find((p) => p.type === type && labelsEqual(p.name, cleaned) && p.category === "");
  if (loose) {
    return list.map((p) => (p === loose ? { ...p, category } : p));
  }
  return [...list, { name: cleaned, type, category }];
}

function withProduct(list: Product[], name: string | undefined, category: CategoryId, type: TxType): Product[] {
  if (type !== "income") return list;
  const cleaned = normalizeLabel(name ?? "");
  if (!cleaned || !category) return list;
  if (findProduct(list, cleaned, category)) return list;
  const loose = list.find((p) => labelsEqual(p.name, cleaned) && p.category === "");
  if (loose) {
    return list.map((p) => (p === loose ? { ...p, category } : p));
  }
  return [...list, { name: cleaned, category }];
}

function postBills(
  bills: RecurringBill[],
  ids: Set<string> | null,
  today: string,
```

```
  force: boolean,
): { recurrences: RecurringBill[]; transactions: Transaction[]; payees: Payee[]; posted: PostedItem[] } {
  const current = useLedger.getState();
  const recurrences = bills.map((r) => ({ ...r }));
  const transactions: Transaction[] = [];
  let payees = current.payees;
  const posted: PostedItem[] = [];

  for (const row of recurrences) {
    if (ids && !ids.has(row.id)) continue;
    if (row.paused) continue;
    if (!force && !row.autoPost) continue;
    const { dates, nextDate } = planDue(row, today);
    if (dates.length === 0) continue;
    for (const date of dates) {
      transactions.push({
        id: crypto.randomUUID(),
        type: row.type,
        amount: row.amount,
        category: row.category,
        date,
        payee: row.payee,
        note: row.note,
      });
      posted.push({ payee: row.payee, amount: row.amount, date });
    }
    row.nextDate = nextDate;
    payees = withPayee(payees, row.payee, row.type, row.category);
  }

  return { recurrences, transactions, payees, posted };
}

export const useLedger = create<LedgerState>((_set, get) => ({
  hydrated: false,
  transactions: [],
  categories: [],
  payees: [],
  products: [],
  budgets: [],
  recurrences: [],
  range: "30d",
  savedAt: null,
  persistOk: true,

  hydrate: async () => {
    if (get().hydrated) return [];
    try {
      const local = readLocalSnapshot();
      if (local) {
        applySnapshot(local);
      } else {
        const snap = emptySnapshot();
        applySnapshot(snap, persistSnapshot(snap));
      }
      const newest = await loadSnapshot();
      if (newest && newest.updatedAt > (get().savedAt ?? 0)) {
        applySnapshot(newest);
        persistSnapshot(newest);
      }
      return get().postDueRecurrences();
    } catch {
      applySnapshot(emptySnapshot(), false);
      return [];
    }
  },

  setRange: (range) => commit({ range }),

  addTransaction: (input) => {
    const row: Transaction = { ...input, id: crypto.randomUUID() };
    commit({
      transactions: [row, ...get().transactions],
      payees: withPayee(get().payees, row.payee, row.type, row.category),
      products: withProduct(get().products, row.product, row.category, row.type),
    });
  },

  deleteTransaction: (id) => {
    commit({ transactions: get().transactions.filter((t) => t.id !== id) });
```

```
  },

  addCategory: ({ label, type }) => {
    const cleaned = normalizeLabel(label);
    if (!cleaned || cleaned.length > 40) return null;
    const current = get().categories;
    if (hasDuplicateLabel(current, type, cleaned)) return null;
    const row: Category = {
      id: crypto.randomUUID(),
      label: cleaned,
      type,
      color: nextColor(current),
    };
    commit({ categories: [...current, row] });
    return row;
  },

  updateCategory: (id, label) => {
    const cleaned = normalizeLabel(label);
    if (!cleaned || cleaned.length > 40) return false;
    const current = get().categories;
    const existing = findCategory(current, id);
    if (!existing) return false;
    if (hasDuplicateLabel(current, existing.type, cleaned, id)) return false;
    commit({
      categories: current.map((c) => (c.id === id ? { ...c, label: cleaned } : c)),
    });
    return true;
  },

  deleteCategory: (id) => {
    commit({
      categories: get().categories.filter((c) => c.id !== id),
      payees: get().payees.filter((p) => p.category !== id),
      products: get().products.filter((p) => p.category !== id),
      budgets: get().budgets.filter((b) => b.category !== id),
      recurrences: get().recurrences.filter((r) => r.category !== id),
    });
  },

  addPayee: (name, type, category) => {
    const cleaned = normalizeLabel(name);
    if (!cleaned || cleaned.length > 80 || !category) return null;
    const existing = findPayee(get().payees, cleaned, type, category);
    if (existing) return existing.name;
    commit({ payees: withPayee(get().payees, cleaned, type, category) });
    return cleaned;
  },

  renamePayee: (from, to, type, category) => {
    const cleaned = normalizeLabel(to);
    if (!cleaned || cleaned.length > 80) return false;
    const current = get().payees;
    const source = findPayee(current, from, type, category);
    if (!source) return false;
    const clash = findPayee(current, cleaned, type, category);
    if (clash && clash !== source) return false;
    commit({
      payees: current.map((p) => (p === source ? { ...p, name: cleaned } : p)),
      transactions: get().transactions.map((t) =>
        t.type === type && t.category === category && t.payee === source.name
          ? { ...t, payee: cleaned }
          : t,
      ),
      recurrences: get().recurrences.map((r) =>
        r.type === type && r.category === category && r.payee === source.name
          ? { ...r, payee: cleaned }
          : r,
      ),
    });
    return true;
  },

  deletePayee: (name, type, category) => {
    const existing = findPayee(get().payees, name, type, category);
    if (!existing) return;
    commit({
      payees: get().payees.filter((p) => p !== existing),
    });
  },
```

```
319
320   addProduct: (name, category) => {
321     const cleaned = normalizeLabel(name);
322     if (!cleaned || cleaned.length > 80 || !category) return null;
323     const existing = findProduct(get().products, cleaned, category);
324     if (existing) return existing.name;
325     commit({ products: withProduct(get().products, cleaned, category, "income") });
326     return cleaned;
327   },
328
329   renameProduct: (from, to, category) => {
330     const cleaned = normalizeLabel(to);
331     if (!cleaned || cleaned.length > 80) return false;
332     const current = get().products;
333     const source = findProduct(current, from, category);
334     if (!source) return false;
335     const clash = findProduct(current, cleaned, category);
336     if (clash && clash !== source) return false;
337     commit({
338       products: current.map((p) => (p === source ? { ...p, name: cleaned } : p)),
339       transactions: get().transactions.map((t) =>
340         t.type === "income" && t.category === category && t.product && labelsEqual(t.product, source.name)
341           ? { ...t, product: cleaned }
342           : t,
343       ),
344     });
345     return true;
346   },
347
348   deleteProduct: (name, category) => {
349     const existing = findProduct(get().products, name, category);
350     if (!existing) return;
351     commit({
352       products: get().products.filter((p) => p !== existing),
353     });
354   },
355
356   addBudget: ({ category, monthly }) => {
357     if (!Number.isFinite(monthly) || monthly <= 0) return null;
358     const cat = findCategory(get().categories, category);
359     if (!cat || cat.type !== "expense") return null;
360     if (get().budgets.some((b) => b.category === category)) return null;
361     const row: Budget = { id: crypto.randomUUID(), category, monthly };
362     commit({ budgets: [...get().budgets, row] });
363     return row;
364   },
365
366   updateBudget: (id, monthly) => {
367     if (!Number.isFinite(monthly) || monthly <= 0) return false;
368     const current = get().budgets;
369     if (!current.some((b) => b.id === id)) return false;
370     commit({
371       budgets: current.map((b) => (b.id === id ? { ...b, monthly } : b)),
372     });
373     return true;
374   },
375
376   deleteBudget: (id) => {
377     commit({ budgets: get().budgets.filter((b) => b.id !== id) });
378   },
379
380   addRecurrence: (input) => {
381     const row: RecurringBill = { ...input, id: crypto.randomUUID() };
382     commit({
383       recurrences: [...get().recurrences, row],
384       payees: withPayee(get().payees, row.payee, row.type, row.category),
385     });
386     const posted =
387       !row.paused && row.autoPost ? get().postDueRecurrences() : [];
388     return { row, posted };
389   },
390
391   updateRecurrence: (id, patch) => {
392     const current = get().recurrences;
393     if (!current.some((r) => r.id === id)) return false;
394     const recurrences = current.map((r) => (r.id === id ? { ...r, ...patch } : r));
395     const next = recurrences.find((r) => r.id === id);
396     commit({
397       recurrences,
398       payees: next ? withPayee(get().payees, next.payee, next.type, next.category) : get().payees,
```

```
    });
    return true;
  },

  deleteRecurrence: (id) => {
    commit({ recurrences: get().recurrences.filter((r) => r.id !== id) });
  },

  toggleRecurrencePaused: (id) => {
    const current = get().recurrences;
    commit({
      recurrences: current.map((r) => (r.id === id ? { ...r, paused: !r.paused } : r)),
    });
  },

  postRecurrence: (id, today) => {
    const result = postBills(get().recurrences, new Set([id]), todayIso(today), true);
    if (result.posted.length === 0) return [];
    commit({
      recurrences: result.recurrences,
      transactions: [...result.transactions, ...get().transactions],
      payees: result.payees,
    });
    return result.posted;
  },

  skipRecurrence: (id) => {
    const current = get().recurrences;
    const row = current.find((r) => r.id === id);
    if (!row) return;
    commit({
      recurrences: current.map((r) =>
        r.id === id ? { ...r, nextDate: advanceDate(r.nextDate, r.frequency) } : r,
      ),
    });
  },

  postDueRecurrences: (today) => {
    const result = postBills(get().recurrences, null, todayIso(today), false);
    if (result.posted.length === 0) return [];
    commit({
      recurrences: result.recurrences,
      transactions: [...result.transactions, ...get().transactions],
      payees: result.payees,
    });
    return result.posted;
  },

  clearAll: () => {
    commit({
      transactions: [],
      categories: [],
      payees: [],
      products: [],
      budgets: [],
      recurrences: [],
      range: "30d",
    });
  },

  exportBackup: () => {
    downloadBackup(snapshotFrom(get()));
  },

  importBackup: async (file) => {
    const raw = await readFileAsJson(file);
    const snap = await parseImport(raw);
    commit({
      transactions: snap.transactions,
      categories: snap.categories,
      payees: snap.payees,
      products: snap.products,
      budgets: snap.budgets,
      recurrences: snap.recurrences,
      range: snap.range,
    });
  },

  importCsv: async (file) => {
    const incoming = csvToLedger(await file.text());
```

```
    const current = get();
    const budgets = current.budgets.flatMap((b) => {
      const old = findCategory(current.categories, b.category);
      if (!old) return [];
      const next = incoming.categories.find(
        (c) => c.type === old.type && labelsEqual(c.label, old.label),
      );
      return next ? [{ ...b, category: next.id }] : [];
    });
    const recurrences = current.recurrences.flatMap((r) => {
      const old = findCategory(current.categories, r.category);
      if (!old) return [];
      const next = incoming.categories.find(
        (c) => c.type === old.type && labelsEqual(c.label, old.label),
      );
      if (!next) return [];
      return [{ ...r, category: next.id, payee: findPayee(incoming.payees, r.payee, r.type, next.id)?.name ?? r.payee
}];
    });
    commit({
      transactions: incoming.transactions,
      categories: incoming.categories,
      payees: incoming.payees,
      products: incoming.products,
      budgets,
      recurrences,
    });
    return { imported: incoming.transactions.length, truncated: incoming.truncated };
  },
}));
```

src/lib/ledger/types.ts

```ts
export const RANGE_PRESETS = ["7d", "30d", "90d", "6m", "1y"] as const;
export type RangePreset = (typeof RANGE_PRESETS)[number];

export const TX_TYPES = ["income", "expense"] as const;
export type TxType = (typeof TX_TYPES)[number];

export const FREQUENCIES = ["weekly", "monthly", "yearly"] as const;
export type Frequency = (typeof FREQUENCIES)[number];

export type CategoryId = string;

export type Category = {
  id: CategoryId;
  label: string;
  type: TxType;
  color: string;
};

export type Transaction = {
  id: string;
  type: TxType;
  amount: number;
  category: CategoryId;
  date: string;
  payee: string;
  note?: string;
  product?: string;
  quantity?: number;
};

export type Budget = {
  id: string;
  category: CategoryId;
  monthly: number;
};

export type RecurringBill = {
  id: string;
  type: TxType;
  amount: number;
  category: CategoryId;
  payee: string;
  note?: string;
  frequency: Frequency;
  nextDate: string;
  autoPost: boolean;
  paused: boolean;
};

export type Payee = {
  name: string;
  type: TxType;
  category: CategoryId;
};

export type Product = {
  name: string;
  category: CategoryId;
};

export type Snapshot = {
  v: 2;
  updatedAt: number;
  range: RangePreset;
  transactions: Transaction[];
  categories: Category[];
  payees: Payee[];
  products: Product[];
  budgets: Budget[];
  recurrences: RecurringBill[];
};

export type PostedItem = {
  payee: string;
  amount: number;
  date: string;
};
```

src/lib/multiplayer/index.ts

```ts
export { P2PRoom, defaultIceServers } from "./p2p";
export type {
  PeerInfo,
  P2PRoomOptions,
  SignalKind,
  PeerRow,
  SignalRow,
  RtcPollResponse,
} from "./p2p";
```

src/lib/multiplayer/p2p.ts

```ts
/**
 * Full-mesh WebRTC rooms: one RTCPeerConnection per remote peer, signaled
 * through /api/rtc (see signaling.server.ts), game data flowing directly
 * browser-to-browser afterwards. Client-authoritative by construction — see
 * the multiplayer-p2p skill for when NOT to use this.
 *
 * Negotiation follows the "perfect negotiation" pattern: on a glare (both
 * sides offering at once) the polite peer — the lexicographically smaller id —
 * rolls back and accepts, so pairs converge without wedging.
 */

export type SignalKind = "offer" | "answer" | "ice";

/**
 * Wire contract between this client and the signaling relay the app provides
 * at /api/rtc (see the multiplayer-p2p skill for a reference implementation).
 * The client only needs these shapes — the relay's storage is the app's choice.
 */
export interface PeerRow {
  id: string;
  name: string;
}
export interface SignalRow {
  id: number;
  from: string;
  kind: SignalKind;
  payload: unknown;
}
export interface RtcPollResponse {
  peers: PeerRow[];
  signals: SignalRow[];
}

export interface PeerInfo {
  id: string;
  name: string;
  connectionState: RTCPeerConnectionState;
  /** Selected local ICE candidate type: host | srflx | prflx | relay. */
  candidateType: string | null;
  /** Data-channel ping RTT (ms), measured every 2s once connected. */
  rttMs: number | null;
}

export interface P2PRoomOptions {
  room: string;
  selfId: string;
  name?: string;
  /** Defaults to VITE_STUN_URLS (comma-separated) or Google public STUN. */
  iceServers?: RTCIceServer[];
  onPeersChanged?: (peers: PeerInfo[]) => void;
  /** Fires for both the unreliable "state" and reliable "reliable" channels. */
  onMessage?: (from: string, data: unknown, channel: "state" | "reliable") => void;
  /** Fires once, on the first successful signaling poll (registration). */
  onConnected?: () => void;
}

interface PeerSlot {
  pc: RTCPeerConnection;
  state?: RTCDataChannel;
  reliable?: RTCDataChannel;
  makingOffer: boolean;
  ignoreOffer: boolean;
  /** ICE candidates that arrived before the remote description (buffered). */
  pendingCandidates: RTCIceCandidateInit[];
  /** Last time this pair made observable progress toward connected. */
  lastProgressAt: number;
  /** Watchdog recreations (dialer) / stall windows (receiver) so far. */
  recoveryAttempts: number;
  /** Gave up after MAX_RECOVERY_ATTEMPTS — excluded from fast-poll pressure. */
  terminal?: boolean;
  /** One-shot: pc was already recreated to absorb a failing remote offer. */
  recreatedForOffer?: boolean;
  info: PeerInfo;
  pingSentAt?: number;
}

const FAST_POLL_MS = 400;
const IDLE_POLL_MS = 2000;
```

```
const PING_INTERVAL_MS = 2000;
const STALL_MS = 10_000;
const MAX_RECOVERY_ATTEMPTS = 3;
const SIGNAL_RETRY_DELAYS_MS = [250, 750];

export function defaultIceServers(): RTCIceServer[] {
  const urls = (import.meta.env.VITE_STUN_URLS as string | undefined)
    ?.split(",")
    .map((u) => u.trim())
    .filter(Boolean);
  // Two independent providers: ICE queries all of them in parallel during
  // gathering, so either one being unreachable costs nothing.
  return [
    {
      urls: urls?.length ? urls : ["stun:stun.l.google.com:19302", "stun:stun.cloudflare.com:3478"],
    },
  ];
}

export class P2PRoom {
  private readonly opts: P2PRoomOptions;
  private readonly peers = new Map<string, PeerSlot>();
  /** Per-remote-peer signal delivery chains (order-preserving). */
  private readonly signalQueues = new Map<string, Promise<void>>();
  private cursor = 0;
  private pollTimer: ReturnType<typeof setTimeout> | null = null;
  private pingTimer: ReturnType<typeof setInterval> | null = null;
  private closed = false;
  private everPolled = false;
  private lastPeersFingerprint = "";

  constructor(opts: P2PRoomOptions) {
    this.opts = opts;
  }

  /**
   * The first poll IS the join: it registers this peer and returns the
   * roster. A failed first poll (cold DB, offline tab) must not strand the
   * room: the loop and timers start regardless and the next poll retries.
   */
  async join(): Promise<void> {
    try {
      await this.pollOnce();
    } catch {
      // First poll can fail transiently; the scheduled loop below retries.
    }
    if (this.closed) return;
    this.schedulePoll(this.anyPairConnecting() ? FAST_POLL_MS : IDLE_POLL_MS);
    this.pingTimer = setInterval(() => {
      this.pingAll();
      this.watchdog();
    }, PING_INTERVAL_MS);
  }

  close(): void {
    this.closed = true;
    if (this.pollTimer) clearTimeout(this.pollTimer);
    if (this.pingTimer) clearInterval(this.pingTimer);
    for (const slot of this.peers.values()) slot.pc.close();
    this.peers.clear();
    // Leaving the roster is the teardown broadcast: everyone's next poll
    // drops this peer and closes their side of the pair.
    void fetch("/api/rtc", {
      method: "POST",
      headers: { "content-type": "application/json" },
      body: JSON.stringify({ op: "leave", room: this.opts.room, peer: this.opts.selfId }),
      keepalive: true,
    }).catch(() => {});
  }

  /** Send on the unreliable game-state channel (drops stale packets). */
  broadcast(data: unknown): void {
    const wire = JSON.stringify({ t: "d", d: data });
    for (const slot of this.peers.values()) {
      if (slot.state?.readyState === "open") slot.state.send(wire);
    }
  }

  /** Send reliably (ordered) to one peer, or to all when peerId is omitted. */
  send(data: unknown, peerId?: string): void {
```

```
    const wire = JSON.stringify({ t: "d", d: data });
    const targets = peerId ? [this.peers.get(peerId)] : [...this.peers.values()];
    for (const slot of targets) {
      if (slot?.reliable?.readyState === "open") slot.reliable.send(wire);
    }
  }

  peerList(): PeerInfo[] {
    return [...this.peers.values()].map((s) => ({ ...s.info }));
  }

  // ── signaling loop ──────────────────────────────────────────────

  private schedulePoll(delay: number): void {
    if (this.closed) return;
    if (this.pollTimer) clearTimeout(this.pollTimer);
    this.pollTimer = setTimeout(() => void this.poll(), delay);
  }

  private anyPairConnecting(): boolean {
    for (const s of this.peers.values()) {
      // Terminal pairs (NAT-blocked after all recovery attempts) must not pin
      // the session at the 400ms fast-poll rate.
      if (s.terminal) continue;
      if (s.info.connectionState !== "connected") return true;
    }
    return false;
  }

  private async pollOnce(): Promise<void> {
    const params = new URLSearchParams({
      room: this.opts.room,
      peer: this.opts.selfId,
      name: this.opts.name ?? "",
      since: String(this.cursor),
    });
    const res = await fetch(`/api/rtc?${params}`);
    if (this.closed) return;
    if (!res.ok) throw new Error(`signaling poll failed: ${res.status}`);
    const body = (await res.json()) as RtcPollResponse;
    if (this.closed) return;
    if (!this.everPolled) {
      this.everPolled = true;
      this.opts.onConnected?.();
    }
    this.reconcileRoster(body.peers);
    const roster = new Set(body.peers.map((p) => p.id));
    for (const sig of body.signals) {
      this.cursor = Math.max(this.cursor, sig.id);
      await this.onSignal(sig.from, sig.kind, sig.payload, roster);
      if (this.closed) return;
    }
  }

  private async poll(): Promise<void> {
    if (this.closed) return;
    try {
      await this.pollOnce();
    } catch {
      // Transient poll failures are expected (tab sleep, deploy roll); retry.
    }
    this.schedulePoll(this.anyPairConnecting() ? FAST_POLL_MS : IDLE_POLL_MS);
  }

  private reconcileRoster(peers: { id: string; name: string }[]): void {
    const alive = new Set(peers.map((p) => p.id));
    for (const p of peers) {
      if (p.id === this.opts.selfId) continue;
      const existing = this.peers.get(p.id);
      if (existing) {
        existing.info.name = p.name;
      } else {
        // Exactly one side dials each pair; the other waits for the offer.
        this.connectTo(p.id, p.name, this.opts.selfId > p.id);
      }
    }
    for (const [id, slot] of this.peers) {
      if (!alive.has(id)) {
        slot.pc.close();
        this.peers.delete(id);
```

```
      }
    }
    this.emitPeers();
  }

  // ── per-pair connection ──────────────────────────────────────────

  private connectTo(peerId: string, name: string, initiator: boolean): PeerSlot | null {
    if (this.closed) return null;
    const pc = new RTCPeerConnection({
      iceServers: this.opts.iceServers ?? defaultIceServers(),
    });
    const slot: PeerSlot = {
      pc,
      makingOffer: false,
      ignoreOffer: false,
      pendingCandidates: [],
      lastProgressAt: Date.now(),
      recoveryAttempts: 0,
      info: {
        id: peerId,
        name,
        connectionState: pc.connectionState,
        candidateType: null,
        rttMs: null,
      },
    };
    this.peers.set(peerId, slot);

    pc.onicecandidate = (e) => {
      if (e.candidate) void this.sendSignal(peerId, "ice", e.candidate.toJSON());
    };
    pc.onconnectionstatechange = () => {
      slot.info.connectionState = pc.connectionState;
      if (pc.connectionState === "connecting" || pc.connectionState === "connected") {
        slot.lastProgressAt = Date.now();
      }
      if (pc.connectionState === "connected") {
        slot.recoveryAttempts = 0;
        slot.terminal = false;
        void this.readCandidateType(slot);
      }
      this.emitPeers();
      if (pc.connectionState === "failed") {
        // Refires negotiationneeded → a fresh offer through signaling, so a
        // lost offer or dead path cannot wedge the pair (glare-safe).
        pc.restartIce();
      }
      if (pc.connectionState === "failed" || pc.connectionState === "disconnected") {
        this.schedulePoll(FAST_POLL_MS);
      }
    };
    pc.onnegotiationneeded = async () => {
      try {
        slot.makingOffer = true;
        await pc.setLocalDescription();
        await this.sendSignal(peerId, "offer", pc.localDescription!.toJSON());
      } catch {
        // A failed offer is retried on the next negotiationneeded.
      } finally {
        slot.makingOffer = false;
      }
    };
    pc.ondatachannel = (e) => this.attachChannel(slot, e.channel);

    if (initiator) {
      // Creating the channels triggers negotiationneeded → the offer.
      this.attachChannel(
        slot,
        pc.createDataChannel("state", { ordered: false, maxRetransmits: 0 }),
      );
      this.attachChannel(slot, pc.createDataChannel("reliable", { ordered: true }));
    }
    return slot;
  }

  private attachChannel(slot: PeerSlot, channel: RTCDataChannel): void {
    if (channel.label === "state") slot.state = channel;
    else slot.reliable = channel;
    channel.onopen = () => {
```

```
      slot.lastProgressAt = Date.now();
    };
    channel.onmessage = (e) => {
      let msg: { t: string; d?: unknown };
      try {
        msg = JSON.parse(e.data as string) as { t: string; d?: unknown };
      } catch {
        return;
      }
      if (msg.t === "ping") {
        if (slot.state?.readyState === "open") {
          slot.state.send(JSON.stringify({ t: "pong" }));
        }
      } else if (msg.t === "pong") {
        if (slot.pingSentAt) {
          slot.info.rttMs = Math.round(performance.now() - slot.pingSentAt);
          slot.pingSentAt = undefined;
          this.emitPeers();
        }
      } else {
        this.opts.onMessage?.(
          slot.info.id,
          msg.d,
          channel.label === "state" ? "state" : "reliable",
        );
      }
    };
  }

  /** Apply buffered ICE candidates once a remote description is in place. */
  private async flushPendingCandidates(slot: PeerSlot): Promise<void> {
    while (slot.pendingCandidates.length > 0) {
      const candidate = slot.pendingCandidates.shift()!;
      try {
        await slot.pc.addIceCandidate(candidate);
      } catch (err) {
        if (!slot.ignoreOffer) console.warn("[p2p] addIceCandidate failed:", err);
      }
      if (this.closed) return;
    }
  }

  private async onSignal(
    from: string,
    kind: SignalKind,
    payload: unknown,
    roster: Set<string>,
  ): Promise<void> {
    if (this.closed) return;
    let slot = this.peers.get(from);
    if (!slot) {
      // New peers dial us in the same poll that adds them to the roster.
      // Signals outlive membership, so drop senders the roster doesn't vouch for.
      if (!roster.has(from)) return;
      const created = this.connectTo(from, "", false);
      if (!created) return;
      slot = created;
    }
    const polite = this.opts.selfId < from;

    try {
      if (kind === "offer" || kind === "answer") {
        const description = payload as RTCSessionDescriptionInit;
        const collision =
          kind === "offer" && (slot.makingOffer || slot.pc.signalingState !== "stable");
        slot.ignoreOffer = !polite && collision;
        if (slot.ignoreOffer) return;
        try {
          await slot.pc.setRemoteDescription(description); // implicit rollback when polite
        } catch (err) {
          // A pc resumed from suspend can be unable to take any new remote
          // offer (stale DTLS fingerprint). Rebuild the pair once and apply
          // the same offer to the fresh pc before giving up.
          if (kind !== "offer" || slot.recreatedForOffer) throw err;
          const attempts = slot.recoveryAttempts;
          const name = slot.info.name;
          slot.pc.close();
          this.peers.delete(from);
          const fresh = this.connectTo(from, name, false);
          if (!fresh) return;
```

```
          fresh.recoveryAttempts = attempts;
          fresh.recreatedForOffer = true;
          slot = fresh;
          await slot.pc.setRemoteDescription(description);
        }
        if (this.closed) return;
        await this.flushPendingCandidates(slot);
        if (this.closed) return;
        if (kind === "offer") {
          await slot.pc.setLocalDescription();
          if (this.closed) return;
          await this.sendSignal(from, "answer", slot.pc.localDescription!.toJSON());
        }
      } else if (kind === "ice") {
        const candidate = payload as RTCIceCandidateInit;
        if (!slot.pc.remoteDescription) {
          // Candidate raced ahead of its SDP — hold it until the description
          // lands (flushed after every successful setRemoteDescription).
          slot.pendingCandidates.push(candidate);
          return;
        }
        try {
          await slot.pc.addIceCandidate(candidate);
        } catch (err) {
          // The enclosing catch would swallow a rethrow; log the real signal.
          if (!slot.ignoreOffer) console.warn("[p2p] addIceCandidate failed:", err);
        }
      }
    } catch {
      // Negotiation errors resolve on the next offer cycle; state is visible
      // to the app via connectionState.
    }
  }

  /**
   * Signals are serialized per remote peer (a candidate must never overtake
   * its SDP into the DB) and retried on failure with short backoff.
   */
  private sendSignal(to: string, kind: SignalKind, payload: unknown): Promise<void> {
    const prev = this.signalQueues.get(to) ?? Promise.resolve();
    const next = prev.then(() => this.postSignal(to, kind, payload));
    this.signalQueues.set(
      to,
      next.catch(() => {}),
    );
    return next;
  }

  private async postSignal(to: string, kind: SignalKind, payload: unknown): Promise<void> {
    for (let attempt = 0; ; attempt++) {
      if (this.closed) return;
      try {
        const res = await fetch("/api/rtc", {
          method: "POST",
          headers: { "content-type": "application/json" },
          body: JSON.stringify({
            op: "signal",
            room: this.opts.room,
            from: this.opts.selfId,
            to,
            kind,
            payload,
          }),
        });
        if (res.ok) return;
        throw new Error(`signal POST failed: ${res.status}`);
      } catch (err) {
        if (attempt >= SIGNAL_RETRY_DELAYS_MS.length) {
          // Delivery gave up; the pair converges on the next offer cycle (or
          // the watchdog rebuilds it). Logged once so failures are visible.
          console.warn(`[p2p] signal ${kind} to ${to} failed after retries`, err);
          return;
        }
        await new Promise((r) => setTimeout(r, SIGNAL_RETRY_DELAYS_MS[attempt]));
      }
    }
  }

  // ── diagnostics + recovery ──────────────────────────────────────────
```

```
  private pingAll(): void {
    const wire = JSON.stringify({ t: "ping" });
    for (const slot of this.peers.values()) {
      if (slot.state?.readyState !== "open") continue;
      const stale =
        slot.pingSentAt !== undefined && performance.now() - slot.pingSentAt > 2 * PING_INTERVAL_MS;
      if (slot.pingSentAt === undefined || stale) {
        // A lost pong must not freeze rttMs forever: expire and re-ping.
        slot.pingSentAt = performance.now();
        slot.state.send(wire);
      }
    }
  }

  /**
   * Stuck-pair recovery, piggybacked on the ping interval. A pair that has
   * made no progress for STALL_MS gets rebuilt by the dialer with a FRESH
   * RTCPeerConnection (new DTLS identity — fixes the suspend/resume
   * fingerprint wedge). After MAX_RECOVERY_ATTEMPTS the pair is terminal:
   * visible to the app as its last connectionState, ignored by fast-poll.
   */
  private watchdog(): void {
    if (this.closed) return;
    const now = Date.now();
    for (const [peerId, slot] of this.peers) {
      // pc.close() and some suspend/resume wedges never fire
      // connectionstatechange — read the LIVE state so a silently-dead pc
      // still trips the stall timer instead of hiding behind a cached
      // "connected". Only live progress states refresh the stall clock.
      const live = slot.pc.connectionState;
      if (live !== slot.info.connectionState) {
        slot.info.connectionState = live;
        if (live === "connecting" || live === "connected") slot.lastProgressAt = now;
        this.emitPeers();
      }
      if (slot.terminal || live === "connected") continue;
      if (now - slot.lastProgressAt <= STALL_MS) continue;
      if (slot.recoveryAttempts >= MAX_RECOVERY_ATTEMPTS) {
        slot.terminal = true;
        this.emitPeers();
        continue;
      }
      slot.recoveryAttempts += 1;
      slot.lastProgressAt = now; // re-arm the stall window
      if (this.opts.selfId > peerId) {
        // We are the dialer: rebuild the pair from scratch.
        const { name } = slot.info;
        const attempts = slot.recoveryAttempts;
        slot.pc.close();
        this.peers.delete(peerId);
        const fresh = this.connectTo(peerId, name, true);
        if (fresh) fresh.recoveryAttempts = attempts;
        this.schedulePoll(FAST_POLL_MS);
      }
      // Receiver side: count the stall window and wait for the dialer's
      // fresh offer (onSignal absorbs it, recreating our pc if needed).
    }
  }

  private async readCandidateType(slot: PeerSlot): Promise<void> {
    // relay = TURN (none configured by default); srflx/host = direct path.
    try {
      const stats = await slot.pc.getStats();
      let selected: RTCIceCandidatePairStats | undefined;
      stats.forEach((s) => {
        if (s.type === "candidate-pair" && (s as RTCIceCandidatePairStats).nominated) {
          selected = s as RTCIceCandidatePairStats;
        }
      });
      const localId = selected?.localCandidateId;
      if (localId) {
        const local = stats.get(localId) as { candidateType?: string } | undefined;
        slot.info.candidateType = local?.candidateType ?? null;
        this.emitPeers();
      }
    } catch {
      // getStats is best-effort diagnostics only.
    }
  }

```

```
  private emitPeers(): void {
    // Only notify when something observable actually changed — React state
    // setters otherwise re-render consumers on every poll/ping.
    const list = this.peerList();
    const fingerprint = JSON.stringify(
      list.map((p) => [p.id, p.name, p.connectionState, p.candidateType, p.rttMs]),
    );
    if (fingerprint === this.lastPeersFingerprint) return;
    this.lastPeersFingerprint = fingerprint;
    this.opts.onPeersChanged?.(list);
  }
}
```

src/lib/og/site.json

```
{
  "title": "Ledger",
  "color": "08090B"
}
```

src/lib/preview-embedder-origin.ts

```ts
export function isGrokEmbedderOrigin(origin: string): boolean {
  try {
    const url = new URL(origin);
    if (url.protocol !== "https:" && url.protocol !== "http:") return false;
    const host = url.hostname.toLowerCase();
    if (host === "grok.com" || host.endsWith(".grok.com")) return true;
    if (host === "localhost" || host === "127.0.0.1" || host === "[::1]") return true;
    return false;
  } catch {
    return false;
  }
}

export function isSandboxPreviewGuestHost(hostname: string): boolean {
  const host = hostname.toLowerCase();
  return host === "grok-sandbox.com" || host.endsWith(".grok-sandbox.com");
}

function isRemintPreviewPair(guestHost: string, parentHost: string): boolean {
  const guest = guestHost.toLowerCase();
  const parent = parentHost.toLowerCase();
  const sep = ".preview.";
  const i = guest.indexOf(sep);
  if (i <= 0) return false;
  const label = guest.slice(0, i);
  const rest = guest.slice(i + sep.length);
  if (label.includes(".") || !rest.includes(".")) return false;
  return parent === rest || parent === `grok.${rest}`;
}

export function resolveParentEmbedderOrigin(
  parentIsSelf: boolean,
  referrer: string,
  ancestorOrigin?: string | null,
  guestHostname: string = "",
): string | null {
  if (parentIsSelf) return null;
  for (const candidate of [referrer, ancestorOrigin ?? ""].filter(Boolean)) {
    try {
      const url = new URL(
        candidate.includes("://") ? candidate : `https://${candidate}`,
      );
      if (url.protocol !== "https:" && url.protocol !== "http:") continue;
      if (isGrokEmbedderOrigin(url.origin)) return url.origin;
      if (
        isSandboxPreviewGuestHost(guestHostname) ||
        isRemintPreviewPair(guestHostname, url.hostname)
      ) {
        return url.origin;
      }
    } catch {
      // try next candidate
    }
  }
  return null;
}
```

src/lib/preview-host-bridge.ts

```ts
/**
 * Guest side of the grok-web ↔ sandbox preview postMessage bridge.
 *
 * Activates only when this page is framed by an allowlisted Grok embedder.
 * Top-level runs (download/export, local `npm run dev`, deployed sites) noop.
 */

import { z } from "zod";
import { resolveParentEmbedderOrigin } from "./preview-embedder-origin";

export {
  isGrokEmbedderOrigin,
  isSandboxPreviewGuestHost,
  resolveParentEmbedderOrigin,
} from "./preview-embedder-origin";

export const PREVIEW_BRIDGE_CHANNEL = "grok-preview-bridge" as const;
export const PREVIEW_BRIDGE_VERSION = 1 as const;

const EnvelopeSchema = z.object({
  channel: z.literal(PREVIEW_BRIDGE_CHANNEL),
  version: z.number().int().positive(),
  type: z.string().min(1),
});

const HelloSchema = EnvelopeSchema.extend({
  type: z.literal("hello"),
});

const NavigateSchema = EnvelopeSchema.extend({
  type: z.literal("navigate"),
  path: z.string().min(1),
});

const HistorySchema = EnvelopeSchema.extend({
  type: z.literal("history"),
  delta: z.union([z.literal(-1), z.literal(1)]),
});

export type PreviewHostBridgeOptions = {
  /** Prefer the app router when available; falls back to history.pushState. */
  navigate?: (path: string) => void;
  /** Best-effort registered paths for host autosuggest (may be empty). */
  getRoutePaths?: () => string[];
};

export function isSafeBridgePath(path: string): boolean {
  if (!path.startsWith("/") || path.startsWith("//") || path.includes("\\")) {
    return false;
  }
  try {
    const resolved = new URL(path, "https://preview.invalid");
    return resolved.origin === "https://preview.invalid";
  } catch {
    return false;
  }
}

/**
 * Install host↔guest messaging. Returns a dispose function.
 * Noops (returns a no-op dispose) when not embedded under a Grok parent.
 */
export function installPreviewHostBridge(
  options: PreviewHostBridgeOptions = {},
): () => void {
  if (typeof window === "undefined") return () => {};

  const ancestorOrigin =
    typeof location.ancestorOrigins !== 'undefined' && location.ancestorOrigins.length > 0
      ? location.ancestorOrigins[0]
      : null;
  const parentOrigin = resolveParentEmbedderOrigin(
    window.parent === window,
    document.referrer,
    ancestorOrigin,
    window.location.hostname,
  );
  if (parentOrigin === null) return () => {};
```

```

  const ROOT_STATE_KEY = "__grokPreviewBridgeRoot";
  const originalPushState = window.history.pushState.bind(window.history);
  const originalReplaceState = window.history.replaceState.bind(window.history);

  const isAtHistoryRoot = () => {
    const state = window.history.state;
    return Boolean(
      state && typeof state === "object" && state[ROOT_STATE_KEY] === true,
    );
  };

  // Floor for chrome Back: only the first install in a fresh history stack is
  // root. Full document navigations reinstall this bridge on a deep URL; if we
  // re-stamped that entry as root, Back would no-op while earlier in-preview
  // history still exists. Preserve an existing tag (bfcache / SPA remount).
  try {
    const current = window.history.state;
    const alreadyTagged =
      current !== null &&
      typeof current === "object" &&
      Object.prototype.hasOwnProperty.call(current, ROOT_STATE_KEY);
    if (!alreadyTagged) {
      const isRoot = window.history.length <= 1;
      const marked =
        current && typeof current === "object"
          ? { ...current, [ROOT_STATE_KEY]: isRoot }
          : { [ROOT_STATE_KEY]: isRoot };
      originalReplaceState(marked, "", window.location.href);
    }
  } catch {
    // ignore if the document cannot be marked
  }

  const post = (message: object) => {
    window.parent.postMessage(message, parentOrigin);
  };

  const reportLocation = () => {
    post({
      channel: PREVIEW_BRIDGE_CHANNEL,
      version: PREVIEW_BRIDGE_VERSION,
      type: "location",
      path: window.location.pathname || "/",
      search: window.location.search,
      hash: window.location.hash,
    });
  };

  const reportRoutes = () => {
    const paths = options.getRoutePaths?.() ?? [];
    post({
      channel: PREVIEW_BRIDGE_CHANNEL,
      version: PREVIEW_BRIDGE_VERSION,
      type: "routes",
      paths,
    });
  };

  const defaultNavigate = (path: string) => {
    if (!isSafeBridgePath(path)) return;
    try {
      const url = new URL(path, window.location.origin);
      if (url.origin !== window.location.origin) return;
      const next = `${url.pathname}${url.search}${url.hash}`;
      window.history.pushState(window.history.state, "", next);
      window.dispatchEvent(new PopStateEvent("popstate", { state: window.history.state }));
    } catch {
      // ignore malformed paths
    }
  };

  const navigate = (path: string) => {
    if (!isSafeBridgePath(path)) return;
    if (options.navigate) {
      options.navigate(path);
      return;
    }
    defaultNavigate(path);
  };
```

```
159
160   const announce = () => {
161     reportLocation();
162     reportRoutes();
163     post({
164       channel: PREVIEW_BRIDGE_CHANNEL,
165       version: PREVIEW_BRIDGE_VERSION,
166       type: "ready",
167     });
168   };
169
170   const onMessage = (event: MessageEvent) => {
171     if (event.source !== window.parent) return;
172     if (event.origin !== parentOrigin) return;
173
174     const envelope = EnvelopeSchema.safeParse(event.data);
175     if (!envelope.success || envelope.data.version !== PREVIEW_BRIDGE_VERSION) return;
176
177     // Host re-handshake: it may have (re)mounted after our install-time
178     // announce, or asked before we hydrated. Announce again.
179     if (envelope.data.type === "hello") {
180       if (!HelloSchema.safeParse(event.data).success) return;
181       announce();
182       return;
183     }
184
185     if (envelope.data.type === "navigate") {
186       const parsed = NavigateSchema.safeParse(event.data);
187       if (!parsed.success) return;
188       navigate(parsed.data.path);
189       // Router navigations often update location asynchronously; report after a tick.
190       queueMicrotask(reportLocation);
191       return;
192     }
193
194     if (envelope.data.type === "history") {
195       const parsed = HistorySchema.safeParse(event.data);
196       if (!parsed.success) return;
197       // Do not history.go(-1) off the first entry — that leaves the preview.
198       if (parsed.data.delta === -1 && isAtHistoryRoot()) return;
199       // Location sync comes from the popstate listener once history settles.
200       window.history.go(parsed.data.delta);
201     }
202   };
203
204   const onPopState = () => {
205     reportLocation();
206   };
207
208   // Same-document `#` navigations fire hashchange, not popstate / pushState.
209   const onHashChange = () => {
210     reportLocation();
211   };
212
213   // Patch history so in-app SPA navigations sync the host address bar.
214   window.history.pushState = (data, unused, url) => {
215     const next =
216       data && typeof data === "object"
217         ? { ...data, [ROOT_STATE_KEY]: false }
218         : data;
219     originalPushState(next, unused, url);
220     reportLocation();
221   };
222   window.history.replaceState = (data, unused, url) => {
223     const next =
224       isAtHistoryRoot()
225         ? {
226             ...(data && typeof data === "object" ? data : {}),
227             [ROOT_STATE_KEY]: true,
228           }
229         : data;
230     originalReplaceState(next, unused, url);
231     reportLocation();
232   };
233
234   window.addEventListener("message", onMessage);
235   window.addEventListener("popstate", onPopState);
236   window.addEventListener("hashchange", onHashChange);
237
238   announce();
```

```
239
240   return () => {
241     window.removeEventListener("message", onMessage);
242     window.removeEventListener("popstate", onPopState);
243     window.removeEventListener("hashchange", onHashChange);
244     window.history.pushState = originalPushState;
245     window.history.replaceState = originalReplaceState;
246   };
247 }
248
249 /** Collect static path patterns from a TanStack route tree (best-effort). */
250 export function collectRoutePathsFromTree(routeTree: unknown): string[] {
251   const paths = new Set<string>();
252
253   const walk = (node: unknown) => {
254     if (!node || typeof node !== "object") return;
255     const record = node as {
256       fullPath?: unknown;
257       path?: unknown;
258       children?: unknown;
259     };
260     const full =
261       typeof record.fullPath === "string"
262         ? record.fullPath
263         : typeof record.path === "string"
264           ? record.path
265           : null;
266     if (full !== null && full !== "") {
267       paths.add(full.startsWith("/") ? full : `/${full}`);
268     } else if (full === "") {
269       paths.add("/");
270     }
271     const children = record.children;
272     if (Array.isArray(children)) {
273       for (const child of children) walk(child);
274     } else if (children && typeof children === "object") {
275       for (const child of Object.values(children as Record<string, unknown>)) {
276         walk(child);
277       }
278     }
279   };
280
281   walk(routeTree);
282   return [...paths];
283 }
```

src/lib/utils.ts

```ts
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

src/routeTree.gen.ts

```ts
/* eslint-disable */

// @ts-nocheck

// noinspection JSUnusedGlobalSymbols

// This file was automatically generated by TanStack Router.
// You should NOT make any changes in this file as it will be overwritten.
// Additionally, you should also exclude this file from your linter and/or formatter to prevent it from being checked or modified.

import { Route as rootRouteImport } from './routes/__root'
import { Route as IndexRouteImport } from './routes/index'

const IndexRoute = IndexRouteImport.update({
  id: '/',
  path: '/',
  getParentRoute: () => rootRouteImport,
} as any)

export interface FileRoutesByFullPath {
  '/': typeof IndexRoute
}
export interface FileRoutesByTo {
  '/': typeof IndexRoute
}
export interface FileRoutesById {
  __root__: typeof rootRouteImport
  '/': typeof IndexRoute
}
export interface FileRouteTypes {
  fileRoutesByFullPath: FileRoutesByFullPath
  fullPaths: '/'
  fileRoutesByTo: FileRoutesByTo
  to: '/'
  id: '__root__' | '/'
  fileRoutesById: FileRoutesById
}
export interface RootRouteChildren {
  IndexRoute: typeof IndexRoute
}

declare module '@tanstack/react-router' {
  interface FileRoutesByPath {
    '/': {
      id: '/'
      path: '/'
      fullPath: '/'
      preLoaderRoute: typeof IndexRouteImport
      parentRoute: typeof rootRouteImport
    }
  }
}

const rootRouteChildren: RootRouteChildren = {
  IndexRoute: IndexRoute,
}
export const routeTree = rootRouteImport
  ._addFileChildren(rootRouteChildren)
  ._addFileTypes<FileRouteTypes>()

import type { getRouter } from './router.tsx'
import type { createStart } from '@tanstack/react-start'
declare module '@tanstack/react-start' {
  interface Register {
    ssr: true
    router: Awaited<ReturnType<typeof getRouter>>
  }
}
```

src/router.tsx

```
import { createRouter } from "@tanstack/react-router";
import { AppErrorComponent } from "@/lib/error-component";
import { routeTree } from "./routeTree.gen";

export function getRouter() {
  return createRouter({ routeTree, defaultErrorComponent: AppErrorComponent });
}
```

src/routes/__root.tsx

```tsx
import { createRootRoute, HeadContent, Outlet, Scripts } from "@tanstack/react-router";
import { AuthProvider } from "@/lib/auth/provider";
import { PreviewHostBridge } from "@/components/preview-host-bridge";
import { TooltipProvider } from "@/components/ui/tooltip";
import { Toaster } from "sonner";
import appCss from "../styles.css?url";

const APP_NAME = "Ledger";

export const Route = createRootRoute({
  head: () => ({
    meta: [
      { charSet: "utf-8" },
      { name: "viewport", content: "width=device-width, initial-scale=1, maximum-scale=1, viewport-fit=cover, interactive-widget=resizes-content" },
      { title: APP_NAME },
      {
        name: "description",
        content:
          "Private money tracker. Cashflow, spend mix, and transactions in RON — stored on your device.",
      },
      { name: "theme-color", content: "#08090b" },
    ],
    links: [
      { rel: "icon", type: "image/svg+xml", href: "/favicon.svg" },
      { rel: "stylesheet", href: appCss },
      { rel: "manifest", href: "/__grok/manifest.webmanifest" },
      { rel: "apple-touch-icon", href: "/__grok/icon-180.png" },
    ],
  }),
  component: () => (
    <html lang="en" className="dark antialiased" suppressHydrationWarning>
      <head>
        <HeadContent />
      </head>
      <body>
        <PreviewHostBridge />
        <AuthProvider>
          <TooltipProvider>
            <Outlet />
            <Toaster
              theme="dark"
              position="top-center"
              toastOptions={{
                className: "font-sans",
              }}
            />
          </TooltipProvider>
        </AuthProvider>
        <Scripts />
      </body>
    </html>
  ),
});
```

src/routes/index.tsx

```
import { createFileRoute } from "@tanstack/react-router";
import { Dashboard } from "@/components/ledger/dashboard";

export const Route = createFileRoute("/")({
  loader: () => ({ nowIso: new Date().toISOString() }),
  component: Home,
});

function Home() {
  const { nowIso } = Route.useLoaderData();
  return <Dashboard nowIso={nowIso} />;
}
```

src/styles.css

```
@import "tailwindcss";
@import "tw-animate-css";
@import "@fontsource/ibm-plex-sans/latin-400.css";
@import "@fontsource/ibm-plex-sans/latin-500.css";
@import "@fontsource/ibm-plex-sans/latin-600.css";
@import "@fontsource/ibm-plex-sans/latin-ext-400.css";
@import "@fontsource/ibm-plex-sans/latin-ext-500.css";
@import "@fontsource/ibm-plex-sans/latin-ext-600.css";
@import "@fontsource/ibm-plex-mono/latin-400.css";
@import "@fontsource/ibm-plex-mono/latin-500.css";
@import "@fontsource/ibm-plex-mono/latin-ext-400.css";
@import "@fontsource/ibm-plex-mono/latin-ext-500.css";

@theme {
  --font-sans: "IBM Plex Sans", ui-sans-serif, system-ui, sans-serif;
  --font-mono: "IBM Plex Mono", ui-monospace, "SF Mono", Menlo, Consolas, monospace;

  --color-background: #08090b;
  --color-foreground: #e8eaed;
  --color-card: #101216;
  --color-card-foreground: #e8eaed;
  --color-popover: #16191e;
  --color-popover-foreground: #e8eaed;
  --color-primary: #c5ccd6;
  --color-primary-foreground: #08090b;
  --color-secondary: #16191e;
  --color-secondary-foreground: #e8eaed;
  --color-muted: #16191e;
  --color-muted-foreground: #8b919a;
  --color-accent: #1c2026;
  --color-accent-foreground: #e8eaed;
  --color-destructive: #c08984;
  --color-destructive-foreground: #08090b;
  --color-border: #23272e;
  --color-input: #23272e;
  --color-ring: #c5ccd6;
  --color-income: #8fa89a;
  --color-expense: #c08984;
  --color-subtle: #5c636c;
  --color-chart-line: #c5ccd6;
  --color-chart-grid: #23272e;

  --radius-sm: 0.375rem;
  --radius-md: 0.5rem;
  --radius-lg: 0.75rem;
  --radius-xl: 1rem;
  --radius-2xl: 1.25rem;
  --radius-3xl: 1.5rem;
  --radius: 0.75rem;

  --shadow-border: 0 0 0 1px rgb(255 255 255 / 0.08);
  --shadow-border-hover: 0 0 0 1px rgb(255 255 255 / 0.13);
}

@layer base {
  html {
    color-scheme: dark;
    background-color: var(--color-background);
    color: var(--color-foreground);
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
    -webkit-text-size-adjust: 100%;
    text-size-adjust: 100%;
    touch-action: manipulation;
  }

  body {
    min-height: 100dvh;
    background-color: var(--color-background);
    color: var(--color-foreground);
    font-family: var(--font-sans);
    touch-action: manipulation;
  }

  button:not(:disabled),
  [role="button"]:not(:disabled) {
    cursor: pointer;
  }
```

```

  * {
    border-color: var(--color-border);
  }

  h1,
  h2,
  h3 {
    text-wrap: balance;
  }

  p {
    text-wrap: pretty;
  }

  input,
  select,
  textarea {
    font-size: 16px;
    scroll-margin-top: 1.25rem;
    scroll-margin-bottom: 6.5rem;
  }

  input[type="date"]::-webkit-calendar-picker-indicator {
    opacity: 0.55;
    filter: invert(1);
    cursor: pointer;
  }

  input[type="number"]::-webkit-outer-spin-button,
  input[type="number"]::-webkit-inner-spin-button {
    -webkit-appearance: none;
    margin: 0;
  }

  input[type="number"] {
    -moz-appearance: textfield;
  }
}

@media (prefers-reduced-motion: reduce) {
  .stagger-in {
    animation: none !important;
    opacity: 1 !important;
    transform: none !important;
    filter: none !important;
  }
}

@keyframes ledger-in {
  from {
    opacity: 0;
    transform: translateY(8px);
    filter: blur(2px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
    filter: blur(0);
  }
}

.stagger-in {
  animation: ledger-in 400ms cubic-bezier(0.22, 1, 0.36, 1) both;
}

.safe-bottom {
  padding-bottom: max(1.25rem, env(safe-area-inset-bottom));
}

.drawer-sheet {
  bottom: var(--keyboard-inset, 0px);
  max-height: min(90dvh, calc(100dvh - var(--keyboard-inset, 0px)));
}
```

public/favicon.svg

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 32 32">
  <rect width="32" height="32" rx="7" fill="#08090b"/>
  <rect x="7" y="18" width="4.5" height="7" rx="1" fill="#6e7884"/>
  <rect x="13.75" y="12" width="4.5" height="13" rx="1" fill="#c5ccd6"/>
  <rect x="20.5" y="8" width="4.5" height="17" rx="1" fill="#8b949f"/>
</svg>
```

public/sw.js

```
/* Ledger offline copy — open instantly from the phone, then refresh. */
const CACHE = "ledger-offline-v3";
const FETCH_MS = 4000;

function timeout(ms) {
  return new Promise((_, reject) => {
    setTimeout(() => reject(new Error("timeout")), ms);
  });
}

async function put(request, response, isDoc) {
  if (!response || !response.ok) return;
  const cache = await caches.open(CACHE);
  const copy = response.clone();
  if (isDoc) await cache.put(new Request("/"), copy);
  else await cache.put(request, copy);
}

async function refresh(request, isDoc) {
  try {
    const response = await fetch(request);
    await put(request, response, isDoc);
  } catch {
    /* keep the cached copy */
  }
}

self.addEventListener("install", (event) => {
  self.skipWaiting();
  event.waitUntil(
    caches.open(CACHE).then((cache) => cache.addAll(["/", "/favicon.svg"]).catch(() => undefined)),
  );
});

self.addEventListener("activate", (event) => {
  event.waitUntil(
    caches
      .keys()
      .then((keys) => Promise.all(keys.filter((key) => key !== CACHE).map((key) => caches.delete(key)))),
  );
});

self.addEventListener("fetch", (event) => {
  const request = event.request;
  if (request.method !== "GET") return;

  const url = new URL(request.url);
  if (url.origin !== self.location.origin) return;
  if (url.pathname === "/sw.js") return;
  if (url.searchParams.get("install") === "1") return;

  const isAsset = url.pathname.startsWith("/assets/");
  const isDoc = request.mode === "navigate" || request.destination === "document";

  event.respondWith(
    (async () => {
      const cached = await caches.match(request);
      const home = isDoc ? await caches.match("/") : undefined;
      const stale = cached || home;

      if (isAsset && cached) return cached;
      if (isDoc && stale) {
        void refresh(request, true);
        return stale;
      }

      try {
        const response = await Promise.race([fetch(request), timeout(FETCH_MS)]);
        await put(request, response, isDoc);
        return response;
      } catch {
        if (stale) return stale;
        if (isDoc) {
          return new Response(
            `<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-wi
dth,initial-scale=1"><title>Ledger</title><style>body{margin:0;min-height:100dvh;background:#08090b;color:#e8eaed;font
-family:-apple-system,sans-serif;display:grid;place-items:center;padding:24px;text-align:center}p{margin:0;max-width:2
0rem;line-height:1.45}</style></head><body><p>Ledger is opening from this phone. Close it and tap the icon again.</p><
```

```
    /body></html>`,
76              { headers: { "content-type": "text/html; charset=utf-8" } },
77            );
78          }
79          return Response.error();
80        }
81      })(),
82    );
83  });
```

package.json

```
{
  "name": "app-builder-workspace",
  "private": true,
  "sideEffects": false,
  "type": "module",
  "overrides": {
    "nf3": "0.3.17"
  },
  "scripts": {
    "dev": "node scripts/with-app-env.mjs vite dev --host 0.0.0.0 --port 8080",
    "build": "node scripts/with-app-env.mjs vite build && npm run db:migrate",
    "db:migrate": "node scripts/migrate.mjs",
    "build:dev": "node scripts/with-app-env.mjs vite build --mode development",
    "preview": "node scripts/with-app-env.mjs vite preview",
    "typecheck": "tsc --noEmit",
    "check:auth": "node scripts/check-auth-invariant.mjs",
    "test": "node --test 'scripts/**/*.test.mjs'",
    "lint": "eslint .",
    "format": "prettier --write ."
  },
  "dependencies": {
    "@electric-sql/pglite": "^0.5.4",
    "@fontsource/ibm-plex-mono": "^5.3.0",
    "@fontsource/ibm-plex-sans": "^5.3.0",
    "@hookform/resolvers": "^5.7.0",
    "@radix-ui/react-accordion": "^1.2.12",
    "@radix-ui/react-alert-dialog": "^1.1.15",
    "@radix-ui/react-avatar": "^1.1.11",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-collapsible": "^1.1.12",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-label": "^2.1.8",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-progress": "^1.1.8",
    "@radix-ui/react-radio-group": "^1.3.8",
    "@radix-ui/react-scroll-area": "^1.2.10",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.8",
    "@radix-ui/react-slider": "^1.3.6",
    "@radix-ui/react-slot": "^1.2.4",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
    "@radix-ui/react-toggle": "^1.1.10",
    "@radix-ui/react-toggle-group": "^1.1.11",
    "@radix-ui/react-tooltip": "^1.2.8",
    "@tailwindcss/vite": "^4.3.0",
    "@tanstack/react-query": "^5.101.0",
    "@tanstack/react-router": "^1.170.0",
    "@tanstack/react-start": "^1.168.0",
    "@tanstack/react-table": "^8.21.0",
    "@tanstack/router-plugin": "^1.168.0",
    "better-auth": "~1.6.30",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^4.0.0",
    "kysely": "^0.28.5",
    "lucide-react": "^0.510.0",
    "pg": "^8.16.3",
    "react": "^19.2.0",
    "react-day-picker": "^9.14.0",
    "react-dom": "^19.2.0",
    "react-hook-form": "^7.54.0",
    "react-resizable-panels": "^4.6.5",
    "recharts": "^2.13.0",
    "sonner": "^2.0.7",
    "tailwind-merge": "^3.5.0",
    "tailwindcss": "^4.3.0",
    "tw-animate-css": "^1.3.4",
    "vaul": "^1.1.2",
    "zod": "^4.4.0",
    "zustand": "^5.0.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.20.0",
    "@types/node": "^22.16.5",
    "@types/pg": "^8.11.10",
```

```
    "@types/react": "^19.2.0",
    "@types/react-dom": "^19.2.0",
    "@vitejs/plugin-react": "^5.2.0",
    "eslint": "^9.20.0",
    "eslint-config-prettier": "^10.1.1",
    "eslint-plugin-prettier": "^5.2.6",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.20",
    "globals": "^15.15.0",
    "lightningcss": "^1.28.0",
    "nitro": "3.0.260610-beta",
    "playwright": "^1.62.0",
    "prettier": "^3.4.0",
    "typescript": "^5.7.0",
    "typescript-eslint": "^8.56.1",
    "vite": "^8.2.0"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "isolatedModules": true,
    "skipLibCheck": true,
    // `src/lib/db.ts` imports scripts/migration-plan.mjs; checkJs types the
    // implementation itself instead of a hand-written declaration beside it.
    "allowJs": true,
    "checkJs": true,
    "noEmit": true,
    "types": ["vite/client", "node"],
    "baseUrl": ".",
    "paths": { "@/*": ["./src/*"] }
  },
  "include": ["src", "server"]
}
```

vite.config.ts

```ts
import { readdirSync } from "node:fs";
import { join } from "node:path";
import type { Plugin } from "vite";
import { defineConfig } from "vite";
import { tanstackStart } from "@tanstack/react-start/plugin/vite";
import viteReact from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";
import { nitro } from "nitro/vite";
// @ts-expect-error JS plugin alongside the TS vite config
import { grokPwaPlugin } from "./scripts/grok-pwa-plugin.mjs";
// @ts-expect-error JS plugin alongside the TS vite config
import { appEnvPlugin } from "./scripts/app-env-plugin.mjs";
import { isMigrationFile } from "./scripts/migration-plan.mjs";

/** The files `src/lib/db.ts` globs — same directory, same non-recursive scope. */
function hasGlobbedMigrations(root: string): boolean {
  try {
    return readdirSync(join(root, "migrations")).some(isMigrationFile);
  } catch {
    return false;
  }
}

/**
 * Finish PGLite bootstrap during dev-server setup (before traffic). Vite awaits
 * async `configureServer` hooks. Production: `src/lib/db` kicks `ensureDbReady`
 * on import.
 *
 * Vite awaiting the hook puts this on time-to-first-render, so an app with no
 * migrations — no schema to apply — skips it entirely rather than paying for a
 * PGLite instance it never queries.
 */
function pgliteBootstrapPlugin(): Plugin {
  return {
    name: "app-builder:pglite-bootstrap",
    apply: "serve",
    async configureServer(server) {
      if (!hasGlobbedMigrations(server.config.root)) return;
      try {
        const mod = (await server.ssrLoadModule("/src/lib/db.ts")) as {
          ensureDbReady?: () => Promise<void>;
        };
        if (typeof mod.ensureDbReady === "function") {
          await mod.ensureDbReady();
        }
      } catch (err) {
        console.error("[app-builder] DB bootstrap failed:", err);
        throw err;
      }
    },
  };
}

/**
 * Live-preview OAuth popup — handled HERE so the agent never has to create a
 * `/auth/popup` route (and cannot break it by scaffolding a React page that
 * paints the full app shell in the popup).
 *
 * `signIn` (client.ts) opens `/auth/popup?providerId=…` in a top-level window.
 * This middleware runs before TanStack Start, calls `handleAuthPopupRequest`,
 * and returns the 302 / completion HTML. Deployed apps do not use the popup
 * (full-page OAuth redirect), so `apply: "serve"` is enough.
 */
function authPopupPlugin(): Plugin {
  return {
    name: "app-builder:auth-popup",
    apply: "serve",
    configureServer(server) {
      // Register immediately (not in a returned post-hook) so we run BEFORE
      // TanStack Start / the SPA HTML fallback. A model-authored
      // `src/routes/auth/popup.tsx` React page must never win this path.
      server.middlewares.use(async (req, res, next) => {
        try {
          const rawUrl = req.url ?? "";
          const pathOnly = rawUrl.split("?", 1)[0] ?? "";
          if (pathOnly !== "/auth/popup") {
            next();
            return;
```

```
          }
          if ((req.method ?? "GET").toUpperCase() !== "GET") {
            res.statusCode = 405;
            res.setHeader("content-type", "text/plain; charset=utf-8");
            res.end("Method Not Allowed");
            return;
          }

          const host = String(
            req.headers["x-forwarded-host"] ?? req.headers.host ?? "localhost:8080",
          );
          const proto = String(
            req.headers["x-forwarded-proto"] ??
              ((req.socket as { encrypted?: boolean } | undefined)?.encrypted ? "https" : "http"),
          );
          const requestHeaders = new Headers();
          for (const [key, value] of Object.entries(req.headers)) {
            if (value === undefined) continue;
            if (Array.isArray(value)) {
              for (const v of value) requestHeaders.append(key, v);
            } else {
              requestHeaders.set(key, value);
            }
          }
          // Ensure Host is the public preview host so Better Auth's dynamic
          // baseURL / redirect_uri match the popup origin.
          if (!requestHeaders.has("host")) requestHeaders.set("host", host);

          const request = new Request(`${proto}://${host}${rawUrl}`, {
            method: "GET",
            headers: requestHeaders,
          });

          const mod = (await server.ssrLoadModule("/src/lib/auth/popup.server.ts")) as {
            handleAuthPopupRequest: (req: Request) => Promise<Response>;
          };
          const response = await mod.handleAuthPopupRequest(request);

          res.statusCode = response.status;
          // Preserve multiple Set-Cookie headers (OAuth state + session).
          const setCookies =
            typeof response.headers.getSetCookie === "function"
              ? response.headers.getSetCookie()
              : [];
          response.headers.forEach((value, key) => {
            if (key.toLowerCase() === "set-cookie") return;
            res.setHeader(key, value);
          });
          for (const cookie of setCookies) {
            res.appendHeader("set-cookie", cookie);
          }
          const body = Buffer.from(await response.arrayBuffer());
          res.end(body);
        } catch (err) {
          console.error("[app-builder] /auth/popup handler failed:", err);
          if (!res.headersSent) {
            res.statusCode = 500;
            res.setHeader("content-type", "text/plain; charset=utf-8");
            res.end("auth popup failed");
          }
        }
      });
    },
  };
}

// `0.0.0.0:8080` is the live-preview contract — don't change host/port.
// The dev server starts once `src/router.tsx` and `src/routes/` exist — see
// AGENTS.md § "First scaffold".
export default defineConfig(({ command, isPreview }) => ({
  server: {
    host: "0.0.0.0",
    port: 8080,
    strictPort: true,
  },
  preview: {
    host: "127.0.0.1",
    port: 8081,
    strictPort: true,
  },
```

```
  resolve: { tsconfigPaths: true },
  plugins: [
    pgliteBootstrapPlugin(),
    // Before tanstackStart so /auth/popup never falls through to the SPA.
    authPopupPlugin(),
    // Dev-only /__app-env, read by scripts/check-auth-invariant.mjs.
    appEnvPlugin(),
    // PWA head + ?install=1 tutorial page; runs before Start/Nitro.
    grokPwaPlugin(),
    tailwindcss(),
    tanstackStart(),
    ...(command === "build" || isPreview
      ? [
          nitro({
            preset: "vercel",
            // Auto-registers server/middleware/* (the PWA install page +
            // manifest + head-tag middleware). Nitro v3 defaults serverDir to
            // false, so removing this silently unwires /?install=1 on deploys.
            serverDir: "./server",
          }),
        ]
      : []),
    viteReact(),
  ],
}));
```

eslint.config.mjs

```
import js from "@eslint/js";
import prettier from "eslint-config-prettier";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
import globals from "globals";
import tseslint from "typescript-eslint";

/** Flat ESLint config for the TanStack Start app-builder template. */
export default tseslint.config(
  {
    ignores: [
      "dist/**",
      ".output/**",
      ".vercel/**",
      ".nitro/**",
      "node_modules/**",
      "src/routeTree.gen.ts",
    ],
  },
  js.configs.recommended,
  ...tseslint.configs.recommended,
  {
    files: ["**/*.{ts,tsx,js,jsx,mjs,cjs}"],
    languageOptions: {
      ecmaVersion: 2022,
      globals: { ...globals.browser, ...globals.node },
    },
    plugins: {
      "react-hooks": reactHooks,
      "react-refresh": reactRefresh,
    },
    rules: {
      ...reactHooks.configs.recommended.rules,
      "react-refresh/only-export-components": [
        "warn",
        { allowConstantExport: true },
      ],
      "@typescript-eslint/no-unused-vars": [
        "warn",
        { argsIgnorePattern: "^_", varsIgnorePattern: "^_" },
      ],
      "@typescript-eslint/no-explicit-any": "off",
    },
  },
  // Disable rules that conflict with Prettier formatting.
  prettier,
);
```

scripts/with-app-env.mjs

```
#!/usr/bin/env node
/**
 * Run a command with `.grok/app-env.json` merged into its environment.
 *
 * `dev`, `build` and `preview` all route through this wrapper, so the dev
 * server, the built bundle and the preview server can never disagree about
 * `VITE_AUTH_ENABLED` — a divergence that only shows up as a built-output
 * mismatch long after the fact. Anything that starts Vite directly bypasses it.
 *
 * Only `VITE_`-prefixed keys are honored: the file is a build flag carrier, not
 * a secret store, and only `VITE_` vars reach the browser anyway. A real
 * `process.env` entry always wins, so an explicit override still works.
 *
 * That precedence also means the file governs this workspace only. A deployed
 * build runs with the provider's project env, where the deployer sets
 * `VITE_AUTH_ENABLED` itself (today unconditionally `"true"`), so the deployed
 * flag is the platform's, not this file's.
 *
 * Vite picks the values up because `loadEnv` prefix-matches entries already in
 * `process.env`, which is why the merge has to happen before Vite starts.
 */
import { spawn } from "node:child_process";
import { readFileSync, realpathSync } from "node:fs";
import { dirname, join } from "node:path";
import { fileURLToPath } from "node:url";

export const APP_ENV_REL_PATH = ".grok/app-env.json";

const VITE_PREFIX = "VITE_";

/**
 * Parse an app-env document, keeping only `VITE_`-prefixed string entries.
 * Anything unparseable is an empty environment — a workspace without the file
 * must behave exactly like today (auth on, no overrides).
 */
export function parseAppEnv(text) {
  let parsed;
  try {
    parsed = JSON.parse(text);
  } catch {
    return {};
  }
  if (parsed === null || typeof parsed !== "object" || Array.isArray(parsed)) return {};
  const env = {};
  for (const [key, value] of Object.entries(parsed)) {
    if (!key.startsWith(VITE_PREFIX)) continue;
    if (typeof value !== "string") continue;
    env[key] = value;
  }
  return env;
}

/** The app env recorded under `root`, or `{}` when the file is absent. */
export function readAppEnv(root) {
  try {
    return parseAppEnv(readFileSync(join(root, APP_ENV_REL_PATH), "utf8"));
  } catch {
    return {};
  }
}

/** File values under the process environment: an explicit override wins. */
export function mergeAppEnv(appEnv, processEnv) {
  return { ...appEnv, ...processEnv };
}

/** The workspace root (this file lives in `<root>/scripts/`). */
export function projectRoot() {
  return dirname(dirname(fileURLToPath(import.meta.url)));
}

/**
 * Whether `moduleUrl` is the script node was asked to run.
 *
 * Both sides are resolved through symlinks: node realpaths `import.meta.url`
 * but leaves `process.argv[1]` as typed, so comparing them raw makes a CLI
 * launched through a symlinked path (`/tmp` on macOS) a silent no-op.
 */
```

```
export function isMainModule(moduleUrl) {
  const entry = process.argv[1];
  if (!entry) return false;
  try {
    return realpathSync(entry) === fileURLToPath(moduleUrl);
  } catch {
    return false;
  }
}

function main(argv) {
  const [command, ...args] = argv;
  if (!command) {
    console.error("usage: node scripts/with-app-env.mjs <command> [args…]");
    process.exit(2);
  }
  const env = mergeAppEnv(readAppEnv(projectRoot()), process.env);
  const child = spawn(command, args, { stdio: "inherit", env });
  // The dev server is long-running and is stopped by signalling this wrapper.
  for (const signal of ["SIGINT", "SIGTERM", "SIGHUP"]) {
    process.on(signal, () => child.kill(signal));
  }
  child.on("error", (err) => {
    console.error(`[with-app-env] failed to run ${command}:`, err?.message || err);
    process.exit(127);
  });
  child.on("exit", (code, signal) => {
    if (signal) {
      // Die the same way the child did, so an interrupted build is never
      // reported as success. The handler above has to go first or it catches
      // the re-raised signal and the wrapper falls through to exit 0.
      process.removeAllListeners(signal);
      process.kill(process.pid, signal);
      return;
    }
    process.exit(code ?? 1);
  });
}

if (isMainModule(import.meta.url)) {
  main(process.argv.slice(2));
}
```

scripts/migrate.mjs

```
#!/usr/bin/env node
/**
 * Deploy-time database migrator (node-postgres, `pg`).
 *
 * Runs during `npm run build` — on every Vercel deploy — applying pending files
 * in ../migrations to DATABASE_URL. Each file is applied in one transaction and
 * recorded in a `_migrations` table, so it runs once and is safe to re-run.
 *
 * The read is non-recursive, so the opt-in auth schema under migrations/auth/
 * is not applied to an app that never asked for sign-in.
 *
 * No DATABASE_URL (local / preview builds) -> skip; the PGLite fallback applies
 * the same files at startup instead (see src/lib/db.ts).
 */
import { readdir, readFile } from "node:fs/promises";
import { fileURLToPath } from "node:url";
import { dirname, join } from "node:path";
import pg from "pg";
import { pendingMigrations } from "./migration-plan.mjs";

const databaseUrl = process.env.DATABASE_URL;
if (!databaseUrl) {
  console.log(
    "[migrate] DATABASE_URL not set — skipping (the PGLite fallback migrates itself).",
  );
  process.exit(0);
}

const migrationsDir = join(dirname(fileURLToPath(import.meta.url)), "..", "migrations");

async function main() {
  let entries;
  try {
    entries = await readdir(migrationsDir);
  } catch {
    console.log("[migrate] no migrations/ directory — nothing to do.");
    return;
  }
  // An app with no schema of its own must not pay for a database connection.
  if (pendingMigrations(entries, []).length === 0) {
    console.log("[migrate] no migrations — nothing to do.");
    return;
  }

  const pool = new pg.Pool({ connectionString: databaseUrl, max: 1 });
  const client = await pool.connect();
  try {
    await client.query(
      "CREATE TABLE IF NOT EXISTS _migrations (name TEXT PRIMARY KEY, applied_at TIMESTAMPTZ NOT NULL DEFAULT now())",
    );
    const applied = (await client.query("SELECT name FROM _migrations")).rows.map(
      (r) => r.name,
    );

    let count = 0;
    for (const { name } of pendingMigrations(entries, applied)) {
      const text = await readFile(join(migrationsDir, name), "utf8");
      try {
        await client.query("BEGIN");
        // pg's simple-query protocol runs a whole multi-statement file at once.
        await client.query(text);
        await client.query("INSERT INTO _migrations (name) VALUES ($1)", [name]);
        await client.query("COMMIT");
      } catch (err) {
        console.error(`[migrate] error applying ${name}`);
        try {
          await client.query("ROLLBACK");
        } catch {
          // ROLLBACK fails when the connection died — keep the original error.
        }
        throw err;
      }
      console.log(`[migrate] applied ${name}`);
      count += 1;
    }
    console.log(count ? `[migrate] done — ${count} migration(s) applied.` : "[migrate] up to date.");
  } finally {
    client.release();
```

```
    await pool.end();
  }
}

main().catch((err) => {
  console.error("[migrate] failed:", err?.message || err);
  // pg errors carry the context needed to debug a bad SQL file.
  for (const key of ["code", "detail", "hint", "position", "where"]) {
    if (err?.[key] != null) console.error(`[migrate]   ${key}: ${err[key]}`);
  }
  process.exit(1);
});
```